队伍编号	MCB260XXXX
赛道	A

基于折外校准、风格敏感性分析与潜在高斯最优传输的啤酒风味建模

摘 要

本文基于 250 款商用啤酒的 231 个化学指标与 50 个标准化感官属性建模。经附件 8 校准数据核实，70 个共享指标中 37 列为 log10 浓度、33 列未取对数；预处理均在训练折内完成，采用 5 次重复×3 折风格分层交叉验证。

针对问题一，将其界定为输入真实缺失与原始评分尺度缺失的双重信息不完备问题。Extra Trees 预测 50 维连续感官响应，宏平均 MAE 0.3428、R^2 0.2330，相对风格均值基线 MAE 改善 4.32%（修正近似 95% 区间 [1.75%, 6.90%]）。以折外预测 P5/P95 为冻结锚点构造 26 项强度的相对评分，24 项勾选属性采用父项门控规则；按折外 R^2 将 50 维分为预测支持度 A/B/C 三档（15/21/14 项），完成 12 张评分卡并修正了模板 summary 表的公式错误。

针对问题二，以附件 2 的 Spearman 相关（$|\rho|\geq 0.6$）构建共表达网络，Louvain 划分出 19 个非单点模块；以 100 次分层 bootstrap 袋外置换重要性确定各维 Top-3，用入选率、模块命中率与 Jaccard 系数量化稳定性：3 个维度单体候选稳定、23 个维度可报告模块线索。风格内置换实验表明：乳酸在多个非酸味属性上的重要性对风格间差异敏感，在酸相关属性上仍保留稳定的风格内预测关联；4VG 对 4VG 感官的风格内重要性保留 96%。

针对问题三，以共同经验 CDF 映射在正态分数空间拟合风格条件收缩高斯，保留检出下限点质量；与首版基线同折比较，Wasserstein 距离由 1.078 降至 0.965，下限占比误差由 0.277 降至 0.168，但相关结构未优于无条件高斯。迁移采用闭式高斯最优传输加缺陷非增投影，按冻结标准 11 款 Pils/Lager 中 8 款达标、1 款未达标、2 款不可完整验收；共同达标的 8 款上 OT 平均源谱改变量 0.491（优化基线 0.699），且未出现向中心塌缩。生成与迁移结果仍属待验证的候选化学谱。

关键词：感官评分；Extra Trees；置换重要性；风格内置换；经验 CDF；Ledoit–Wolf 收缩；高斯最优传输

目录

一、问题重述.....1
1.1 问题背景.....1
1.2 题目数据.....1
1.3 需要解决的问题.....1
二、问题分析.....2
2.1 总体思路.....2
2.2 问题一的分析.....2
2.3 问题二的分析.....2
2.4 问题三的分析.....2
三、模型假设.....3
四、符号说明.....3
五、数据预处理与探索性分析.....4
5.1 数据概况.....4
5.2 对数底数与尺度核验.....5
5.3 缺失值与检出下限处理.....6
5.4 相关结构探索.....6
5.5 验证协议.....7
六、问题一：数据缺失条件下的感官评分.....7
6.1 建模思路.....7
6.2 连续感官响应预测模型.....7
6.3 评价指标.....8
6.4 模型求解与结果.....8
6.5 预测支持度分档.....10
6.6 等级映射模型.....10
6.7 评分结果.....11
6.8 小结.....13
七、问题二：化合物—风味分析.....13
7.1 建模思路.....13
7.2 化合物共表达图模型.....13
7.3 关键化合物识别模型.....14
7.4 求解结果.....15
7.5 风格混杂的敏感性分析.....17
7.6 化学合理性论证.....19
7.7 小结.....19
八、问题三：风格条件生成与跨风格迁移.....20
8.1 建模思路.....20
8.2 经验边际映射模型.....20
8.3 风格条件收缩高斯模型.....21
8.4 生成质量评价指标.....21
8.5 生成结果.....22
8.6 跨风格迁移模型.....23
8.7 迁移结果.....25
8.8 小结.....27

九、模型评价与推广.....27
9.1 模型优点.....27
9.2 模型不足.....27
9.3 模型改进与推广.....28
参考文献.....28
AI 工具使用声明.....29
附录.....29
附录 A　补充结果表.....29
附录 B　软件与源程序.....31

一、问题重述

1.1 问题背景

啤酒以麦芽、水为主要原料，添加啤酒花后经酵母发酵酿成，素有“液体面包”之称。其风味由数百种化合物共同决定：酒花带来的萜类化合物贡献草本与柑橘香，酵母代谢生成的酯类与高级醇产生水果香与溶剂感，麦芽美拉德产物带来焦糖与烘焙香，野生菌群代谢物形成乳酸、醋酸等酸味，陈化反应则生成纸板味、陈旧酒花味等老化风味。二甲基硫（DMS）、双乙酰、4-乙烯基愈创木酚（4VG）、反-2-壬烯醛（t2n）等化合物一旦超过感官阈值，便会形成煮玉米味、黄油味、丁香/药味、纸板味等缺陷。

传统的感官评价依赖经过训练的品评小组，成本高、周期长、主观差异大。如果能够由化学测量预测感官评分、识别决定各类风味的关键化合物，并在给定风格下设计化学谱，就能把“经验酿造”推进为“数据驱动的风味设计”。题目给出的商用啤酒风味代谢数据集为这一目标提供了基础。

1.2 题目数据

附件 1 为 250 款啤酒 × 231 个化学指标的丰度矩阵；附件 2 为化合物间相关矩阵；附件 3 为感官品评表模板（12 张评分卡及汇总表）；附件 4 为 250 款啤酒 50 个感官属性的标准化评分；附件 5、6 分别为感官属性间及化学—感官交叉相关矩阵；附件 7 为论坛评论描述符；附件 8 为感官阈值、属性名称、校准浓度等综合参考数据。数据真实，包含缺失值与异常值。

1.3 需要解决的问题

问题一：数据缺失条件下的感官评分。 以附件 1 化合物数据为基础，给出附件 3 定义的 Aroma 与 Flavour 两套、约 40 个感官维度的 0–6 分制分级评分。

问题二：化合物—风味分析。 利用附件 2 构建化合物共表达图，结合附件 1，给出每个感官维度的 Top-3 关键化合物，并通过 bootstrap 入选频率与不同方法间 Top-3 的 Jaccard 系数论证稳定性，同时论证化学合理性。

问题三：风格条件生成与风味指纹迁移。 以 tasting_category_fine 为风格标签学习风格条件生成模型，在 Tripel、Kriek、Lambic 等风格下生成合理的化合物谱并给出感官画像；将 Pils/Lager 的化合物谱迁移为 Tripel 风格，且保证 DMS、双乙酰、4VG、t2n、Stale hops 等缺陷不恶化。

二、问题分析

2.1 总体思路

三个问题构成一条递进链条：问题一建立“化学谱 → 感官”的前向映射，并解决评分尺度问题；问题二在这一映射上回答“哪些化合物起作用”，并用网络结构与重采样检验稳定性；问题三反过来在化学空间中进行生成与迁移，再借助问题一的前向模型得到感官画像。三问共用同一套数据预处理与验证协议，保证数字可以互相引用。

2.2 问题一的分析

问题一的难点不在于回归本身，而在于两层信息缺失。其一，附件 1 存在 73 个真实缺失值，需要在训练折内插补；其二，附件 4 给出的是标准化后的连续值（50 列中多数有 200–250 个不同取值），原始 0–6 分制评分没有提供，无法直接作为监督标签。因此本文采用“两段式”建模：先以多输出回归预测 50 维连续感官响应，用交叉验证评价其精度；再建立从连续预测到 0–6 分的映射模型，用于填写评分卡。由于回归预测必然向均值收缩，映射的锚点必须取自预测值自身的分布，而不能直接取自真实标签。此外，附件 8 表明 50 项中 26 项为强度型（numeric）、24 项为勾选型（categorical），二者在模板中的计分方式不同，需要分别建模。

2.3 问题二的分析

“关键化合物”需要同时满足三点：模型确实依赖它（重要性高）、换一批样本仍能选中它（稳定）、它不只是风格的代理（不受混杂）。本文以袋外置换重要性度量第一点，以 100 次分层 bootstrap 的入选频率和两种模型间的 Jaccard 系数度量第二点，以风格内置换实验度量第三点。化合物共表达图把高度相关的化合物组织为模块，使结论可以在“单体”与“模块”两个层次上给出，避免把共线化合物之间的偶然排序误读为化学差异。

2.4 问题三的分析

化学谱的生成有两个结构性难点：大量化合物处于检出下限（79 列的下限占比超过 10%），普通高斯分布无法产生这种“点质量”；每个风格只有 3–31 个样本，维度却高达 231，样本协方差奇异。本文先用逐列经验分布函数把数据映射到正态分数空间并保留下限点质量，再用 Ledoit–Wolf 收缩估计风格协方差。迁移问题本质上是“把一个分布搬运到另一个分布”，最优传输给出了在平方欧氏代价下最省力的搬运方式，且两个高斯分布之间存在闭式解；缺陷约束则通过投影施加。

全文技术路线如图 1 所示。

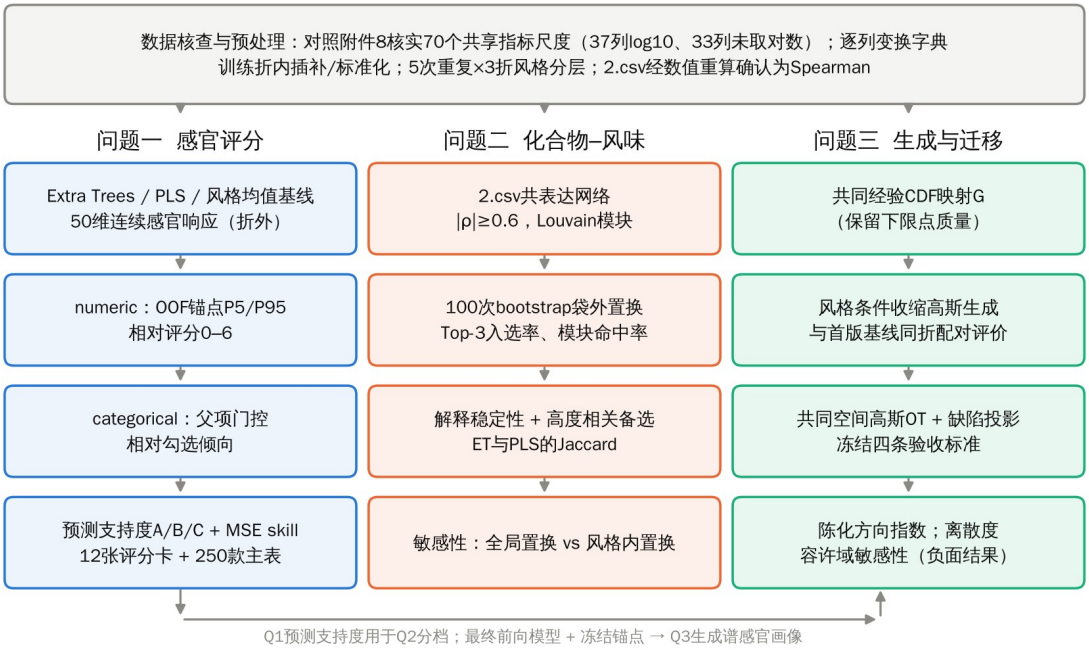


图 1 全文技术路线

三、模型假设

（1）附件 1 中同一列在不同样本间采用一致的测量与校准流程；检出下限处的重复值代表“未检出或低于下限”的状态。

（2）附件 4 的标准化感官值在样本间可比，与原始评分之间保持单调关系，可作为连续回归目标。

（3）250 款啤酒可视为同一总体的独立样本，按风格分层的交叉验证能够反映模型在同类啤酒上的泛化能力。

（4）同一风格内的化学谱经逐列经验分布变换后近似服从多元正态分布，风格间差异主要体现在均值与协方差上。

（5）缺陷“不恶化”以同一储存尺度上的浓度非增作为判据；Stale hops 无对应化学测量，以陈化方向指数作为辅助代理。

四、符号说明

符号	含义
n, p, q	样本数（250）、化学指标数（231）、感官属性数（50）
$x_i \in R^p$	第 i 款啤酒的化学谱（附件 1 储存尺度）
y_{ij}	第 i 款啤酒第 j 个感官属性的标准化值（附件 4）
c_i	第 i 款啤酒的风格标签

符号	含义
$f(\cdot)$	化学谱到感官的多输出回归模型
$\hat{z}_{ij}$	第 j 个属性的折外预测（5 次重复平均）
R_j^2, ρ_j, MAE_j	第 j 维的决定系数、Spearman 秩相关、平均绝对误差
$skill_j$	相对风格均值基线的 MSE 技能分数
$q_{0.05,j}^{OOF}, q_{0.95,j}^{OOF}$	折外预测的第 5、95 百分位（冻结锚点）
s_{ij}, b_{ij}	numeric 属性的相对评分（0–6）、categorical 属性的勾选结果（0/1）
$W=(w_{kl})$	化合物共表达图的加权邻接矩阵
Q	模块度
I_{jk}	化合物 k 对感官属性 j 的置换重要性
π_{jk}	化合物 k 在 bootstrap 中进入属性 j 的 Top-3 的频率
$J(A,B)$	集合 A,B 的 Jaccard 系数
F_k, G	第 k 列的经验分布函数、逐列共同映射 $z=G(x)$
$\mu_c, \hat{\Sigma}_c$	风格 c 在共同空间中的均值与收缩协方差
δ	Ledoit–Wolf 收缩强度
$T(\cdot), \alpha$	高斯最优传输映射、迁移强度
W_2	2-Wasserstein 距离
D	四项实测缺陷化合物的下标集合
$A(x)$	陈化方向指数
$d(\cdot,\cdot)$	共同空间中按 $\sqrt{p}$ 归一化的欧氏距离

五、数据预处理与探索性分析

5.1 数据概况

图 2 给出数据的三个基本特征。（a）22 种风格的样本数极不均衡，最多的 Blond 有 31 款，最少的 Faro 仅 3 款，因此所有交叉验证都必须按风格分层；（b）73 个缺失值集中在 9 列，其中 amyl acetate 缺失 27 个，问题三涉及的 2,3-丁二酮（双乙酰）缺失 9 个；（c）大量列存在重复的最小值，即检出下限，79 列的下限占比超过 10%，17 列超过 50%，4VG 的下限占比达 67.2%。附件 1 实际为 250 款啤酒，与题面所述 226 款不一致，本文以数据为准。

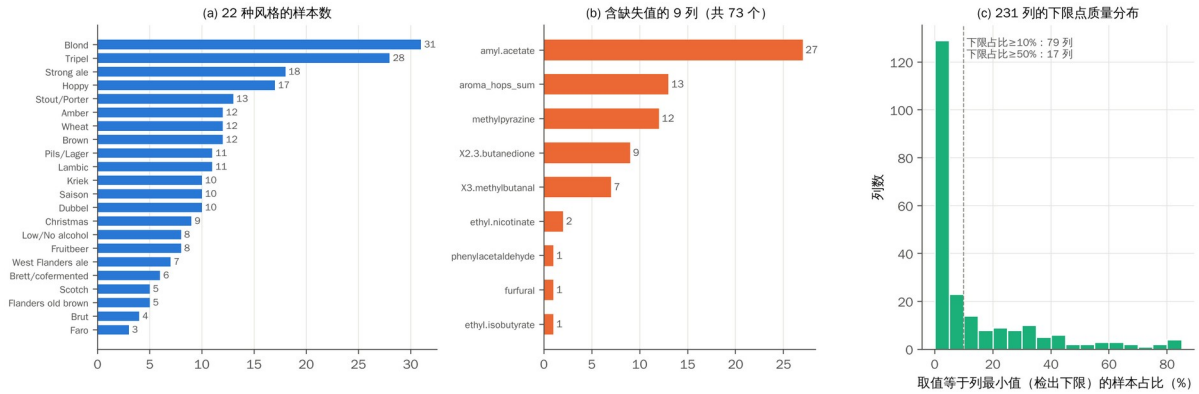


图 2 数据概况：风格样本数、缺失值与检出下限

5.2 对数底数与尺度核验

题面指出部分列经过对数变换、需自行核实底数。附件 8「Calibrated chemical dataset」给出了 70 个指标的校准浓度 c_{ik}，可与附件 1 的取值 v_{ik} 逐列比对。对第 k 列定义两种偏差：

$$e_k^{\log}=median_{i:c_{ik}>0}\left|v_{ik}-\log_{10}c_{ik}\right|,e_k^{raw}=median_i\left|v_{ik}-c_{ik}\right|.\quad(5.1)$$

若 $e_k^{\log}<10^{-3}$ 则判为 log10 列，若 $e_k^{raw}<10^{-6}$ 则判为原值列。结果为 37 列 log10、33 列原值，没有不能归类的列。log10 列满足

$$v_{ik}=\log_{10}c_{ik},v_{ik}<0\Leftrightarrow c_{ik}<1,\quad(5.2)$$

即负值对应原始浓度小于 1 的化合物。需要注意，题面称乙醇、pH、色度、苦度、CO_2 等理化参数“保留原始量纲”，但核验表明它们同样是 log10 值。图 3 给出四个典型列的核验散点，所有点都严格落在 $y=x$ 上。对未被校准数据覆盖的列，本文建立逐列变换字典，不对其尺度作额外假设；已取对数的列不再重复取对数。

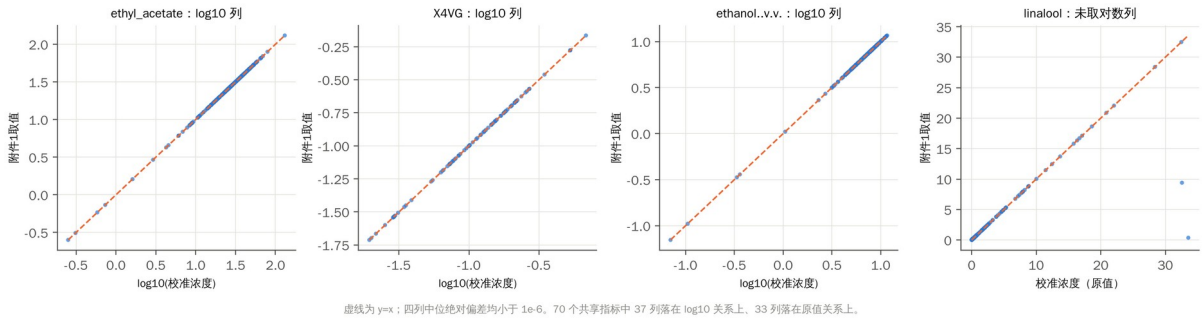


图 3 附件 1 取值与附件 8 校准浓度的逐列核验

附件 2 的相关系数同样需要核验。设附件 1 第 k,l 列的秩分别为 r_{ik},r_{il}，Spearman 相关系数为

$$\rho_{kl}=\frac{\sum_i\left(r_{ik}-\acute{r}_k\right)\left(r_{il}-\acute{r}_l\right)}{\sqrt{\sum_i\left(r_{ik}-\acute{r}_k\right)^2\sum_i\left(r_{il}-\acute{r}_l\right)^2}}.\quad(5.3)$$

将附件 2 的化合物名称映射到附件 1 字段后，用式（5.3）重算的矩阵与附件 2 最

大差小于 10^{-12}，确认附件 2 为 Spearman 相关矩阵，而非题面所说的 Pearson 相关。

5.3 缺失值与检出下限处理

记第 k 列在训练集 T 上的最小值为 $m_k = min_{i \in T} v_{ik}$，下限占比为

$$\phi_k = \frac{1}{|T|} \sum_{i \in T} 1\left(v_{ik} = m_k\right). \quad (5.4)$$

缺失值在每个训练折内用中位数插补，测试折沿用训练折的插补值：

$$\tilde{v}_{ik} = \begin{cases} v_{ik}, & v_{ik} \text{已观测} \\ median_{l \in T}\, v_{lk}, & v_{ik} \text{缺失} \end{cases} \quad (5.5)$$

本文没有对极端值做全局缩尾：缩尾会改动缺陷化合物的源测量值，使问题三的“不恶化”判据失去依据。树模型对单调变换和极端值本身不敏感；PLS 在训练折内先做分位数正态化。问题一曾检验在特征中加入下限指示变量 $1\left(v_{ik} = m_k\right)$（$\phi_k \geq 0.4$ 的列）：宏平均 MAE 由 0.342834 变为 0.343579，未达到预设的 1% 改善门槛，因此最终模型保持原特征。

5.4 相关结构探索

图 4（a）为附件 5 的 50 个感官属性间相关矩阵。同类属性（如麦芽香各子项、酒花香各子项）形成明显的正相关块，酸度、乳酸感、醋酸感三者高度相关，并与甜度、酒体负相关。图 4（b）为附件 6 中化学指标与感官属性交叉相关最强的 25 行：乙醇与酒精感（$\rho = 0.82$）、与酒体（0.72）最强，热量值次之；异 α-酸与苦感相关 0.68；甘油、辛酸乙酯、己酸乙酯也与酒精感正相关。这些边际相关为后续建模提供了基准，但它们无法区分直接作用与风格共现，需要在问题二中用模型和置换实验进一步分析。

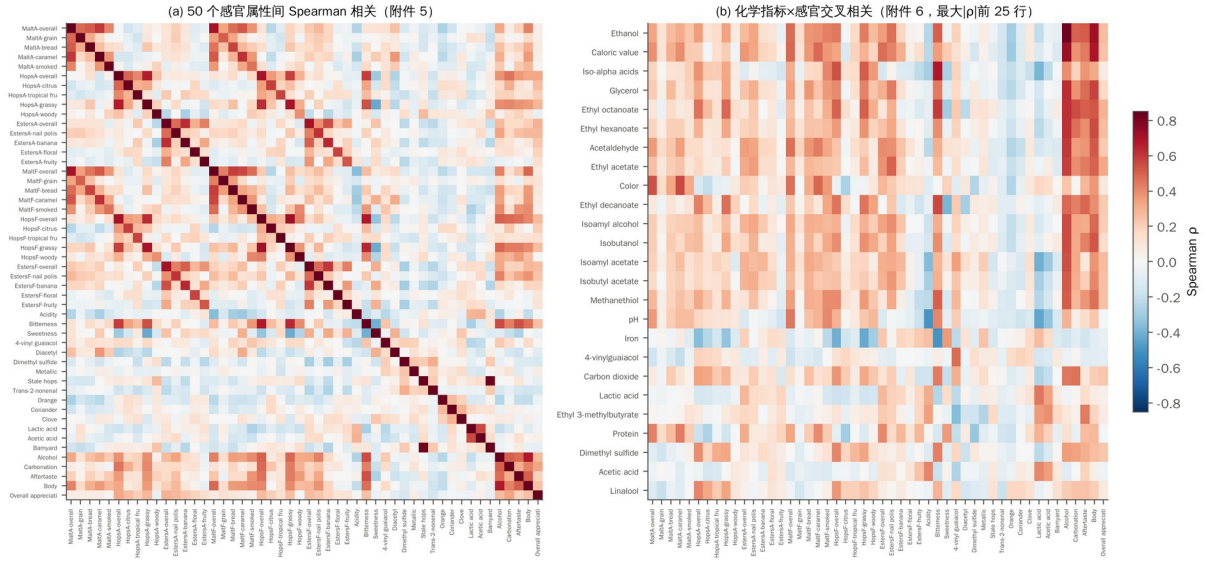


图 4 感官属性间相关与化学—感官交叉相关

5.5 验证协议

所有预测评价采用 5 次重复、每次 3 折、按风格分层的交叉验证，共 15 个划分 $\{(T_r, V_r)\}_{r=1}^{15}$。插补、标准化、分位数变换均只在 T_r 上拟合。Faro 仅 3 款，3 折恰好保证每折至少含一款。

比较两种方法 A、B 时，记第 r 个划分上的误差差为 $d_r = MAE_r^A - MAE_r^B$。由于不同划分共享训练样本，d_r 并不独立，直接使用普通 t 区间会低估方差。本文采用 Nadeau–Bengio 修正 [6]：

$$\hat{\sigma}_{corr}^2 = \left(\frac{1}{R} + \frac{n_{test}}{n_{train}}\right) s_d^2, \quad \bar{d} \pm t_{R-1,0.975}\,\hat{\sigma}_{corr}, \tag{5.6}$$

其中 $R = 15$，s_d^2 为 d_r 的样本方差，$n_{test}/n_{train} \approx 1/2$。

六、问题一：数据缺失条件下的感官评分

6.1 建模思路

问题一按“连续预测 → 等级映射 → 模板填写”三步求解。第一步以化学谱预测 50 维标准化感官值，并用交叉验证评价；第二步依据预测能力把 50 维划分为预测支持度三档；第三步把连续预测映射为 0–6 分，强度型属性用折外分位锚点，勾选型属性用父项门控规则，最后按模板公式写入评分卡。

6.2 连续感官响应预测模型

多输出回归。 设预处理后的化学谱为 $\tilde{x}_i$，目标为 $y_i = (y_{i1}, \ldots, y_{iq})^\top$，建立多输出回归

$$\hat{y}_i = f(\tilde{x}_i), \quad f = \arg\min_{f \in F} \frac{1}{|T|} \sum_{i \in T} \sum_{j=1}^{q} \left(y_{ij} - f_j(\tilde{x}_i)\right)^2. \tag{6.1}$$

Extra Trees。 主模型为极端随机树 [2]。每棵树在节点 t 上随机抽取 $\lceil 0.7\,p \rceil$ 个特征，对每个特征在其取值范围内随机产生一个切分点 s，在这些候选中选择使多输出方差下降最大的切分：

$$\Delta(t; k, s) = \sum_{j=1}^{q} \left[N_t \mathrm{Var}_t(y_j) - N_{t_L} \mathrm{Var}_{t_L}(y_j) - N_{t_R} \mathrm{Var}_{t_R}(y_j) \right], \tag{6.2}$$

其中 $t_L = \{i \in t : \tilde{x}_{ik} \le s\}$、$t_R = t \setminus t_L$，$N$ 为节点样本数。叶节点最少 3 个样本，共 $B = 160$ 棵树，预测取平均：

$$f(\tilde{x}) = \frac{1}{B} \sum_{b=1}^{B} h_b(\tilde{x}). \tag{6.3}$$

随机切分点降低了单棵树的方差，多棵树平均进一步降低方差，适合样本少、特征多、存在大量并列值的本数据。

PLS 回归。 作为线性对照，采用 8 成分偏最小二乘 [10]。设 X、Y 为中心化后的特征与目标矩阵，PLS 寻找潜变量 $T=XW$，使

$$X=TP^{\top}+E,\ Y=TC^{\top}+F,\ w_h=\arg\max_{\|w\|=1}\ \mathrm{cov}^2(Xw,Y),\tag{6.4}$$

逐个提取 8 个成分后得到 $\hat{Y}=XW(P^{\top}W)^{-1}C^{\top}$。PLS 之前在训练折内做分位数正态化。

基线。 训练折全体均值 $\hat{y}_{ij}=\acute{y}_j^{T}$；训练折风格均值 $\hat{y}_{ij}=\acute{y}_j^{T,c_i}$，即同风格训练样本的均值。后者利用了真实风格标签，是一个较强的基线：化学模型只有超过它，才说明化学谱提供了风格以外的信息。

6.3 评价指标

对第 j 维，以折外预测 $\hat{y}_{ij}$ 计算

$$MAE_j=\frac{1}{n}\sum_i\left|y_{ij}-\hat{y}_{ij}\right|,\ R_j^2=1-\frac{\sum_i\left(y_{ij}-\hat{y}_{ij}\right)^2}{\sum_i\left(y_{ij}-\acute{y}_j\right)^2},\tag{6.5}$$

Spearman 秩相关 ρ_j 按式（5.3）计算。为衡量化学模型相对风格均值的增量，定义 MSE 技能分数

$$skill_j=1-\frac{MSE_j^{ET}}{MSE_j^{style}}.\tag{6.6}$$

$skill_j>0$ 表示化学模型的平方误差小于风格均值基线。宏平均指标对 50 维等权平均，再对 5 次重复平均。

6.4 模型求解与结果

表 1 问题一连续感官预测的折外结果（5 次重复×3 折）

方法	宏平均 MAE↓	宏平均 R^2↑	宏平均 Spearman↑
训练折全体均值	0.4172	−0.0080	—
训练折风格均值	0.3584	0.1498	0.3550
PLS（8 成分）	0.3701	0.1647	—
Extra Trees	**0.3428**	**0.2330**	**0.4124**

注：PLS 为单次三折结果，其余为 5 次重复×3 折结果。

由表 1，Extra Trees 在三项指标上均最优：相较全体均值，宏平均 MAE 下降 17.84%；相较风格均值下降 4.34%，宏平均 R^2 提高 0.083。按式（5.6）对 15 个划分的配对差值作修正推断，Extra Trees 相对风格均值的平均相对改善为 4.32%，近似 95% 区间为 [1.75%, 6.90%]，区间不含 0，说明化学谱在风格之外提供了稳定的增量信息。分类型看，numeric 26 维 R^2 为 0.301，categorical 24 维为 0.159；50 维中有 43 维的 MAE、48 维的 R^2 优于风格均值。

图 5 给出逐维指标分布。预测能力在维度间差异明显：酸度（$R^2=0.721$）、酒精

感（0.676）、醋酸感（0.662）、乳酸感（0.659）、苦感（0.560）预测最好，这些属性都有明确的化学来源（有机酸、乙醇、异 α-酸）；花香风味、陈旧酒花、马厩味的 R^2 接近 0，它们由痕量化合物或复杂混合效应决定。酒精感虽然 R^2 很高，但 $skill=-0.057$，说明它的可预测性主要来自风格间差异（如高度 Tripel 与低醇啤酒），化学模型并未超过风格均值。

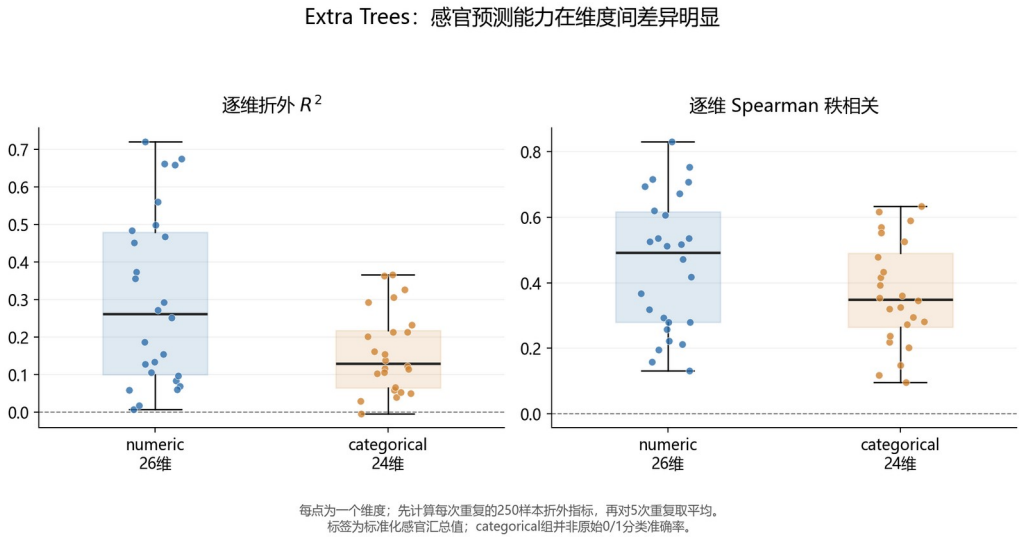


图 5 Extra Trees 逐维折外 R^2 与 Spearman 相关分布

图 6 选取不同档位的 6 个属性，比较折外预测与真实值。预测与真实值整体沿 $y=x$ 方向分布，但斜率明显小于 1，即预测值向均值收缩。这是回归模型在信噪比有限时的必然结果：最小化平方误差的预测是条件期望 $E[y/x]$，其方差为 $Var(y)-E[Var(y/x)]$，总是小于真实值的方差。这一现象直接决定了 6.6 节等级映射锚点的选取方式。

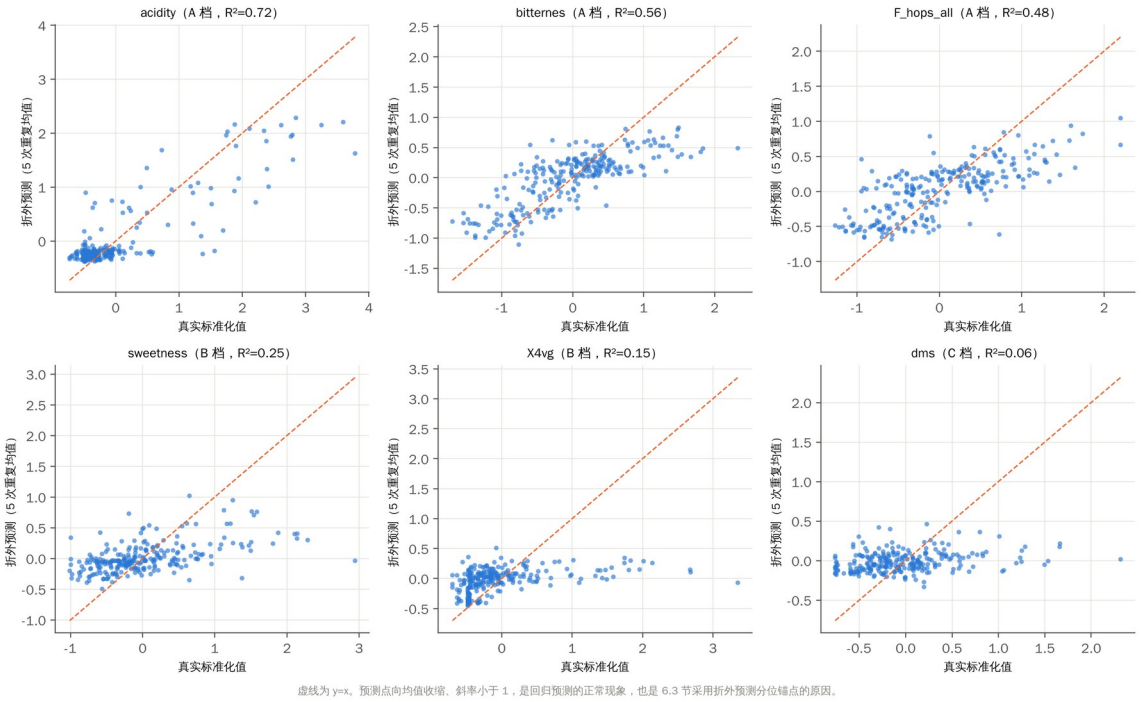


图 6 不同预测支持度属性的折外预测与真实值

缺失鲁棒性。 在自然缺失之外，再以概率 0.1 随机遮蔽已观测的特征值，即对每个 (i,k) 独立抽取 $M_{ik}\sim Bernoulli(0.1)$，令 $M_{ik}=1$ 的值缺失，再按式（5.5）插补后预测。Extra Trees 全 50 维平均 R^2 由 0.224 降至 0.207，下降 0.017；PLS 由 0.165 变为 0.165，基本不变。说明在 10% 随机缺失下，模型性能只发生小幅变化。

6.5 预测支持度分档

为了让后续交付与解释都有据可依，按 5 次重复折外平均 R_j^2 把 50 维划分为三档：

$$tier_j=\begin{cases}A, & R_j^2\geq 0.3\\ B, & 0.1\leq R_j^2<0.3\\ C, & R_j^2<0.1\end{cases}\quad(6.7)$$

同时单列 $skill_j\leq 0$ 的属性。分档结果为 A 档 15 项、B 档 21 项、C 档 14 项（numeric 11/8/7，categorical 4/13/7），见图 7（a）；酒精感、酒体位于 A 档但 $skill\leq 0$，标注“未优于风格均值”。五个缺陷感官维度中，dms、diacetyl、stale_hops 位于 C 档，X4vg、t2n 位于 B 档。A、B 档的预测可直接用于评分与解释，C 档的评分只作参考分。

6.6 等级映射模型

numeric 属性：折外分位锚点映射。 由 6.4 节，预测值向均值收缩，若以真实标签的分位点作锚点，几乎没有样本能得到高分（诊断实验中 5、6 分合计仅约 2%）。因此以预测值自身的分布为参考：取每款啤酒 5 次重复折外预测的均值

$$\hat{z}_{ij}=\frac{1}{5}\sum_{r=1}^{5}\hat{y}_{ij}^{(r)},\quad(6.8)$$

对每个 numeric 属性计算 250 个 $\hat{z}_{ij}$ 的第 5、95 百分位 $q_{0.05,j}^{OOF}$、$q_{0.95,j}^{OOF}$ 并冻结，线性映射到 0–6 并取整：

$$s_{ij}=round\left[clip\left(6\cdot\frac{\hat{z}_{ij}-q_{0.05,j}^{OOF}}{q_{0.95,j}^{OOF}-q_{0.05,j}^{OOF}},0,6\right)\right].\quad(6.9)$$

该映射对每个属性单调保序，因此不改变样本间的排序，也不影响 6.4 节的任何精度指标；它把预测分布的中间 90% 均匀铺展到 0–6 分。所有 26 个属性均有 $q_{0.05,j}^{OOF}<q_{0.95,j}^{OOF}$。12 张评分卡、250 款主表以及问题三生成谱均使用同一组锚点。

categorical 属性：父项门控勾选。 模板中每个子项（如“麦芽香—谷物”）只能勾选或不勾选，其展示分等于父项强度乘以勾选结果。附件 4 的勾选型属性是多人勾选汇总后的标准化值，可理解为“勾选倾向”。本文以训练折均值为参考，定义勾选规则

$$b_{ij}=1\left(s_{i,pa(j)}>0\right)\cdot 1\left(\hat{z}_{ij}>\acute{y}_{ij}^{T}\right),\quad(6.10)$$

其中 $pa(j)$ 为父项（如 A_malt_grain 的父项为 A_malt_all），$\acute{y}_{ij}^{T}$ 为第 i 款啤酒作为测试样本时 5 个对应训练折真实值的平均，保证规则严格折外。第一个因子保证“父项强度为 0 时子项不勾选”，与模板逻辑一致。模板展示值为

$$disp_{ij}=s_{i,pa(j)}\cdot b_{ij}.\quad(6.11)$$

6.7 评分结果

图 7（b）为 26 项 numeric 相对评分在 250 款中的分布：0 分 18.5%，1–5 分各占 12%–16%，6 分 9.6%，分布均匀，评分卡有良好的区分度。categorical 属性经父项门控后，各属性最终勾选率为 29.2%–58.4%，中位数 44.2%。

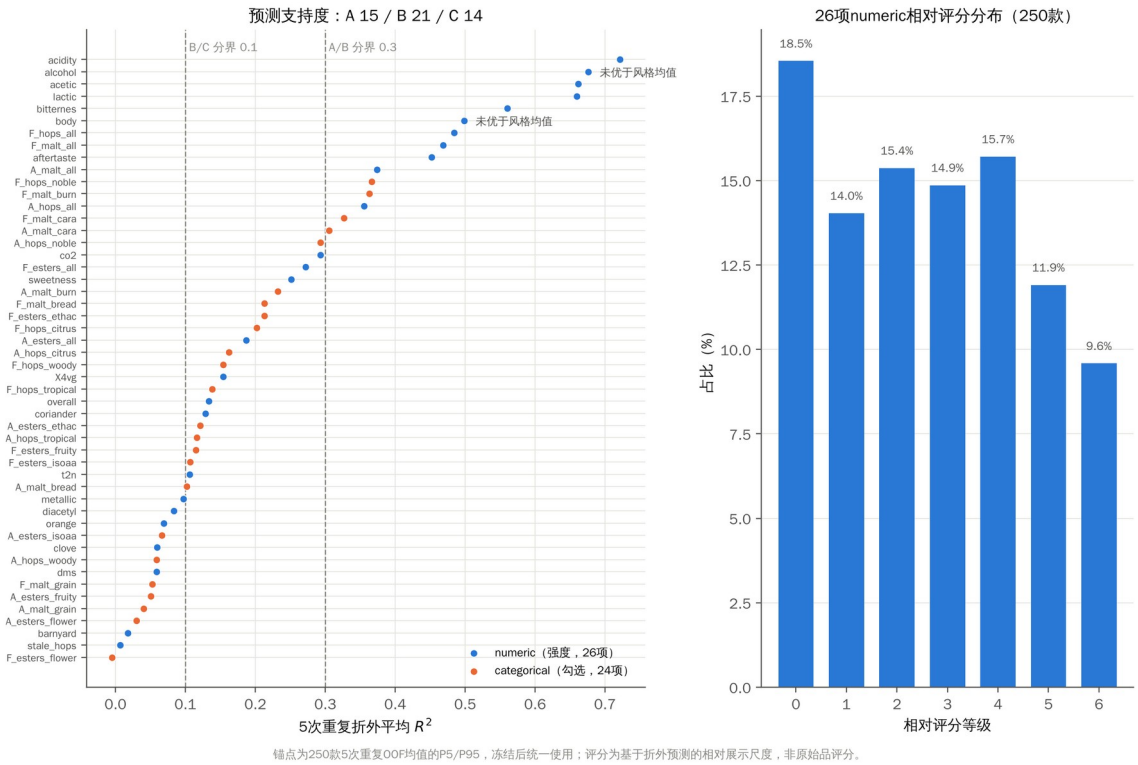


图 7 预测支持度分档与相对评分分布

附件 3 实际包含 beer_1–beer_12 共 12 张评分卡及 summary 汇总表。本文按每张评分卡的 Beer# 单元格与样本编号匹配，在 0–6 对应格标记强度、在勾选格标记子项，由模板原有公式计算分数。核对时发现模板 summary 表 barnyard 列（AX 列）误引用了 Stale hops 单元格 $AG$76，本文在填写副本中改为 $AG$90，重算后 summary 表与程序计算值逐格一致。评分卡中的 Acetaldehyde 一行在附件 4 中没有对应属性，予以留空。

图 8 为 12 款啤酒 26 项 numeric 的评分。结果与各风格的公认特征高度一致：

- Lambic（beer 12）的乳酸感 4 分、酸度 3 分、醋酸感 3 分，在 12 款中最高，苦感 0 分、酒花香 0 分，符合自然发酵酸啤“高酸、低苦、无酒花香”的特征；
- 两款低/无醇啤酒（beer 2、10）的酒精感、酒体、苦感均为 0，橙皮评分 5–6 分，符合其加入果味与香料调味的常见做法；
- Strong ale（beer 4）的酯香 6 分、麦芽风味 6 分、酒精感 6 分、总体评价 6 分，体现了烈性艾尔“浓郁、醇厚、高酒精”的特点；
- 两款 Hoppy（beer 8、9）的酒花香 4–5 分、苦感 5 分，均处于高位。

带“*”的为 C 档属性，仅作参考分。

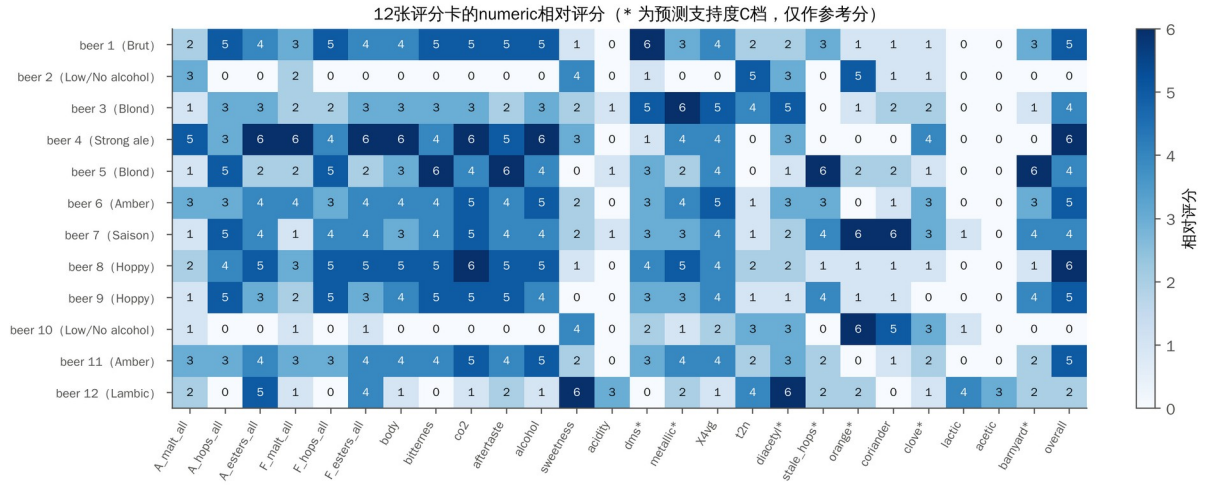


图 8 12 张评分卡的 numeric 相对评分

6.8 小结

问题一建立了“Extra Trees 连续预测 + 折外分位锚点映射 + 父项门控勾选”的评分模型。连续预测在 15 个划分上稳定优于风格均值基线（相对改善 4.32%，区间 [1.75%, 6.90%]），酸度、苦感、乳酸感等有明确化学来源的属性预测最好；等级映射保序且锚点冻结，使评分卡既有区分度又不改变模型精度；预测支持度三档为问题二、三的解释提供了统一依据。

七、问题二：化合物—风味分析

7.1 建模思路

问题二分四步：（1）由附件 2 构建化合物共表达图并划分模块；（2）以 bootstrap 袋外置换重要性给出每个感官维度的 Top-3 候选及其入选频率；（3）结合问题一的预测支持度与模块信息，对 50 维的结论进行两维分档；（4）以风格内置换检验 Top-3 是否只是风格代理，并结合化学知识论证合理性。

7.2 化合物共表达图模型

以化合物为节点，以附件 2 的 Spearman 相关构建加权无向图：

$$w_{kl}=\begin{cases}|\rho_{kl}|, & |\rho_{kl}|\geq\tau,k\neq l\\ 0, & \text{其他}\end{cases},\tau=0.6.\quad(7.1)$$

阈值 0.6 对应“强相关”，既保留主要的共表达关系，又避免图过于稠密。在该图上以 Louvain 算法 [5] 最大化模块度

$$Q=\frac{1}{2m}\sum_{k,l}\left(w_{kl}-\frac{d_k d_l}{2m}\right)1(g_k=g_l),\quad(7.2)$$

其中 $d_k=\sum_l w_{kl}$ 为节点加权度，$m=\frac{1}{2}\sum_{k,l} w_{kl}$，$g_k$ 为节点 k 所属模块。Louvain 交替

执行“节点局部移动以增大 Q”与“模块合并为超节点”两步，直至 Q 不再增加；随机种子固定以保证可复现。

得到的网络有 266 条边、130 个非孤立节点，划分出 19 个非单点模块，最大模块含 23 个指标，另有 101 个孤立节点（图 9）。模块与化学来源基本对应：酵母酯类与高级醇、酒花萜类与倍半萜、麦芽美拉德产物、野生菌群代谢物各自聚集。去除 14 个理化、派生或混合指标（如 pH、色度、kcalperc、esters_sum）后，217 个单体化学指标作为 Top-3 的候选集合 K。

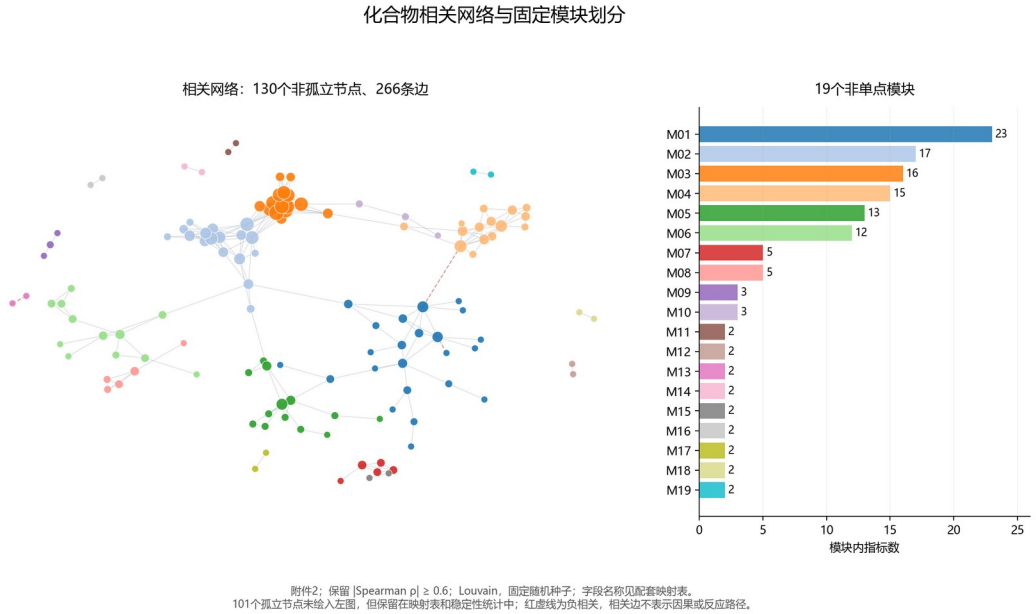


图 9 化合物共表达网络与模块划分

7.3 关键化合物识别模型

置换重要性。 对已训练的模型 f 与评价集 V，把第 k 列在 V 内随机打乱得到 $\widetilde{X}^{(k)}$，第 j 个属性的重要性定义为误差的增加量 [3]：

$$I_{jk}=\frac{1}{L}\sum_{\ell=1}^{L} MAE_j\left(f;\widetilde{X}_V^{(k,\ell)}\right)-MAE_j\left(f;X_V\right),L=3.\quad(7.3)$$

打乱切断了化合物 k 与感官之间的联系而保持其边际分布不变，因此 I_{jk} 直接度量模型对化合物 k 的依赖。与基于不纯度的重要性相比，它不偏向取值多的特征，也可同样用于 PLS。

分层 bootstrap。 第 b 次（$b=1,\dots,100$）在每种风格内有放回抽取与原风格等量的样本作为训练集 T_b，未被抽中的样本为袋外集 O_b（平均约 88 款）。在 T_b 上训练 Extra Trees 与 PLS，在 O_b 上按式（7.3）计算 $I_{jk}^{(b)}$。最终名单按平均重要性取前三：

$$Top3_j=arg\,top3_{k\in K}\acute{I}_{jk},\acute{I}_{jk}=\frac{1}{100}\sum_{b=1}^{100} I_{jk}^{(b)}.\quad(7.4)$$

入选频率与 Wilson 区间。 记 $Top3_j^{(b)}$ 为第 b 次的前三名，化合物 k 的入选频率为

$$\pi_{jk}=\frac{1}{100}\sum_{b=1}^{100}\mathbb{1}\left(k\in Top3_j^{(b)}\right).\tag{7.5}$$

为描述 100 次重采样的 Monte Carlo 精度，给出 Wilson 区间

$$\frac{\pi+\frac{z^2}{2N}\pm z\sqrt{\frac{\pi(1-\pi)}{N}+\frac{z^2}{4N^2}}}{1+\frac{z^2}{N}},\quad N=100,\ z=1.96.\tag{7.6}$$

模块命中率。 第 b 次的前三名中至少有一个属于模块 g，则记一次命中：

$$\eta_{jg}=\frac{1}{100}\sum_{b=1}^{100}\mathbb{1}\left(\exists\, k\in Top3_j^{(b)}:\ g_k=g\right).\tag{7.7}$$

方法间一致性。 Extra Trees 与 PLS 的 Top-k 名单 A、B 之间的 Jaccard 系数为

$$J(A,B)=\frac{|A\cap B|}{|A\cup B|}.\tag{7.8}$$

把名单中的化合物替换为所属模块后再计算，即得到模块层面的 Jaccard 系数。

7.4 求解结果

图 10 给出 50 个感官维度的 Top-3 及其入选率和模块命中率。入选率最高的组合包括：酸度、乳酸感、醋酸感 ← 乳酸（均为 100%）、酒精感 ← 乙醇（93%）、酒体 ← 乙醇（94%）、F_malt_all ← 乳酸（99%）、苦感 ← 辛酸乙酯（91%）。

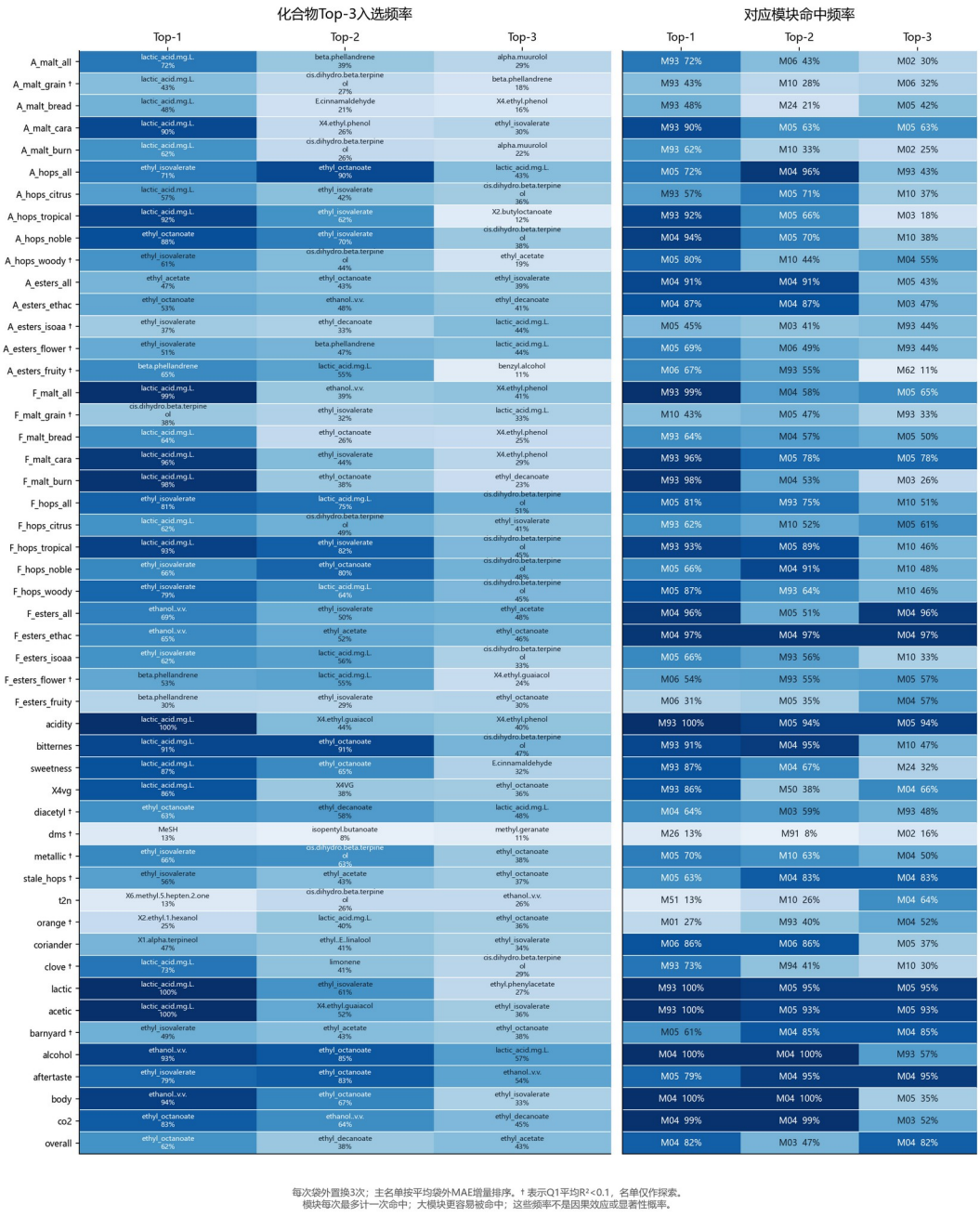


图 10 Top-3 入选率与模块命中率（100 次分层 bootstrap）

图 11（b）比较两种模型的名单。Top-3 层面化合物 Jaccard 中位数为 0.000，模块层面为 0.200；Top-10 时分别为 0.111 与 0.223。树模型与线性模型在单体层面很少一致但在模块层面的一致性明显更高。原因在于同一模块内的化合物高度相关，两种模型可能各自选中其中不同的成员。因此，在模块层面报告结论比在单体层面更稳健。

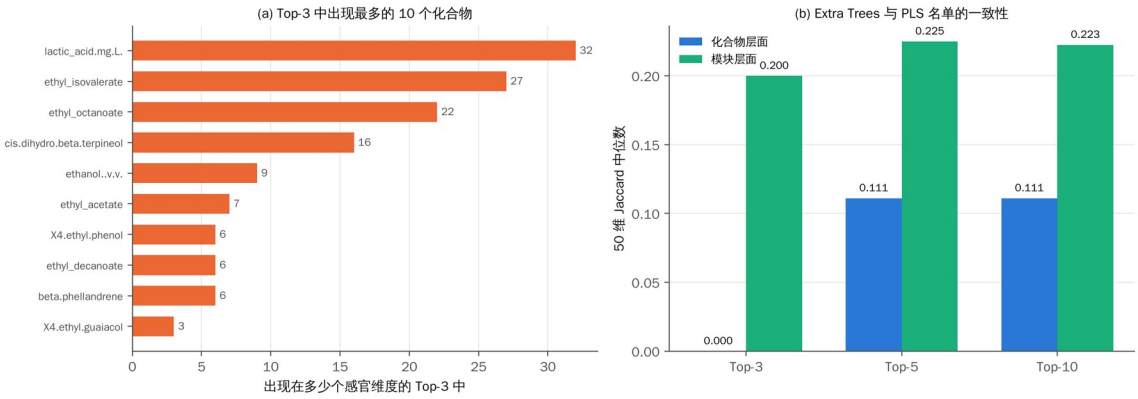


图 11 Top-3 中高频化合物与两种模型名单的一致性

两维分档。 结合问题一的预测支持度（式 6.7）与本节的稳定性信息，定义解释稳定性：

$$stab_j=\begin{cases}\text{单体稳定}, & \min\limits_{k\in Top3_j}\pi_{jk}\geq 0.5 \text{ 且 } \min\limits_{k\in Top3_j}\acute{I}_{jk}>0\\ \text{模块线索}, & \text{否则，若 } \max\limits_{g:|g|\geq 2}\eta_{jg}\geq 0.7\\ \text{探索性}, & \text{其他}\end{cases} \tag{7.9}$$

对每个 Top-3 候选，另列出同模块内 $|\rho|\geq 0.8$ 的“高度相关备选”（至多 3 个），如 4-乙基苯酚 ↔ 4-乙基愈创木酚（+0.81）；理化与派生指标（如乙醇 ↔ kcalperc，+0.85）单列为“派生相关指标”。

表 2 预测支持度与解释稳定性的交叉分布（维度数）

预测支持度	单体稳定	模块线索	探索性	合计
A	3	8	4	15
B	0	11	10	21
C	0	4	10	14

由表 2，单体稳定的 3 个维度全部来自 A 档，说明“可预测”与“可解释”高度一致：F_hops_all（异戊酸乙酯、乳酸、cis-二氢-β-萜品醇）、aftertaste（异戊酸乙酯、辛酸乙酯、乙醇）、alcohol（乙醇、辛酸乙酯、乳酸）。其中 alcohol 未优于风格均值，其 Top-3 主要反映风格差异。另有 23 个维度可以在模块层面给出稳定结论。C 档 14 个维度的 Top-3 只作为探索线索。各维完整名单见附录表 A1。

7.5 风格混杂的敏感性分析

图 11（a）显示 Top-3 高度集中：乳酸出现在 32 个维度的 Top-3 中，异戊酸乙酯 27 个，辛酸乙酯 22 个，cis-二氢-β-萜品醇 16 个。乳酸是 Lambic、Kriek 等自然发酵酸啤的标志物，这提示部分重要性可能来自“酸啤与非酸啤”的风格差异，而不是乳酸对这些感官属性的直接作用。

风格内置换。 为区分两者，把式（7.3）中的全局置换改为只在同一风格内部打乱：

$$I_{jk}^{within}=\frac{1}{L}\sum_{\ell=1}^{L}MAE_j\left(f;\widetilde{X}_{V/c}^{(k,\ell)}\right)-MAE_j\left(f;X_V\right), \tag{7.10}$$

其中 $\widetilde{X}_{V/c}$ 表示第 k 列只在 $\{i \in V : c_i = c\}$ 内随机排列。风格内置换保留了化合物 k 与风格之间的关联，只破坏风格内部的变异 [9]。若化合物的作用完全来自风格间差异，则 $I_{jk}^{within} \approx 0$；若在同风格内仍有预测作用，则 $I_{jk}^{within} > 0$。定义保留比例

$$\kappa_{jk} = \frac{I_{jk}^{within}}{I_{jk}^{glob}}. \quad (7.11)$$

由于风格内变异本就小于全局变异，κ 天然小于 1，因此同时考察风格内排名。

实验沿用问题一的 15 个划分与相同的 Extra Trees 配置重新拟合，重拟合预测与原折外预测逐值一致（最大差 7×10^{-16}）。在同一测试折、同一模型上配对比较两种置换评价集为测试折中同风格至少 2 款的样本（1215/1250）。

表 3 原 Top-3 候选在风格内置换下的变化（150 条）

指标	结果
全局置换重要性 > 0	99.3%
风格内置换重要性 > 0	83.3%
保留比例 κ 中位数	0.187
风格内仍在 217 个候选的 Top-10	72%
风格内仍在 Top-3	54%

表 4 高频化合物的风格内置换结果

化合物	出现维度数	κ 中位数	风格内 >0 比例	风格内仍在 Top-10
乳酸	32	0.020	59.4%	43.8%
异戊酸乙酯	27	0.244	92.6%	85.2%
辛酸乙酯	22	0.236	86.4%	81.8%
cis-二氢-β-萜品醇	16	0.258	93.8%	87.5%
乙醇	9	0.131	66.7%	55.6%
乙酸乙酯	7	0.206	100%	85.7%

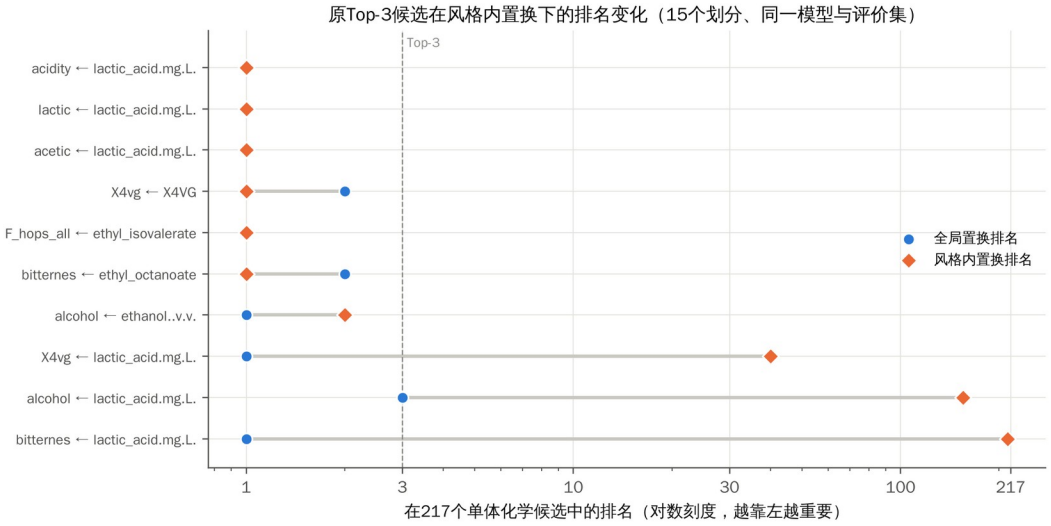


图 12 典型候选在全局置换与风格内置换下的排名

结合表 3、表 4 与图 12，得到三点结论：

（1）**乳酸在多个非酸味属性上的重要性对风格间差异敏感，而在酸相关属性上仍**

保留稳定的风格内预测关联。 对苦感、酒精感、4VG 感官，乳酸的风格内排名分别降至第 212、155、40 位；对酸度、乳酸感、醋酸感，乳酸在风格内仍排第 1，15 个划分中有 13–14 个为正。这与乳酸菌产酸的机理一致。

（2）**4VG 是最清晰的化合物—感官对应。** 4VG 对 X4vg 感官维度的风格内保留比例为 96%，风格内排名由全局第 2 升至第 1。4VG 由酵母的酚酸脱羧酶作用于阿魏酸生成，是小麦啤、Saison 丁香味的直接来源，其作用不依赖风格。

（3）**酯类与酒花萜类候选在风格内仍然有效。** 异戊酸乙酯、辛酸乙酯、cis-二氢-β-萜品醇的保留比例约 0.24–0.26，风格内仍在 Top-10 的比例为 82%–88%，说明它们对酒花香、苦感、余味等属性的作用并不只是风格代理。

7.6 化学合理性论证

表 5 主要关键化合物的来源与作用机理

化合物	来源（附件 8）	主要关联感官	机理
乳酸	野生菌群	酸度、乳酸感、醋酸感	乳酸菌发酵产酸，降低 pH，形成柔和酸味
4-乙烯基愈创木酚	酵母	4VG（丁香、烟熏）	酚酸脱羧酶将阿魏酸转化为 4VG
乙醇	酵母	酒精感、酒体、余味	糖酵解终产物，提供温热感与饱满度
异戊酸乙酯	其他（陈化、酒花降解）	酒花香、余味、马厩味	异戊酸（亮氨酸代谢或酒花 α-酸降解产物）与乙醇酯化，带水果香，随陈化增加
辛酸乙酯、癸酸乙酯	酵母	苦感、余味、酯香	中链脂肪酸乙酯，赋予苹果、蜡质香，与高发酵度共现
乙酸乙酯	酵母	酯香、溶剂感	醇酰基转移酶生成，高浓度时呈指甲油味
cis-二氢-β-萜品醇、β-水芹烯	酒花	酒花香与风味	酒花精油萜类及其转化产物
4-乙基苯酚/4-乙基愈创木酚	野生菌群	酸度、麦芽风味	酒香酵母（Brettanomyces）还原乙烯基酚生成，带马厩、烟熏味
甲硫醇（MeSH）	酵母	DMS 维度（C 档）	含硫化合物，与 DMS 同属煮蔬菜类异味

表 5 中的对应关系与酿造化学的公认知识一致，说明模型识别出的不是统计偶然。对 DMS、双乙酰、陈旧酒花等 C 档维度，Top-3 只能作为线索：它们由痕量化合物决定，且感官评分本身噪声较大。

7.7 小结

问题二构建了 19 个非单点模块的化合物共表达网络，以 100 次分层 bootstrap 袋外置换重要性给出 50 维的 Top-3，并用入选率、模块命中率、Jaccard 系数和风格内置换

四个层次检验稳定性。结果表明：3 个维度的单体候选稳定，23 个维度可给出模块层面的稳定结论；乳酸的普遍入选很大程度上源于风格差异，但它在酸味属性上的作用在风格内依然成立；4VG 与 4VG 感官的对应最清晰。

八、问题三：风格条件生成与跨风格迁移

8.1 建模思路

问题三的核心是在化学空间中建立风格条件分布 $p(x\mid c)$。本文先构造逐列经验分布映射 G，把化学谱变换到正态分数空间并保留检出下限的点质量；在该空间中对每个风格拟合 Ledoit–Wolf 收缩高斯，采样后经 G^{-1} 回到原尺度即得到生成谱。迁移时，在同一空间中求源风格与目标风格两个高斯之间的最优传输映射，沿传输路径移动源谱，再对缺陷化合物施加非增投影，并按运行前冻结的四条标准验收。

8.2 经验边际映射模型

对插补后的训练数据逐列构造经验分布。设第 k 列不同取值为 $v_1<v_2<\dots<v_M$，对应样本数 $n_1,\dots,n_M$，记累计比例

$$b_l=\frac{1}{n}\sum_{r\le l} n_r,\; a_l=b_l-\frac{n_l}{n},\; l=1,\dots,M. \tag{8.1}$$

正向映射。 并列值取中间秩，再经标准正态分位函数变换：

$$F_k(v_l)=\frac{a_l+b_l}{2},\; z_k=G_k(v)=\Phi^{-1}\big(F_k(v)\big), \tag{8.2}$$

两个取值之间线性插值，超出训练范围时截断到端点，从而不会出现无穷大的正态分数。

逆映射。 对并列块（$n_l\ge 2$）把整个概率区间映射为同一个值，对单点在相邻点之间线性插值：

$$G_k^{-1}(z)=\begin{cases} v_l, & \Phi(z)\in(a_l,b_l),\, n_l\ge 2 \\ \text{相邻单点间线性插值}, & \text{其他} \end{cases} \tag{8.3}$$

点质量保持性质。 若潜在变量 $z_k\sim N(\mu,\sigma^2)$，则生成值等于下限 v_1 的概率为

$$P\big(G_k^{-1}(z_k)=v_1\big)=P\big(\Phi(z_k)<b_1\big)=\Phi\left(\frac{\Phi^{-1}(b_1)-\mu}{\sigma}\right). \tag{8.4}$$

当某风格在潜在空间中的均值与方差贴合其真实分布时，生成样本的下限占比也就贴合真实下限占比。普通高斯生成器在原尺度上是连续分布，生成值恰好等于下限的概率为 0，只能靠截断产生下限值，这正是首版基线下限占比严重偏低的原因。G 由全体风格共同的训练数据构建，生成与迁移在同一坐标中进行，风格之间的距离可以直接比较。

8.3 风格条件收缩高斯模型

在共同空间中，风格 c 的样本 $\{z_i : c_i = c\}$ 只有 n_c 个（Tripel 28、Kriek 10、Lambic 11、Pils/Lager 11），而 $p = 231$，样本协方差 S_c 奇异。采用 Ledoit–Wolf 收缩估计 [4]：

$$\hat{\Sigma}_c = (1 - \delta_c) S_c + \delta_c \nu_c I_p, \quad \nu_c = \frac{\operatorname{tr} S_c}{p}, \tag{8.5}$$

收缩强度由最小化期望 Frobenius 损失 $\mathbb{E}\,\| \hat{\Sigma}_c - \Sigma_c \|_F^2$ 给出：

$$\delta_c = \min\left(1, \frac{\hat{\beta}_c^2}{\| S_c - \nu_c I_p \|_F^2}\right), \quad \hat{\beta}_c^2 = \frac{1}{n_c^2} \sum_{i : c_i = c} \left\| (z_i - \bar{z}_c)(z_i - \bar{z}_c)^{\top} - S_c \right\|_F^2 . \tag{8.6}$$

$\hat{\Sigma}_c$ 恒为正定。生成时采样 $z \sim N(\mu_c, \hat{\Sigma}_c)$，即 $z = \mu_c + \hat{\Sigma}_c^{1/2} \varepsilon$、$\varepsilon \sim N(0, I_p)$，再令 $x = G^{-1}(z)$。由式（8.3），生成值自动落在训练观测范围内，无需额外截断。

全数据拟合时的收缩强度为 Tripel 0.42、Kriek 0.66、Lambic 0.68，样本越少收缩越强；收缩后相关矩阵非对角元素的平方和占总平方和的 72%–87%，模型保留了相当的相关结构。

8.4 生成质量评价指标

沿用首版基线的三折风格分层划分与相同随机数，在首版的评价坐标（训练折中位数插补加标准化）中比较三种生成器：本文潜在高斯、首版储存尺度风格高斯（原尺度拟合高斯并截断到观测范围）、无条件高斯。对每个“折×风格”单元生成 $N_g = 300$ 条，与留出真实样本 $\{u_i\}_{i=1}^{N_r}$ 比较以下指标（$\tilde{z}$ 为生成样本）：

$$\mathrm{ME} = \frac{1}{p} \sum_{k=1}^{p} \left| \bar{\tilde{z}}_k - \bar{u}_k \right|, \quad W_1 = \frac{1}{p} \sum_{k=1}^{p} \int_0^1 \left| \tilde{F}_k^{-1}(t) - F_{u,k}^{-1}(t) \right| dt , \tag{8.7}$$

$$\mathrm{FE} = \frac{1}{|L|} \sum_{k \in L} \left| \hat{\phi}_k^{gen} - \hat{\phi}_k^{real} \right|, \quad \mathrm{CR} = \sqrt{\frac{2}{p(p-1)} \sum_{k<l} \left(\tilde{r}_{kl} - r_{kl} \right)^2 }, \tag{8.8}$$

其中 W_1 为逐坐标一维 Wasserstein 距离的平均，L 为训练下限占比不低于 10% 的列，$\hat{\phi}$ 为下限占比，CR 为去除风格中心后合并计算的相关矩阵均方根误差。另以最近邻距离比

$$\mathrm{NNR} = \frac{\dfrac{1}{N_g} \sum_{i} \min_{l \in T_c} \| \tilde{z}_i - z_l \|}{\dfrac{1}{N_r} \sum_{i} \min_{l \in T_c} \| u_i - z_l \|} \tag{8.9}$$

衡量生成样本相对训练样本的贴近程度：显著小于 1 意味着过度贴近训练集，显著大于 1 意味着偏离数据流形。

8.5 生成结果

表 6 生成器留出保真比较（9 个“折×风格”单元等权平均，越小越好）

指标	潜在高斯（本文）	首版风格高斯	无条件高斯
逐坐标均值误差 ME	**0.884**	0.917	1.059
一维 Wasserstein W_1	**0.965**	1.078	1.235
下限占比误差 FE	**0.168**	0.277	0.305
相关矩阵 RMSE CR	0.316	0.323	**0.303**
最近邻距离比 NNR	0.870	0.829	1.078

由表 6 与图 13，潜在高斯在均值、边际分布与下限点质量三项上均优于首版：W_1 下降 10.5%，下限占比误差下降 39.4%。全数据拟合时，4VG 的下限占比在 Tripel 中实测 82.1%、生成 80.7%（首版仅 33.7%），Kriek 实测 100%、生成 91.3%（首版 54.7%），Lambic 实测 90.9%、生成 92.3%（首版 45.7%），与式（8.4）的理论预期一致。相关结构方面，三种模型差异较小，无条件高斯略优，说明在小样本下联合结构的估计仍有改进空间。生成样本中精确复制训练样本的数量为 0，NNR 为 0.870，没有出现对训练样本的复制。

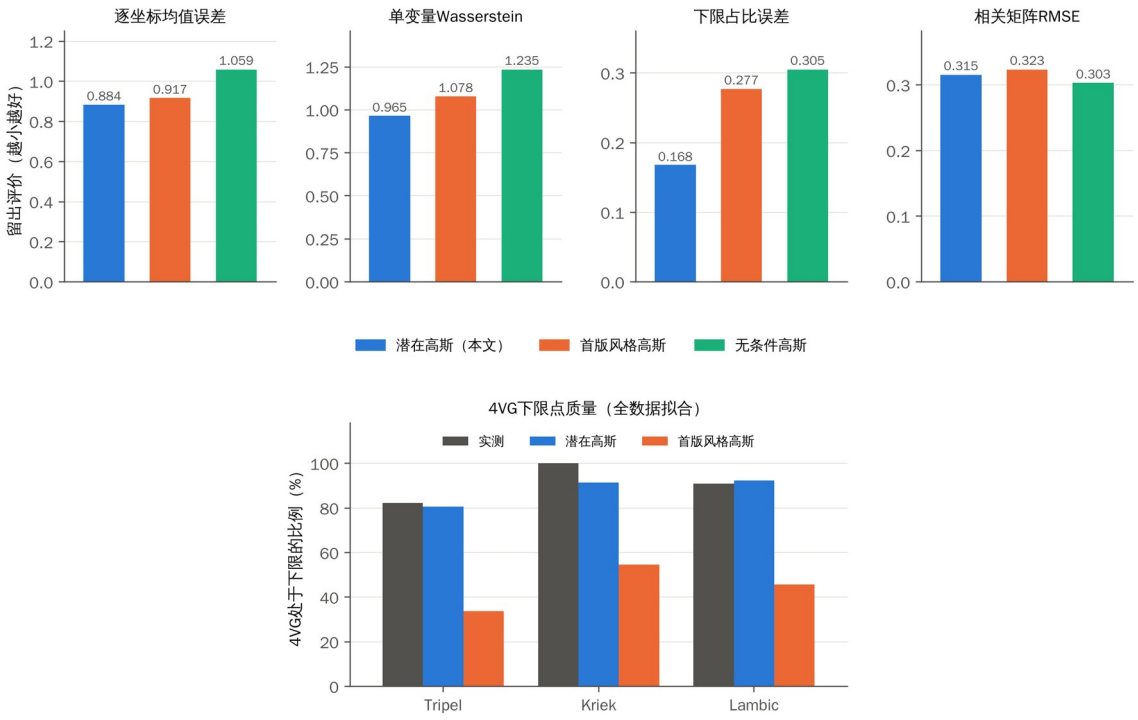


图 13 生成器留出保真与 4VG 下限点质量

图 14（a）把实测样本与 900 条生成谱投影到共同空间的前两个主成分上（解释方差 15.4%、10.5%）。Tripel 生成谱覆盖了真实 Tripel 所在区域；Kriek 与 Lambic 生成谱集中在第二主成分负方向，与两种酸啤的真实位置重合，并与 Tripel 明显分开。

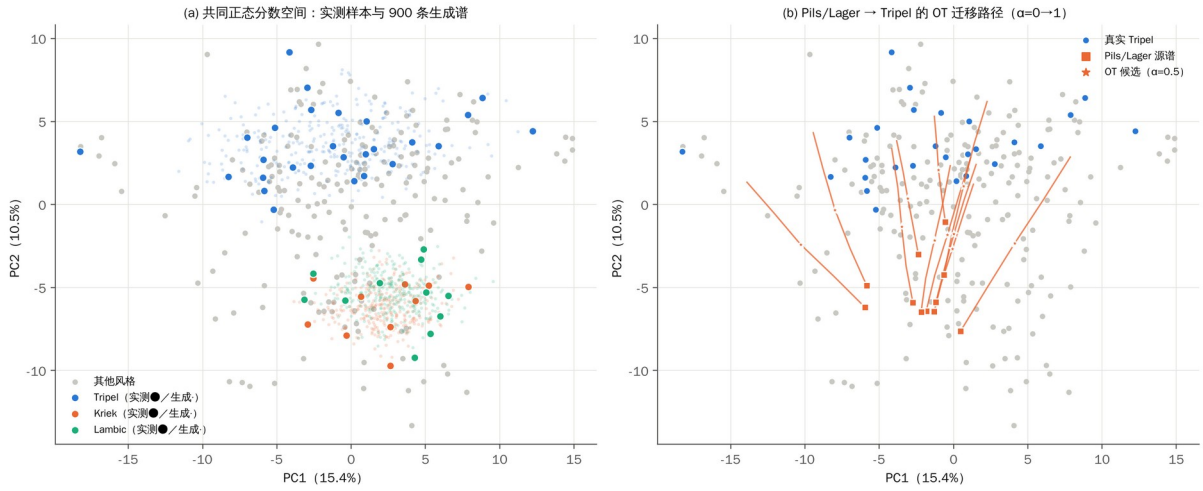


图 14 共同正态分数空间中的生成谱与迁移路径（主成分投影）

感官画像。 以问题一在全体样本上拟合的前向模型预测生成谱的感官值，并按式（6.9）的冻结锚点换算评分。图 15 比较 A、B 档 numeric 属性的生成谱预测均值与实测风格均值：三种风格的方向全部一致。Kriek、Lambic 的酸度为 +1.29、+1.41（实测 +1.86、+1.79），乳酸感 +1.04、+1.17，苦感 −0.70、−0.57，呈现“高酸、低苦”的酸啤画像；Tripel 的酒精感 +0.28（实测 +0.41）、苦感 +0.17、酒花风味 +0.17，呈现“高酒精、中等苦度、果香酯香突出”的画像。预测幅度普遍小于实测，来自前向模型的回归收缩。

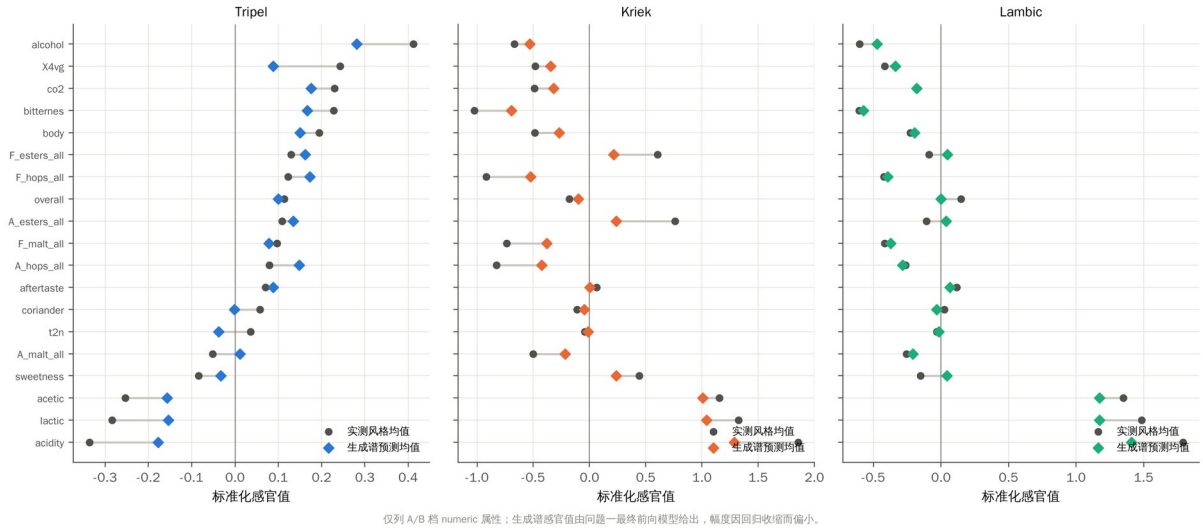


图 15 生成谱感官画像与实测风格均值对比

8.6 跨风格迁移模型

高斯最优传输。 在共同空间中，源风格（Pils/Lager）与目标风格（Tripel）分别为 $N(\mu_s,\Sigma_s)$ 与 $N(\mu_t,\Sigma_t)$。在平方欧氏代价下，最优传输问题

$$\min_{T:T_\#N(\mu_s,\Sigma_s)=N(\mu_t,\Sigma_t)} E\left\|z-T(z)\right\|^2 \quad (8.10)$$

的最优值即 2-Wasserstein 距离的平方，两个高斯之间有闭式解 [7]：

$$W_2^2=\left\|\mu_s-\mu_t\right\|^2+tr\left(\Sigma_s+\Sigma_t-2\left(\Sigma_s^{1/2}\Sigma_t\Sigma_s^{1/2}\right)^{1/2}\right), \quad (8.11)$$

对应的最优映射为仿射映射

$$T(z)=\mu_t+A(z-\mu_s),\ A=\Sigma_s^{-1/2}\left(\Sigma_s^{1/2}\Sigma_t\Sigma_s^{1/2}\right)^{1/2}\Sigma_s^{-1/2}. \tag{8.12}$$

验证。 A 对称正定，若 $z\sim N(\mu_s,\Sigma_s)$，则 $T(z)$ 服从均值为 μ_t、协方差为 $A\Sigma_sA$ 的高斯分布，而

$$A\Sigma_sA=\Sigma_s^{-1/2}\left(\Sigma_s^{1/2}\Sigma_t\Sigma_s^{1/2}\right)^{1/2}\left(\Sigma_s^{1/2}\Sigma_t\Sigma_s^{1/2}\right)^{1/2}\Sigma_s^{-1/2}=\Sigma_t, \tag{8.13}$$

即 T 恰好把源分布推送为目标分布。由于 A 是对称正定矩阵，T 是凸函数 $\frac{1}{2}(z-\mu_s)^{\top}A(z-\mu_s)+\mu_t^{\top}z$ 的梯度，由 Brenier 定理它就是式（8.10）的最优解。数值上 $\lVert A\hat{\Sigma}_sA-\hat{\Sigma}_t\rVert_{max}/\lVert\hat{\Sigma}_t\rVert_{max}=2\times10^{-14}$。与“把样本拉向目标中心”的做法不同，OT 保持源样本在分布中的相对位置：偏离源中心的样本会被映射到目标分布中同样偏离中心的位置，因此迁移后的群体保持多样性。

迁移路径。 以强度 α 在源谱与其传输像之间做线性插值（McCann 插值）：

$$z_\alpha=(1-\alpha)z+\alpha T(z),\ \alpha\in\{0,0.25,0.5,0.75,1\}, \tag{8.14}$$

并逆映射回原尺度 $x'_\alpha=G^{-1}(z_\alpha)$。

缺陷非增投影。 记四项实测缺陷（DMS、双乙酰、4VG、t2n）的下标集合为 D，在原储存尺度上投影：

$$x''_{\alpha,k}=\begin{cases}\min(x'_{\alpha,k},x_k), & k\in D\text{且}x_k\text{已观测}\\ x'_{\alpha,k}, & \text{其他}\end{cases} \tag{8.15}$$

这是到可行域 $\{x'':x''_k\le x_k,k\in D\}$ 的欧氏投影。源缺陷值处于下限时，投影后仍等于下限；源缺陷值缺失时无法施加约束，该样本记为“不可完整验收”。

陈化方向指数。 Stale hops 没有对应的化学测量。附件 8 阈值表的 aging 列标注了随陈化增加的 17 项（集合 J^{inc}，如 β-大马酮、3-甲基丁醛、糠醛、t2n）与随陈化减少的 7 项（集合 J^{dec}，如芳樟醇、香叶醇、乙酸异戊酯）。以训练集经验百分位 F_k 构造陈化方向指数：

$$A(x)=\frac{1}{2}\left(\acute{F}^{inc}+\acute{F}^{dec}\right),\ \acute{F}^{inc}=\frac{1}{|J^{inc}|}\sum_{k\in J^{inc}}F_k(x_k),\ \acute{F}^{dec}=\frac{1}{|J^{dec}|}\sum_{k\in J^{dec}}\left(1-F_k(x_k)\right). \tag{8.16}$$

$A(x)\in(0,1)$，越大表示越接近陈化状态。两组分别平均，避免项数多的一组获得更大权重。

验收标准（运行前冻结）。 记距离 $d(u,v)=\lVert u-v\rVert/\sqrt{p}$，$R_t$ 为真实 Tripel 样本集，$z''=G(x'')$。候选须同时满足：

$$
\begin{aligned}
&\text{C1：}\ x''_k \le x_k,\ \forall k \in D,\text{四项均可评价且下限状态保持;}\\
&\text{C2：}\ 1-\frac{d\left(z'',\mu_t\right)}{d\left(z,\mu_t\right)} \ge 0.20;\\
&\text{C3：}\ 1-\frac{\min\limits_{u\in R_t} d\left(z'',u\right)}{\min\limits_{u\in R_t} d\left(z,u\right)} \ge 0.10;\\
&\text{C4：}\ A\left(x''\right) \le A\left(x\right),\text{且两者均可完整评价。}
\end{aligned}
\tag{8.17}
$$

按 α 从小到大检查，取第一个同时满足 C1–C4 的候选，即在满足要求的前提下对源谱改动最小：

$$
\alpha^{\wedge} = min\left\{\alpha>0 : \text{C1–C4 均成立}\right\}. \tag{8.18}
$$

作为对照，首版优化基线求解有界二次规划 $min_z \frac{1}{2}\left\| z-z_0\right\|^2+\frac{\lambda}{2}\left(z-\mu_t\right)^{\top}\hat{\Sigma}_t^{-1}\left(z-\mu_t\right)$ 权重 $\lambda \in \{0.1,0.3,1,3,10\}$，同样施加缺陷上界，并按式（8.17）重新验收。

8.7 迁移结果

冻结标准下的验收。 OT 方法：8 款达标（均在 $\alpha^{\wedge}=0.5$），1 款未达标（beer 209），2 款因源双乙酰缺失不可完整验收（beer 28、119）。首版优化基线：9 款达标（$\lambda=0.3$ 两款、$\lambda=1$ 七款），2 款不可完整验收。两种方法的所有可评价候选均满足 C1 与 C4。

图 14（b）给出 11 条迁移路径在主成分平面上的投影：源谱从 Pils/Lager 区域出发沿 OT 方向进入真实 Tripel 所在区域，各条路径大致平行而不汇聚到一点。

缺陷约束的作用。 图 16（a）给出 OT 候选（$\alpha=0.5$）四项缺陷的浓度与源谱之比。42 个可评价的缺陷坐标中，38 个恰好等于源值，即被式（8.15）截断。原因在于目标风格本身缺陷化合物更高：Tripel 的 DMS、双乙酰、t2n 中位浓度分别约为 Pils/Lager 的 2.6、5.6、5.6 倍，单纯向 Tripel 迁移会提高这些缺陷。非增投影在这里实质性地起了作用，把“像 Tripel”与“不变差”这两个相互冲突的目标统一起来。图 16（b）显示，所有可评价样本的陈化方向指数均下降，平均由 0.603 降至 0.505。

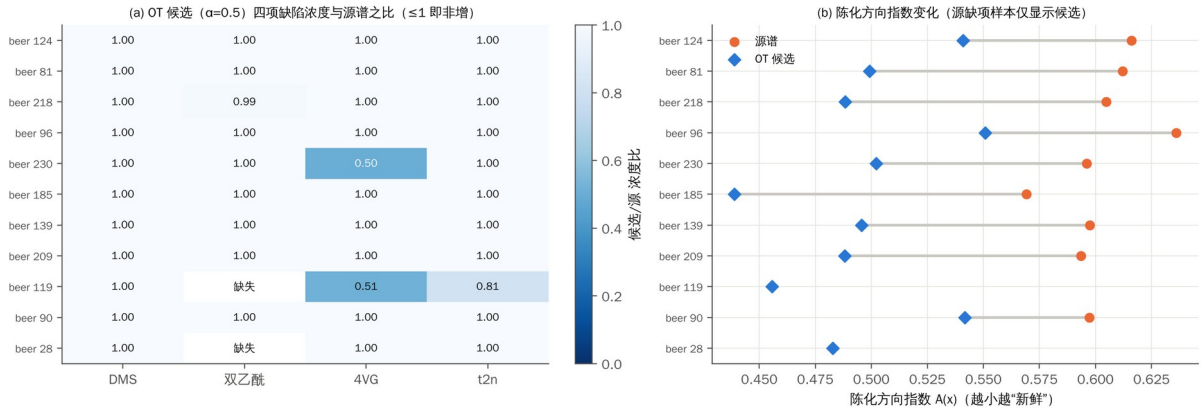


图 16 OT 候选的缺陷浓度比与陈化方向指数变化

beer 209 未达标的原因。 该样本在 $\alpha=0.5$ 时最近邻距离下降 30.1%，陈化指数由 0.593 降至 0.488，C1、C3、C4 均满足，但中心距离只下降 17.4%，且随 α 增大先升后降（$\alpha=1$ 时仅 7.9%）。这由 OT 的性质决定：OT 把样本送到目标分布中的对应位置，而不是目标中心。真实 Tripel 到自身中心的平均距离为 0.840，而 Pils 源到 Tripel 中心的距离为 0.93–1.15，即使候选落在一个“典型真实 Tripel”的位置，11 款源中也有 6 款只能获得 10%–19% 的中心距离下降。判据 C2 因而偏向向中心收缩的方法，而优化基线的目标函数恰好包含到中心的马氏距离。

目标接近与干预幅度。 图 17（左）显示，在相同源谱改变量 $RMS(z''-z)$ 下，OT 获得的最近邻距离下降始终大于优化基线。在两法共同达标的 8 款上，按各自首次达标点比较，OT 平均改变量为 0.491，优化基线为 0.699，8 款中有 7 款 OT 更小。两法的强度参数含义与网格不同，这一比较为描述性比较。

多样性保留。 图 17（右）显示，优化基线随权重增大明显向中心塌缩：$\lambda=10$ 时迁移群体的平均两两距离只有 0.219，仅为真实 Tripel（1.213）的 18%；OT 的群体离散度保持在 0.89–0.94，接近源谱水平（0.982）。这说明 OT 迁移后的候选保留了源谱之间的差异，而不是全部变成同一种“平均 Tripel”。另一方面，OT 候选的离散度仍低于真实 Tripel，$\alpha\geq0.5$ 时到中心的平均距离（≤0.735）也小于真实 Tripel 的 0.840，即经逆变换、下限并列与缺陷投影后，候选仍比真实 Tripel 更集中。

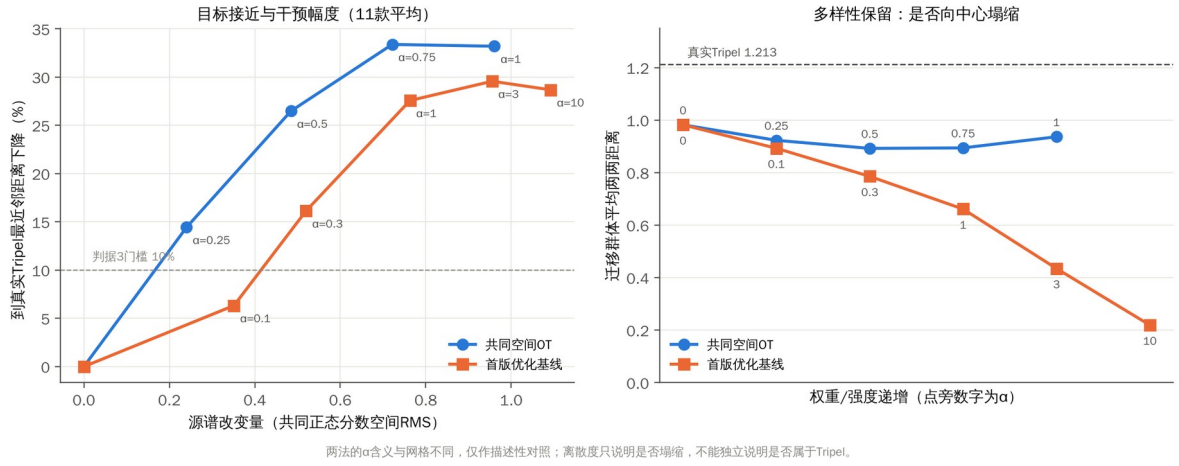


图 17 迁移的目标接近、干预幅度与多样性保留

事后敏感性分析。 在结果出现后，把 C2 换成目标风格经验容许域：

$$d(z'',\mu_t)\leq q_{0.95}\{d(z_u,\mu_{t,-u}):u\in R_t\},\quad(8.19)$$

其中 $\mu_{t,-u}$ 为剔除样本 u 后的 Tripel 中心，P95 半径为 1.183。该容许域已包含全部 11 款未迁移的 Pils 源谱，在本数据上缺乏风格区分能力；替换后风格接近性主要由 C3 决定，缺陷与陈化约束仍然有效。这一负面结果说明，高维空间中的中心距离难以单独刻画风格归属，冻结的主结果不因此改变。

8.8 小结

问题三以“经验边际映射 + 风格条件收缩高斯”构建生成模型，在均值、边际分布与下限点质量上均优于首版基线，生成谱的感官画像与各风格的实测特征方向一致。迁移采用共同空间高斯最优传输加缺陷非增投影，在冻结标准下 9 款可完整评价的源谱中 8 款达标；与优化基线相比，OT 以更小的源谱改动达到目标接近，并避免了群体向中心塌缩。在本数据及评价口径下，OT 在目标接近、源谱改动与多样性保留之间提供了一种不同的折中。生成与迁移结果是候选化学谱，其风格归属与感官表现仍需酿造试验与独立品评验证。

九、模型评价与推广

9.1 模型优点

（1）**数据核查扎实。** 通过附件 8 的校准数据核实了 70 个指标的对数底数，通过数值重算确认附件 2 为 Spearman 相关，并发现了评分卡模板的公式错误，避免了建立在错误前提上的建模。

（2）**验证严格。** 所有预处理在训练折内完成；评分锚点与勾选阈值都来自折外信息；方法比较使用修正方差的配对推断；问题三的验收标准在运行前冻结。

（3）**结论分级。** 预测支持度、解释稳定性、风格内置换三层信息共同约束 Top-3 的解释范围，避免把风格代理误认为化学驱动。

（4）**生成器贴合数据结构。** 经验边际映射在不增加模型复杂度的前提下保留了检出下限的点质量；闭式最优传输无需迭代优化，并保持迁移群体的多样性。

9.2 模型不足

（1）原始 0–6 评分尺度缺失，相对评分是保序的展示尺度，无法用原始评分直接验证。

（2）约一半感官维度的预测支持度较低，缺陷感官（dms、diacetyl、stale_hops）尤其如此。

（3）风格内置换只排除了已记录风格造成的混杂，未覆盖酿造工艺等其他混杂因素。

（4）生成器的相关结构未优于无条件高斯，小样本风格的协方差依赖较强的收缩。

（5）迁移的风格判定依赖距离指标，Stale hops 缺少化学测量，只能用陈化方向指数作代理。

9.3 模型改进与推广

若能获得原始品评记录，可直接用有序回归替代分位锚点映射；若能获得更多同风格样本，可用 copula 或归一化流改进联合结构。本文“折外锚点评分 + 预测支持度分档”的思路适用于只有标准化标签的各类感官评价问题；“经验 CDF 潜在高斯 + 闭式最优传输 + 约束投影”可推广到葡萄酒、咖啡、乳制品等含大量检出下限的代谢组数据的配方设计。

参考文献

[1] Schreurs M, Piampongsant S, Roncoroni M, et al. Predicting and improving complex beer flavor through machine learning. Nature Communications, 15: 2368, 2024.

[2] Geurts P, Ernst D, Wehenkel L. Extremely randomized trees. Machine Learning, 63(1): 3–42, 2006.

[3] Breiman L. Random forests. Machine Learning, 45(1): 5–32, 2001.

[4] Ledoit O, Wolf M. A well-conditioned estimator for large-dimensional covariance matrices. Journal of Multivariate Analysis, 88(2): 365–411, 2004.

[5] Blondel V D, Guillaume J L, Lambiotte R, Lefebvre E. Fast unfolding of communities in large networks. Journal of Statistical Mechanics: Theory and Experiment, 2008(10): P10008, 2008.

[6] Nadeau C, Bengio Y. Inference for the generalization error. Machine Learning, 52(3): 239–281, 2003.

[7] Peyré G, Cuturi M. Computational optimal transport. Foundations and Trends in Machine Learning, 11(5–6): 355–607, 2019.

[8] Flamary R, et al. POT: Python Optimal Transport, Gaussian optimal transport（ot.gaussian）, https://pythonot.github.io/gen_modules/ot.gaussian.html, 访问时间 2026 年 10 月 26 日。

[9] Strobl C, Boulesteix A L, Kneib T, Augustin T, Zeileis A. Conditional variable importance for random forests. BMC Bioinformatics, 9: 307, 2008.

[10] Wold S, Sjöström M, Eriksson L. PLS-regression: a basic tool of chemometrics. Chemometrics and Intelligent Laboratory Systems, 58(2): 109–130, 2001.

[11] Saison D, De Schutter D P, Uyttenhove B, Delvaux F, Delvaux F R. Contribution of staling compounds to the aged flavour of lager beer by studying their flavour thresholds. Food Chemistry, 114(4): 1206–1215, 2009.

[12] Santos J R, Carneiro J R, Guido L F, et al. Determination of E-2-nonenal by high-performance liquid chromatography with UV detection: assay for the evaluation of beer ageing. Journal of Chromatography A, 985(1–2): 395–402, 2003.

[13] Claude, Claude Opus 5.5, Anthropic, 2026-10-26.

[14] 【待队伍补充】OpenAI Codex，版本，OpenAI，2026-10-24 至 2026-10-26。

AI 工具使用声明

本参赛队在竞赛中使用了 AI 工具，详细说明见支撑材料《AI 工具使用详情说明》

- **Claude（Claude Opus 5.5，Anthropic）[13]**：用于数据尺度与模板结构核查、v3.1 方案中 Q1 相对评分与评分卡填写、Q2 结果重组与风格内置换实验、Q3 潜在高斯生成器与最优传输迁移的代码实现与运行、论文图表绘制与初稿撰写。相应源程序文件头部已注明 AI 辅助情况。
- **OpenAI Codex [14]**：用于早期数据审计、问题一基线、问题二 bootstrap 主流程及问题三首版基线的代码实现（见 analysis/ 下各脚本头部注释）。

【待队伍补充：建模方案的提出与决策、关键判据的确定由哪些队员完成，以及对 AI 生成内容的人工核查与修改情况。】

附录

附录 A　补充结果表

表 A1 50 维 Top-3 候选与两维分档

感官维度	R^2	支持度	解释稳定性	Top-3 候选
acidity	0.721	A	模块线索	lactic_acid.mg.L.；X4.ethyl.guaiacol；X4.ethyl.phenol
alcohol	0.676	A	单体稳定	ethanol..v.v.；ethyl_octanoate；lactic_acid.mg.L.
acetic	0.662	A	模块线索	lactic_acid.mg.L.；X4.ethyl.guaiacol；ethyl_isovalerate
lactic	0.659	A	模块线索	lactic_acid.mg.L.；ethyl_isovalerate；ethyl.phenylacetate
bitternes	0.56	A	模块线索	lactic_acid.mg.L.；ethyl_octanoate；cis.dihydro.beta.terpineol
body	0.498	A	模块线索	ethanol..v.v.；ethyl_octanoate；ethyl_isovalerate
F_hops_all	0.484	A	单体稳定	ethyl_isovalerate；lactic_acid.mg.L.；cis.dihydro.beta.terpineol
F_malt_all	0.468	A	探索性	lactic_acid.mg.L.；ethanol..v.v.；X4.ethyl.phenol
aftertaste	0.452	A	单体稳定	ethyl_isovalerate；ethyl_octanoate；ethanol..v.v.
A_malt_all	0.374	A	探索性	lactic_acid.mg.L.；beta.phellandrene；alpha.muurolol
F_hops_noble	0.366	A	模块线索	ethyl_isovalerate；ethyl_octanoate；cis.dihydro.beta.terpineol
F_malt_burn	0.363	A	探索性	lactic_acid.mg.L.；ethyl_octanoate；ethyl_decanoate
A_hops_all	0.355	A	模块线索	ethyl_isovalerate；ethyl_octanoate；lactic_acid.mg.L.
F_malt_cara	0.327	A	模块线索	lactic_acid.mg.L.；ethyl_isovalerate；X4.ethyl.phenol
A_malt_cara	0.305	A	探索性	lactic_acid.mg.L.；X4.ethyl.phenol；ethyl_isovalerate
A_hops_noble	0.293	B	模块线索	ethyl_octanoate；ethyl_isovalerate；cis.dihydro.beta.terpineol
co2	0.293	B	模块线索	ethyl_octanoate；ethanol..v.v.；ethyl_decanoate
F_esters_all	0.272	B	模块线索	ethanol..v.v.；ethyl_isovalerate；ethyl_acetate
sweetness	0.251	B	探索性	lactic_acid.mg.L.；ethyl_octanoate；E.cinnamaldehyde
A_malt_burn	0.232	B	探索性	lactic_acid.mg.L.；cis.dihydro.beta.terpineol；alpha.muurolol
F_malt_bread	0.213	B	探索性	lactic_acid.mg.L.；ethyl_octanoate；X4.ethyl.phenol
F_esters_ethac	0.213	B	模块线索	ethanol..v.v.；ethyl_acetate；ethyl_octanoate
F_hops_citrus	0.202	B	探索性	lactic_acid.mg.L.；cis.dihydro.beta.terpineol；

感官维度	R^2	支持度	解释稳定性	Top-3 候选
				ethyl_isovalerate
A_esters_all	0.187	B	模块线索	ethyl_acetate；ethyl_octanoate；ethyl_isovalerate
A_hops_citrus	0.162	B	模块线索	lactic_acid.mg.L.；ethyl_isovalerate；cis.dihydro.beta.terpineol
F_hops_woody	0.154	B	模块线索	ethyl_isovalerate；lactic_acid.mg.L.；cis.dihydro.beta.terpineol
X4vg	0.154	B	探索性	lactic_acid.mg.L.；X4VG；ethyl_octanoate
F_hops_tropical	0.138	B	模块线索	lactic_acid.mg.L.；ethyl_isovalerate；cis.dihydro.beta.terpineol
overall	0.134	B	模块线索	ethyl_octanoate；ethyl_decanoate；ethyl_acetate
coriander	0.128	B	模块线索	X1.alpha.terpineol；ethyl..E..linalool；ethyl_isovalerate
A_esters_ethac	0.121	B	模块线索	ethyl_octanoate；ethanol..v.v.；ethyl_decanoate
A_hops_tropical	0.116	B	探索性	lactic_acid.mg.L.；ethyl_isovalerate；X2.butyloctanoate
F_esters_fruity	0.115	B	探索性	beta.phellandrene；ethyl_isovalerate；ethyl_octanoate
F_esters_isoaa	0.106	B	探索性	ethyl_isovalerate；lactic_acid.mg.L.；cis.dihydro.beta.terpineol
t2n	0.106	B	探索性	X6.methyl.5.hepten.2.one；cis.dihydro.beta.terpineol；ethanol..v.v.
A_malt_bread	0.102	B	探索性	lactic_acid.mg.L.；E.cinnamaldehyde；X4.ethyl.phenol
metallic	0.097	C	模块线索	ethyl_isovalerate；cis.dihydro.beta.terpineol；ethyl_octanoate
diacetyl	0.084	C	探索性	ethyl_octanoate；ethyl_decanoate；lactic_acid.mg.L.
orange	0.069	C	探索性	X2.ethyl.1.hexanol；lactic_acid.mg.L.；ethyl_octanoate
A_esters_isoaa	0.066	C	探索性	ethyl_isovalerate；ethyl_decanoate；lactic_acid.mg.L.
clove	0.059	C	探索性	lactic_acid.mg.L.；limonene；cis.dihydro.beta.terpineol
A_hops_woody	0.059	C	模块线索	ethyl_isovalerate；cis.dihydro.beta.terpineol；ethyl_acetate
dms	0.058	C	探索性	MeSH；isopentyl.butanoate；methyl.geranate
F_malt_grain	0.053	C	探索性	cis.dihydro.beta.terpineol；ethyl_isovalerate；lactic_acid.mg.L.
A_esters_fruity	0.05	C	探索性	beta.phellandrene；lactic_acid.mg.L.；benzyl.alcohol
A_malt_grain	0.04	C	探索性	lactic_acid.mg.L.；cis.dihydro.beta.terpineol；beta.phellandrene
A_esters_flower	0.03	C	探索性	ethyl_isovalerate；beta.phellandrene；lactic_acid.mg.L.
barnyard	0.018	C	模块线索	ethyl_isovalerate；ethyl_acetate；ethyl_octanoate
stale_hops	0.006	C	模块线索	ethyl_isovalerate；ethyl_acetate；ethyl_octanoate
F_esters_flower	-0.005	C	探索性	beta.phellandrene；lactic_acid.mg.L.；X4.ethyl.guaiacol

表 A2 风格内置换按化合物汇总（出现于至少 3 个维度的 Top-3）

化合物	出现维度数	保留比例中位数	风格内为正比例	风格内仍在 Top-10
lactic_acid.mg.L.	32	0.02	59.4%	43.8%
ethyl_isovalerate	27	0.244	92.6%	85.2%
ethyl_octanoate	22	0.236	86.4%	81.8%
cis.dihydro.beta.terpineol	16	0.258	93.8%	87.5%
ethanol..v.v.	9	0.131	66.7%	55.6%
ethyl_acetate	7	0.206	100.0%	85.7%

化合物	出现维度数	保留比例中位数	风格内为正比例	风格内仍在 Top-10
X4.ethyl.phenol	6	0.099	66.7%	50.0%
beta.phellandrene	6	0.285	100.0%	83.3%
ethyl_decanoate	6	0.485	100.0%	83.3%
X4.ethyl.guaiacol	3	0.098	100.0%	66.7%

表 A3 迁移逐样本验收（冻结判据与事后容许域判据；容许域 P95 半径 1.183）

方法	源样本	冻结判据	首次达标 α	容许域判据	容许域首次达标 α
首版优化基线	28	不可完整验收	—	不可完整验收	—
首版优化基线	81	达标	0.3	达标	0.1
首版优化基线	90	达标	1.0	达标	1.0
首版优化基线	96	达标	0.3	达标	0.3
首版优化基线	119	不可完整验收	—	不可完整验收	—
首版优化基线	124	达标	1.0	达标	0.3
首版优化基线	139	达标	1.0	达标	0.3
首版优化基线	185	达标	1.0	达标	0.3
首版优化基线	209	达标	1.0	达标	0.3
首版优化基线	218	达标	1.0	达标	0.3
首版优化基线	230	达标	1.0	达标	0.3
共同空间 OT	28	不可完整验收	—	不可完整验收	—
共同空间 OT	81	达标	0.5	达标	0.25
共同空间 OT	90	达标	0.5	达标	0.25
共同空间 OT	96	达标	0.5	达标	0.25
共同空间 OT	119	不可完整验收	—	不可完整验收	—
共同空间 OT	124	达标	0.5	达标	0.25
共同空间 OT	139	达标	0.5	达标	0.25
共同空间 OT	185	达标	0.5	达标	0.25
共同空间 OT	209	未达到预设标准	—	达标	0.25
共同空间 OT	218	达标	0.5	达标	0.25
共同空间 OT	230	达标	0.5	达标	0.25

附录 B　软件与源程序

运行环境：Python 3.13、NumPy 2.5.3、pandas 3.0.5、SciPy 1.18.1、scikit-learn

1.6.1、NetworkX、Matplotlib、openpyxl；首版分析环境为 Python 3.9.13、NumPy 1.26.4、scikit-learn 1.6.1（见 analysis/q3/environment_versions.json）。

程序	作用
analysis/A_feasibility/day1_minimum.py、floor_ablation/run_ablation.py、q1_metric_supplement.py	数据审计、问题一基线与 5×3 折外预测、下限指示消融
src/q1_sensory_imputation/build_relative_scores.py	折外锚点相对评分、预测支持度、评分卡填写
analysis/q2/run_q2.py、supplement_reliability.py	共表达网络、100 次 bootstrap、稳定性
src/q2_compound_flavor/build_v31_tables.py、within_style_permutation.py	两维分档、高度相关备选、风格内置换
analysis/q3/run_gaussian_baseline.py	首版风格高斯与优化迁移基线
src/q3_style_generation/latent_gaussian_ot.py、sensitivity_and_checks.py	潜在高斯生成、OT 迁移、验收与敏感性分析
src/make_paper_figures.py、src/make_paper_figures_2.py	论文图表

源程序清单

analysis/A_feasibility/audit_and_pilot.py

```
"""Exploratory audit only; not a final contest model.

AI assistance: OpenAI Codex, 2026-10-24; model version unavailable in runtime.
All reported metrics use out-of-fold predictions with fold-local preprocessing.
"""
from pathlib import Path
import json
import numpy as np
import pandas as pd
from openpyxl import load_workbook
from sklearn.base import clone
from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, QuantileTransformer
from sklearn.linear_model import Ridge
from sklearn.cross_decomposition import PLSRegression
from sklearn.ensemble import ExtraTreesRegressor
from sklearn.dummy import DummyRegressor
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import r2_score, mean_absolute_error

ROOT = Path(__file__).resolve().parents[2]
OUT = Path(__file__).resolve().parent
DATA = next(ROOT.glob('2026年第七届*/2026*赛道A*')) / '附件'
Xraw = pd.read_csv(DATA / '1.csv').set_index('beer')
Yraw = pd.read_csv(DATA / '4.csv').set_index('beer').loc[Xraw.index]
X = Xraw.drop(columns=['beer_id', 'tasting_category_fine'])
Y = Yraw.drop(columns=['beer_id', 'tasting_category_fine'])
ref = pd.read_excel(DATA / '8.xlsx', sheet_name=None)
cal = ref['Calibrated chemical dataset'].set_index('beer').loc[X.index]

transforms = []
for col in cal.columns:
    if col not in X:
        continue
    a = X[col].to_numpy(float)
    b = cal[col].to_numpy(float)
    ok = np.isfinite(a) & np.isfinite(b) & (b > 0)
    candidates = [
        ('identity', b), ('ln', np.log(np.where(b > 0, b, np.nan))),
```

```
        ('log10', np.log10(np.where(b > 0, b, np.nan)))
    ]
    errors = {name: float(np.median(abs(a[ok]-v[ok]))) for name, v in candidates}
    best = min(errors, key=errors.get)
    expected = dict(candidates)[best]
    max_error = float(np.max(abs(a[ok]-expected[ok])))
    transforms.append({'column': col, 'n_positive_pairs': int(ok.sum()),
        'best_transform_by_median': best, 'median_error': errors[best],
        'max_error': max_error, 'all_positive_pairs_verified':max_error < 1e-7,
        'agreement_fraction_1e_7':float(np.mean(abs(a[ok]-expected[ok]) < 1e-7)),
        'raw_zero_count': int((b == 0).sum()),
        'stored_values_when_raw_zero': np.unique(a[b == 0]).tolist()[:8]})
pd.DataFrame(transforms).to_csv(OUT / 'verified_transforms.csv', index=False, encoding='utf-8-sig')

threshold = ref['Sensory thresholds']
name_map = dict(zip(threshold.field_name_R, threshold.chemical_name_short))
name_map.update(dict(zip(threshold.field_name_R, threshold.chemical_name)))
matrix2 = pd.read_csv(DATA / '2.csv', index_col=0)
mapped = {c: name_map[c] for c in X if c in name_map and name_map[c] in matrix2.index}
corr_errors = {}
for method in ['pearson', 'spearman']:
    corr = X[list(mapped)].corr(method=method).to_numpy()
    supplied = matrix2.loc[list(mapped.values()),list(mapped.values())].to_numpy()
    ij = np.triu_indices_from(corr, k=1)
    corr_errors[method] = {'pairs': len(ij[0]), 'MAE': float(np.nanmean(abs(corr[ij]-
supplied[ij]))),
        'max_abs_error': float(np.nanmax(abs(corr[ij]-supplied[ij])))}

matrix5 = pd.read_csv(DATA / '5.csv', index_col=0)
sn = ref['Sensory names']
ymap = dict(zip(sn.R_name, sn.CleanName_PanelFeedback))
validy = {c: ymap[c] for c in Y if c in ymap and ymap[c] in matrix5.index}
corr5_errors = {}
for method in ['pearson', 'spearman']:
    a = Y[list(validy)].corr(method=method).to_numpy()
    b = matrix5.loc[list(validy.values()), list(validy.values())].to_numpy()
    ij = np.triu_indices_from(a, k=1)
    corr5_errors[method] = {'matched_dimensions':len(validy), 'MAE':float(np.nanmean(abs(a[ij]-
b[ij])))}

wb = load_workbook(DATA / '3.xlsx', data_only=False, read_only=True)
headers = [v for v in next(wb['summary'].values) if v is not None]
styles = Xraw.tasting_category_fine
models = {
    'training_mean': make_pipeline(SimpleImputer(strategy='median'), DummyRegressor()),
    'ridge_alpha100': make_pipeline(SimpleImputer(strategy='median'), StandardScaler(),
Ridge(alpha=100)),
    'pls_8': make_pipeline(SimpleImputer(strategy='median'), StandardScaler(),
PLSRegression(n_components=8, scale=False)),
    'ridge_quantile_alpha100': make_pipeline(SimpleImputer(strategy='median'),
        QuantileTransformer(n_quantiles=100, output_distribution='normal', random_state=20261024),
Ridge(alpha=100)),
    'pls_quantile_8': make_pipeline(SimpleImputer(strategy='median'),
        QuantileTransformer(n_quantiles=100, output_distribution='normal', random_state=20261024),
PLSRegression(n_components=8, scale=False)),
    'extra_trees_leaf3': make_pipeline(SimpleImputer(strategy='median'), ExtraTreesRegressor(
        n_estimators=160, min_samples_leaf=3, max_features=0.7, random_state=20261024, n_jobs=2))
}
folds = list(StratifiedKFold(n_splits=3, shuffle=True, random_state=20261024).split(X, styles))
records = []
summary = {}
for name, model in models.items():
    pred = np.empty(Y.shape)
    for train, test in folds:
        fit = clone(model).fit(X.iloc[train], Y.iloc[train])
```

```
        pred[test] = fit.predict(X.iloc[test])
    scores = r2_score(Y, pred, multioutput='raw_values')
    for j, col in enumerate(Y):
        records.append({'model':name, 'sensory':col, 'oof_R2':float(scores[j]),
            'oof_MAE':float(mean_absolute_error(Y.iloc[:,j],pred[:,j]))})
    summary[name] = {'mean_oof_R2_50':float(scores.mean()),
        'mean_oof_R2_aroma_flavour_30':float(scores[:30].mean()),
        'positive_R2_count':int((scores > 0).sum()),
        'lowest5':dict(sorted(zip(Y.columns, scores), key=lambda p:p[1])[:5]),
        'highest5':dict(sorted(zip(Y.columns, scores), key=lambda p:p[1], reverse=True)[:5])}
    print(name, json.dumps(summary[name],ensure_ascii=False), flush=True)
pd.DataFrame(records).to_csv(OUT / 'pilot_oof_metrics.csv', index=False, encoding='utf-8-sig')
audit = {'status':'exploratory; fixed hyperparameters; not nested CV; no original-score validation',
    'n':len(X),'p':X.shape[1],'q':Y.shape[1],
    'chemical_missing':X.isna().sum()[lambda v:v>0].to_dict(),
    'chemical_missing_fraction':float(X.isna().sum().sum()/X.size),
    'sensory_missing':int(Y.isna().sum().sum()),
    'styles':styles.value_counts().to_dict(),
    'template_summary_columns':headers,
    'correlation2_audit':corr_errors,'correlation5_audit':corr5_errors,
    'transform_counts_by_median_only':pd.Series([x['best_transform_by_median'] for x in
transforms]).value_counts().to_dict(),
    'fully_verified_transform_counts':pd.Series([x['best_transform_by_median'] for x in transforms
if x['all_positive_pairs_verified']]).value_counts().to_dict(),
    'pilot':summary}
(OUT / 'audit_summary.json').write_text(json.dumps(audit,ensure_ascii=False,indent=2),encoding='utf-
8')
print('AUDIT',json.dumps({k:v for k,v in audit.items() if k!
='pilot'},ensure_ascii=False),flush=True)
```

analysis/A_feasibility/day1_minimum.py

```
"""Day-one minimal analysis; outputs are intermediate data and audit records.

AI assistance: OpenAI Codex, 2026-10-24; exact model version unavailable.
Score mappings are explicit conventions, not recovered raw taster ratings.
"""
from pathlib import Path
import json
import numpy as np
import pandas as pd
from sklearn.base import clone
from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import QuantileTransformer
from sklearn.cross_decomposition import PLSRegression
from sklearn.ensemble import ExtraTreesRegressor
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import r2_score, mean_absolute_error

ROOT = Path(__file__).resolve().parents[2]
OUT = Path(__file__).parent / 'day1'
OUT.mkdir(exist_ok=True)
DATA = next(ROOT.glob('2026年第七届*/2026*赛道A*')) / '附件'
x = pd.read_csv(DATA / '1.csv').set_index('beer')
y = pd.read_csv(DATA / '4.csv').set_index('beer').loc[x.index]
assert x.index.is_unique and y.index.is_unique
assert (x.tasting_category_fine == y.tasting_category_fine).all()
X = x.drop(columns=['beer_id','tasting_category_fine'])
Y = y.drop(columns=['beer_id','tasting_category_fine'])
sn = pd.read_excel(DATA / '8.xlsx',sheet_name='Sensory names')
sn = sn[sn.R_name.isin(Y.columns)].set_index('R_name').loc[Y.columns]
sn.index.name='R_name'
assert sn.index.is_unique and len(sn)==50 and sn.DataType.notna().all()
types = sn.DataType.to_dict()
sn.to_csv(OUT/'sensory_metadata.csv',encoding='utf-8-sig')
threshold = pd.read_excel(DATA / '8.xlsx',sheet_name='Sensory thresholds')
```

```
trans = pd.read_csv(Path(__file__).parent/'verified_transforms.csv').set_index('column')
dictionary = []
for col in X:
    rows = threshold[threshold.field_name_R == col]
    one = rows.iloc[0] if len(rows) else None
    record = {'field':col,'nonmissing_count':int(X[col].notna().sum()),
              'missing_count':int(X[col].isna().sum()),'status':'needs_review'}
    for target, source in [('chemical_name','chemical_name'),('unit','unit'),
                           ('origin','origin_paper'),('threshold_median','threshold_median'),
                           ('threshold_mean','threshold_mean')]:
        val = one[source] if one is not None else None
        record[target] = None if val is None or pd.isna(val) else val
    if col in trans.index:
        t=trans.loc[col]
        record.update({'candidate_transform':t.best_transform_by_median,
                       'positive_pair_agreement_fraction':float(t.agreement_fraction_1e_7),
                       'status':'verified_positive_pairs' if bool(t.all_positive_pairs_verified)
else 'partial_agreement_needs_review'})
    dictionary.append(record)
(OUT/'chemical_dictionary.json').write_text(json.dumps(dictionary,ensure_ascii=False,indent=2),encod
ing='utf-8')
pd.concat([x,Y.add_prefix('sensory__')],axis=1).to_csv(OUT/'joined_analysis_data.csv',encoding='utf-
8-sig')

models = {
    'PLS_quantile8':make_pipeline(SimpleImputer(strategy='median'),QuantileTransformer(

n_quantiles=100,output_distribution='normal',random_state=20261024),PLSRegression(n_components=8,sca
le=False)),
    'ExtraTrees_leaf3':make_pipeline(SimpleImputer(strategy='median'),ExtraTreesRegressor(
        n_estimators=160,min_samples_leaf=3,max_features=0.7,random_state=20261024,n_jobs=2))}
folds = list(StratifiedKFold(3,shuffle=True,random_state=20261024).split(X,x.tasting_category_fine))
rng=np.random.default_rng(20261024)
masked_tests=[]
for train,test in folds:
    xx=X.iloc[test].copy();mask=(rng.random(xx.shape)<.10)&xx.notna().to_numpy()
    masked_tests.append(xx.mask(mask))

records=[];summary={};checks=[]
for name, model in models.items():
    pred=np.empty(Y.shape);masked=np.empty(Y.shape);draft=np.empty(Y.shape)
    for fold,(train,test) in enumerate(folds):
        fit=clone(model).fit(X.iloc[train],Y.iloc[train])
        pred[test]=fit.predict(X.iloc[test]);masked[test]=fit.predict(masked_tests[fold])
        for j,col in enumerate(Y):
            if types[col]=='numeric':
                lo,hi=Y.iloc[train,j].quantile([.05,.95])
                assert hi>lo
                draft[test,j]=np.rint(np.clip(6*(pred[test,j]-lo)/(hi-lo),0,6))
                checks.append({'model':name,'fold':fold,'field':col,'type':'numeric',
                               'anchor_q05':float(lo),'anchor_q95':float(hi)})
            else:
                draft[test,j]=(pred[test,j]>0).astype(int)
                checks.append({'model':name,'fold':fold,'field':col,'type':'categorical',
                               'selection_threshold_standardized':0})
    for condition,pp in [('natural_missing',pred),('extra_10pct_mask',masked)]:
        rr=r2_score(Y,pp,multioutput='raw_values')
        aa=mean_absolute_error(Y,pp,multioutput='raw_values')
        for j,col in enumerate(Y):
            records.append({'model':name,'condition':condition,'field':col,'type':types[col],
                            'oof_R2':float(rr[j]),'oof_MAE_standardized':float(aa[j])})
        summary[name+'__'+condition]={}
        for kind in ['all','numeric','categorical']:
            idx=np.array([True if kind=='all' else types[c]==kind for c in Y])
            summary[name+'__'+condition][kind]={'n_dimensions':int(idx.sum()),
```

```
'mean_oof_R2':float(rr[idx].mean()),'mean_oof_MAE_standardized':float(aa[idx].mean())}

pd.DataFrame(pred,index=Y.index,columns=Y.columns).to_csv(OUT/(name+'_oof_standardized.csv'),encodin
g='utf-8-sig')
    dd=pd.DataFrame(draft.astype(int),index=Y.index,columns=Y.columns)
    assert all(dd[c].between(0,6 if types[c]=='numeric' else 1).all() for c in dd)
    dd.to_csv(OUT/(name+'_draft_relative_scores.csv'),encoding='utf-8-sig')
    display=dd.copy()
    for col in Y:
        if types[col]=='categorical':
            parent='_'.join(col.split('_')[:2])+'_all'
            assert parent in dd and types[parent]=='numeric'
            display[col]=dd[col]*dd[parent]
    display.to_csv(OUT/(name+'_draft_template_display_scores.csv'),encoding='utf-8-sig')
    print(name, json.dumps({k:v for k,v in summary.items() if
k.startswith(name)},ensure_ascii=False),flush=True)
pd.DataFrame(records).to_csv(OUT/'baseline_metrics.csv',index=False,encoding='utf-8-sig')
pd.DataFrame(checks).to_csv(OUT/'fold_score_anchors.csv',index=False,encoding='utf-8-sig')
summary.update({'type_counts':sn.DataType.value_counts().to_dict(),
    'protocol':'fixed parameters; 3-fold style-stratified OOF; train-fold-only imputation and
quantile transform; no nested tuning',
    'score_rule':'DRAFT: numeric q05->0, q95->6 linear clipped and rounded; categorical standardized
prediction>0; not original ratings or validated binary labels',
    'template_display_rule':'categorical displayed score = draft checkbox * parent numeric strength,
following template formulas; checkbox is stored separately',
    'n_rows':len(X),'n_features':X.shape[1],'n_targets':Y.shape[1]})
(OUT/'summary.json').write_text(json.dumps(summary,ensure_ascii=False,indent=2),encoding='utf-8')
print('DONE',summary['type_counts'],flush=True)
```

analysis/A_feasibility/floor_ablation/run_ablation.py

```
"""One fixed paired ablation. AI assistance: OpenAI Codex, 2026-10-25.
See 实验协议.md for frozen decision rule. Does not modify original data.
"""
from pathlib import Path
from time import perf_counter
import json,hashlib
import numpy as np
import pandas as pd
from scipy.stats import t
from sklearn.model_selection import RepeatedStratifiedKFold
from sklearn.impute import SimpleImputer
from sklearn.ensemble import ExtraTreesRegressor
from sklearn.metrics import mean_absolute_error,r2_score

OUT=Path(__file__).resolve().parent
ROOT=OUT.parents[2]
DATA=next(ROOT.glob('2026年第七届*/2026*赛道A*'))/'附件'
protocol_sha=hashlib.sha256((OUT/'实验协议.md').read_bytes()).hexdigest()
x=pd.read_csv(DATA/'1.csv').set_index('beer')
y=pd.read_csv(DATA/'4.csv').set_index('beer').loc[x.index]
X=x.drop(columns=['beer_id','tasting_category_fine']);Y=y.drop(columns=['beer_id','tasting_category_
fine'])
meta=pd.read_csv(OUT.parent/'day1/sensory_metadata.csv',index_col=0)
assert meta.index.is_unique and set(Y.columns)<=set(meta.index)
types=meta.loc[Y.columns,'DataType'].to_numpy()
splitter=RepeatedStratifiedKFold(n_splits=3,n_repeats=5,random_state=20261025)
splits=list(splitter.split(X,x.tasting_category_fine))
split_log=[];fold_rows=[];dim_rows=[]
predictions={name:np.empty((5,len(Y),Y.shape[1])) for name in ['baseline','floor_indicators']}
start=perf_counter()
for idx,(train,test) in enumerate(splits):
    repeat,fold=divmod(idx,3)
    a=X.iloc[train].to_numpy();b=X.iloc[test].to_numpy()
    mins=np.nanmin(a,axis=0)
    flags_train=np.isfinite(a)&np.isclose(a,mins,rtol=0,atol=1e-12)
```

```
    freq=flags_train.sum(axis=0)/np.isfinite(a).sum(axis=0)
    selected=freq>=.4
    flags_test=np.isfinite(b)&np.isclose(b,mins,rtol=0,atol=1e-12)
    names=X.columns[selected].tolist()
    split_log.append({'repeat':repeat,'fold':fold,'train_beer':X.index[train].tolist(),
        'test_beer':X.index[test].tolist(),'floor_columns':names,
        'training_floors':dict(zip(names,mins[selected].tolist()))})

row={'repeat':repeat,'fold':fold,'n_train':len(train),'n_test':len(test),'n_floor_columns':len(names
)}
    for model_name,aa,bb in [
        ('baseline',a,b),
        ('floor_indicators',np.column_stack([a,flags_train[:,selected].astype(float)]),
         np.column_stack([b,flags_test[:,selected].astype(float)]))]:
        tic=perf_counter();imputer=SimpleImputer(strategy='median')
        at=imputer.fit_transform(aa);bt=imputer.transform(bb)
        fit=ExtraTreesRegressor(n_estimators=160,min_samples_leaf=3,max_features=.7,
                                random_state=20261025+idx,n_jobs=2)
        fit.fit(at,Y.iloc[train]);pred=fit.predict(bt)
        assert pred.shape==(len(test),50) and np.isfinite(pred).all()
        predictions[model_name][repeat,test,:]=pred
        err=mean_absolute_error(Y.iloc[test],pred,multioutput='raw_values')
        r2=r2_score(Y.iloc[test],pred,multioutput='raw_values')
        row[model_name+'_MAE']=float(err.mean());row[model_name+'_R2']=float(r2.mean())
        row[model_name+'_seconds']=perf_counter()-tic
        for kind in ['numeric','categorical']:
            row[model_name+'_'+kind+'_MAE']=float(err[types==kind].mean())
        for j,col in enumerate(Y):dim_rows.append({'repeat':repeat,'fold':fold,
            'model':model_name,'field':col,'type':types[j],'MAE':float(err[j]),'R2':float(r2[j])})
    row['MAE_improvement']=row['baseline_MAE']-row['floor_indicators_MAE'];fold_rows.append(row)
    print(f'{idx+1}/15 MAE improvement={row["MAE_improvement"]:.6f},
indicators={len(names)}',flush=True)

df=pd.DataFrame(fold_rows);d=df.MAE_improvement.to_numpy();m=len(d)
ratio=float((df.n_test/df.n_train).mean());se=float(np.sqrt((1/m+ratio)*d.var(ddof=1)))
mean=float(d.mean());crit=float(t.ppf(.975,m-1));lo,hi=mean-crit*se,mean+crit*se
rel=mean/float(df.baseline_MAE.mean());keep=bool(rel>=.01 and lo>0)
assert len(df)==15 and all(len(set(s['train_beer'])&set(s['test_beer']))==0 for s in split_log)
summary={'protocol_sha256':protocol_sha,'n_splits':m,
    'baseline_mean_fold_MAE':float(df.baseline_MAE.mean()),
    'floor_indicators_mean_fold_MAE':float(df.floor_indicators_MAE.mean()),
    'mean_paired_MAE_improvement':mean,'relative_MAE_improvement':rel,
    'corrected_approx_95pct_CI':[lo,hi],'corrected_standard_error':se,
    'mean_test_train_ratio':ratio,'folds_improved':int((d>0).sum()),
    'keep_floor_indicators':keep,'decision':'use_floor_indicators' if keep else 'retain_baseline',
    'total_seconds':perf_counter()-start,
    'indicator_count_range':[int(df.n_floor_columns.min()),int(df.n_floor_columns.max())],
    'note':'Internal selection only; corrected CI approximate; outcomes standardized aggregates, not
raw grades.'}
for name in predictions:
    summary[name+'_mean_repeat_pooled_R2']=float(np.mean([r2_score(Y,p) for p in
predictions[name]]))
    for kind in ['numeric','categorical']:
        summary[name+'_'+kind+'_mean_fold_MAE']=float(df[name+'_'+kind+'_MAE'].mean())

np.savez_compressed(OUT/(name+'_oof.npz'),prediction=predictions[name],beer=Y.index.to_numpy(),field
s=Y.columns.to_numpy())
df.to_csv(OUT/'paired_fold_metrics.csv',index=False,encoding='utf-8-sig')
pd.DataFrame(dim_rows).to_csv(OUT/'dimension_diagnostics.csv',index=False,encoding='utf-8-sig')
(OUT/'split_and_floor_log.json').write_text(json.dumps(split_log,ensure_ascii=False,indent=2),encodi
ng='utf-8')
(OUT/'result.json').write_text(json.dumps(summary,ensure_ascii=False,indent=2),encoding='utf-8')
print(json.dumps(summary,ensure_ascii=False,indent=2),flush=True)
```

analysis/A_feasibility/q1_metric_supplement.py

```
"""Supplement existing Q1 OOF predictions; no estimator training.
```

```
AI assistance: OpenAI Codex, 2026-10-25. Labels remain standardized aggregates.
"""
from pathlib import Path
import json
import numpy as np
import pandas as pd
from scipy.stats import spearmanr, t
from sklearn.metrics import mean_absolute_error, r2_score, mean_squared_error
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from matplotlib.font_manager import FontProperties

BASE=Path(__file__).resolve().parent
ROOT=BASE.parents[1]
OUT=BASE/'q1_metrics'
OUT.mkdir(exist_ok=True)
DATA=next(ROOT.glob('2026年第七届*/2026*赛道A*'))/'附件'
xx=pd.read_csv(DATA/'1.csv').set_index('beer')
yy=pd.read_csv(DATA/'4.csv').set_index('beer').loc[xx.index]
Y=yy.drop(columns=['beer_id','tasting_category_fine'])
meta=pd.read_csv(BASE/'day1/sensory_metadata.csv',index_col=0).loc[Y.columns]
types=meta.DataType.to_numpy()
style=xx.tasting_category_fine
loaded=np.load(BASE/'floor_ablation/baseline_oof.npz',allow_pickle=True)
assert np.array_equal(loaded['beer'],Y.index.to_numpy())
assert np.array_equal(loaded['fields'],Y.columns.to_numpy())
preds={'training_mean':np.full((5,len(Y),50),np.nan),
       'style_mean':np.full((5,len(Y),50),np.nan),
       'ExtraTrees':loaded['prediction']}
splits=json.loads((BASE/'floor_ablation/split_and_floor_log.json').read_text(encoding='utf-8'))
seen=np.zeros((5,len(Y)),dtype=int)
fold_rows=[]
for split in splits:
    rep,fold=split['repeat'],split['fold']
    tr=Y.index.get_indexer(split['train_beer']);te=Y.index.get_indexer(split['test_beer'])
    assert (tr>=0).all() and (te>=0).all() and not set(tr)&set(te)
    seen[rep,te]+=1
    ym=Y.iloc[tr].mean().to_numpy()
    preds['training_mean'][rep,te,:]=ym
    group=Y.iloc[tr].groupby(style.iloc[tr]).mean()
    assert set(style.iloc[te])<=set(group.index)
    preds['style_mean'][rep,te,:]=group.loc[style.iloc[te]].to_numpy()
    for name,arr in preds.items():
        pp=arr[rep,te,:]
        fold_rows.append({'repeat':rep,'fold':fold,'model':name,
                          'MAE':float(mean_absolute_error(Y.iloc[te],pp)),
                          'R2':float(r2_score(Y.iloc[te],pp))})
assert np.all(seen==1) and all(np.isfinite(p).all() for p in preds.values())

rows=[];aggregate=[]
for name,arr in preds.items():
    for rep,pp in enumerate(arr):
        mae=mean_absolute_error(Y,pp,multioutput='raw_values')
        r2=r2_score(Y,pp,multioutput='raw_values')
        mse=mean_squared_error(Y,pp,multioutput='raw_values')
        rho=np.array([float(spearmanr(Y.iloc[:,j],pp[:,j]).statistic) for j in range(50)]) if name!
='training_mean' else np.full(50,np.nan)
        for j,col in enumerate(Y):
            rows.append({'model':name,'repeat':rep,'field':col,'type':types[j],

'MAE':float(mae[j]),'MSE':float(mse[j]),'R2':float(r2[j]),'Spearman':float(rho[j])})
        for kind in ['all','numeric','categorical']:
            mask=np.ones(50,dtype=bool) if kind=='all' else types==kind
            aggregate.append({'model':name,'repeat':rep,'group':kind,'MAE':float(mae[mask].mean()),
```

```
                            'R2':float(r2[mask].mean()),
                            'Spearman':float(np.mean(rho[mask])) if name!='training_mean' else
np.nan})
detail=pd.DataFrame(rows)
dim=detail.groupby(['model','field','type'],as_index=False)[['MAE','MSE','R2','Spearman']].mean()
agg=pd.DataFrame(aggregate).groupby(['model','group'],as_index=False)
[['MAE','R2','Spearman']].mean()
model_dim=dim[dim.model=='ExtraTrees'].set_index('field').loc[Y.columns]
style_dim=dim[dim.model=='style_mean'].set_index('field').loc[Y.columns]
model_dim['relative_MAE_gain_vs_style']=(style_dim.MAE-model_dim.MAE)/style_dim.MAE
model_dim['R2_gain_vs_style']=model_dim.R2-style_dim.R2
model_dim['MSE_skill_vs_style']=1-model_dim.MSE/style_dim.MSE
model_dim['MSE_style_mean']=style_dim.MSE
fold_df=pd.DataFrame(fold_rows)
paired=fold_df.pivot(index=['repeat','fold'],columns='model',values='MAE')
absolute=(paired.style_mean-paired.ExtraTrees).to_numpy()
relative=(absolute/paired.style_mean).to_numpy()
ratios=np.array([len(s['test_beer'])/len(s['train_beer']) for s in splits])
def corrected_interval(values):
    m=len(values);avg=float(np.mean(values))
    se=float(np.sqrt((1/m+ratios.mean())*np.var(values,ddof=1)))
    width=float(t.ppf(.975,m-1)*se)
    return {'mean':avg,'approx_95pct_CI':[avg-width,avg+width],'corrected_standard_error':se}
comparison={'absolute_MAE_improvement':corrected_interval(absolute),
            'mean_fold_relative_MAE_improvement':corrected_interval(relative),
            'note':'Supplementary approximate inference on existing 15 paired folds, not independent
confirmation. Mean fold ratios differ slightly from pooled ratio.'}
summ={}
for kind in ['all','numeric','categorical']:
    subset=model_dim if kind=='all' else model_dim[model_dim.type==kind]
    summ[kind]={'n_dimensions':len(subset),
        'R2_distribution':{'min':float(subset.R2.min()),'q25':float(subset.R2.quantile(.25)),
                           'median':float(subset.R2.median()),'q75':float(subset.R2.quantile(.75)),
                           'max':float(subset.R2.max())},
        'Spearman_distribution':
{'min':float(subset.Spearman.min()),'q25':float(subset.Spearman.quantile(.25)),

'median':float(subset.Spearman.median()),'q75':float(subset.Spearman.quantile(.75)),
                           'max':float(subset.Spearman.max())},
        'positive_R2_dimensions':int((subset.R2>0).sum()),
        'dimensions_better_than_style_MAE':int((subset.relative_MAE_gain_vs_style>0).sum()),
        'dimensions_better_than_style_R2':int((subset.R2_gain_vs_style>0).sum())}
global_row=agg[(agg.model=='training_mean')&(agg.group=='all')].iloc[0]
style_row=agg[(agg.model=='style_mean')&(agg.group=='all')].iloc[0]
et_row=agg[(agg.model=='ExtraTrees')&(agg.group=='all')].iloc[0]
result={'protocol':'Existing 5 repeated 3-fold style-stratified splits; training-fold mean and style
mean; no retraining. Per-repeat full-250 OOF metrics then averages.',
        'MAE_relative_gain_vs_global_mean':float((global_row.MAE-et_row.MAE)/global_row.MAE),
        'MAE_relative_gain_vs_style_mean':float((style_row.MAE-et_row.MAE)/style_row.MAE),
        'R2_gain_vs_style_mean':float(et_row.R2-style_row.R2),'groups':summ,
        'style_baseline_comparison_uncertainty':comparison,

'best5_by_R2':model_dim.sort_values('R2',ascending=False).head(5).reset_index().to_dict('records'),
        'worst5_by_R2':model_dim.sort_values('R2').head(5).reset_index().to_dict('records'),
        'note':'Mean-baseline Spearman omitted: each held-out fold has a constant prediction;
pooling different fold constants gives artificial ranks. Group type means metadata type; targets are
not raw binary labels.'}
agg.to_csv(OUT/'aggregate_metrics.csv',index=False,encoding='utf-8-sig')
detail.to_csv(OUT/'per_repeat_dimension_metrics.csv',index=False,encoding='utf-8-sig')
model_dim.to_csv(OUT/'ExtraTrees_dimension_summary.csv',encoding='utf-8-sig')
fold_df.to_csv(OUT/'same_split_baseline_metrics.csv',index=False,encoding='utf-8-sig')
(OUT/'summary.json').write_text(json.dumps(result,ensure_ascii=False,indent=2),encoding='utf-8')

fontpath=Path(r'C:\Windows\Fonts\msyh.ttc')
if fontpath.exists():
```

```
    font=FontProperties(fname=str(fontpath))
    plt.rcParams['font.family']=font.get_name()
plt.rcParams.update({'axes.unicode_minus':False,'font.size':11,'axes.spines.top':False,
                     'axes.spines.right':False,'svg.fonttype':'none'})
fig,axes=plt.subplots(1,2,figsize=(10.5,5.5))
rng=np.random.default_rng(20261025)
colors=['#2E6FAD','#D18432']
for ax,metric,title in zip(axes,['R2','Spearman'],['逐维折外 $R^2$','逐维 Spearman 秩相关']):
    data=[model_dim.loc[model_dim.type==kind,metric].to_numpy() for kind in
['numeric','categorical']]
    bp=ax.boxplot(data,positions=[1,2],widths=.45,patch_artist=True,showfliers=False,
                  medianprops={'color':'#222222','linewidth':1.8})
    for idx,(vals,col,patch) in enumerate(zip(data,colors,bp['boxes'])):
        patch.set_facecolor(col);patch.set_alpha(.15);patch.set_edgecolor(col)

ax.scatter(idx+1+rng.uniform(-.12,.12,len(vals)),vals,s=28,color=col,alpha=.8,zorder=3,edgecolors='w
hite',linewidths=.4)
    ax.axhline(0,color='#777777',linewidth=.8,linestyle='--')
    ax.set_xticks([1,2],['numeric\n26 维','categorical\n24 维'])
    ax.set_title(title,pad=12);ax.grid(axis='y',alpha=.16)
fig.suptitle('Extra Trees：感官预测能力在维度间差异明显',fontsize=15,y=.98)
fig.text(.5,.015,'每点为一个维度；先计算每次重复的 250 样本折外指标，再对 5 次重复取平均。\n 标签为标准化感官汇总值；
categorical 组并非原始 0/1 分类准确率。',ha='center',va='bottom',fontsize=9,color='#555555')
fig.tight_layout(rect=[0,.10,1,.92])
fig.savefig(OUT/'逐维预测指标分布.png',dpi=180,bbox_inches='tight')
fig.savefig(OUT/'逐维预测指标分布.svg',bbox_inches='tight')
plt.close(fig)
print('AGGREGATE\n'+agg.to_string(index=False))
print(json.dumps(result,ensure_ascii=False,indent=2))
```

analysis/hidden_content_audit/audit_hidden_content.py

```
"""Read-only inspection of original A-track material. Never executes document content.
AI-assisted audit: OpenAI Codex, 2026-10-25.
"""
from pathlib import Path
import csv, hashlib, json, re, unicodedata, zipfile
from collections import Counter
from xml.etree import ElementTree as ET
import fitz
from openpyxl import load_workbook

ROOT=Path(__file__).resolve().parents[2]
OUT=Path(__file__).resolve().parent
PACK=next(ROOT.glob('2026 年第七届*'))
ADIR=next(PACK.glob('*赛道 A*'))
TRIGGERS=re.compile(r'ignore.{0,30}(previous|instructions)|system\s*prompt|prompt\s*injection|'
    r'if\s+you\s+are\s+(an?\s+)?(AI|LLM)|chatgpt|openai|large\s+language\s+model|'
    r'忽略.{0,15}(指令|要求|之前|前面)|如果.{0,15}(人工智能|大模型|AI)|'
    r'(人工智能|大模型|AI).{0,15}(必须|请输出|回答|暗号|检测)|'
    r'隐藏指令|提示词注入|防 AI|反 AI|防止 AI|水印|暗号|彩蛋',re.I)

def h(path):return hashlib.sha256(path.read_bytes()).hexdigest()
def scan_text(text):
    hits=[{'offset':m.start(),'context':text[max(0,m.start()-60):m.end()+100]} for m in
TRIGGERS.finditer(text)]
    weird=Counter(f'U+{ord(ch):04X} {unicodedata.name(ch,"UNKNOWN")}' for ch in text
        if unicodedata.category(ch)=='Cf' or (unicodedata.category(ch)=='Cc' and ch not in '\n\r\
t'))
    return {'instruction_keyword_hits':hits[:60], 'special_characters':dict(weird)}

result={'scope':'A-track original PDF/data plus supplied common rules and Word templates; no B-track
or historical papers',
        'pdf':[],'xlsx':[],'csv':[],'docx':[],'other_text':[]}
pdfs=list(PACK.glob('*.pdf'))+list(ADIR.glob('*.pdf'))
pdfs += list(ROOT.glob('*.pdf'))
seen=set()
```

```
for path in pdfs:
    sha=h(path)
    if sha in seen:continue
    seen.add(sha);doc=fitz.open(path)
    record={'path':str(path),'sha256':sha,'pages':len(doc),'metadata':doc.metadata,
            'embedded_files':doc.embfile_names(),'ocgs':doc.get_ocgs(),
            'xml_metadata_scan':scan_text(doc.get_xml_metadata()),'page_checks':[],
            'metadata_scan':scan_text(json.dumps(doc.metadata,ensure_ascii=False)), 'object_flags':
[]}
    for idx,page in enumerate(doc):
        suspicious=[];n=0;covered=[]
        drawings=page.get_drawings()
        for span in page.get_texttrace():
            n+=1
            text=''.join(chr(c[0]) for c in span['chars'] if 0<=c[0]<=0x10ffff)
            if not text.strip():continue
            reasons=[];color=span.get('color',());bbox=fitz.Rect(span['bbox'])
            if span.get('type')==3:reasons.append('invisible_render_mode')
            if span.get('opacity',1)<.05:reasons.append('near_transparent')
            if color and all(v>.94 for v in color):reasons.append('near_white')
            if span.get('size',10)<3:reasons.append('tiny_font_under_3pt')
            if not page.rect.intersects(bbox):reasons.append('outside_page')
            if
reasons:suspicious.append({'text':text,'reasons':reasons,'bbox':list(bbox),'color':color,'size':span
.get('size')})
            for draw in drawings:
                fill=draw.get('fill')
                if (fill and all(v>.94 for v in fill) and draw.get('fill_opacity',1)>.95
                    and draw.get('seqno',-1)>span.get('seqno',10**9) and
draw['rect'].contains(bbox)):

covered.append({'text':text,'bbox':list(bbox),'cover_rect':list(draw['rect'])});break
        annotations=[]
        for ann in page.annots() or []:annotations.append({'type':ann.type,'info':ann.info})
        links=page.get_links()
        text=page.get_text()

record['page_checks'].append({'page':idx+1,'text_spans':n,'suspicious_text_spans':suspicious,

'later_white_cover_candidates':covered,'annotations':annotations,'links':links,'text_scan':scan_text
(text)})
        if path.parent==ADIR:
            page.get_pixmap(matrix=fitz.Matrix(1.3,1.3)).save(OUT/f'A_page_{idx+1}.png')
            (OUT/f'A_page_{idx+1}_extracted.txt').write_text(text,encoding='utf-8')
    for xr in range(1,doc.xref_length()):
        obj=doc.xref_object(xr)
        flags=[token for token in ['/JavaScript','/OpenAction','/AA','/Launch','/ActualText','/Alt
','/EmbeddedFile','/OCProperties'] if token in obj]
        if
flags:record['object_flags'].append({'xref':xr,'flags':flags,'object_excerpt':obj[:1200]})
    data=path.read_bytes();end=data.rfind(b'%%EOF')
    record['bytes_after_final_eof_nonwhitespace']=len(data[end+5:].strip()) if end>=0 else None
    result['pdf'].append(record)

for path in (ADIR/'附件').glob('*.xlsx'):
    book=load_workbook(path,data_only=False,read_only=False)
    rec={'path':str(path),'sha256':h(path),'sheets':
[],'defined_names':list(book.defined_names),'special_package_parts':[], 'package_text_findings':[]}
    for sheet in book:
        sr={'name':sheet.title,'state':sheet.sheet_state,
            'hidden_rows':[i for i,v in sheet.row_dimensions.items() if v.hidden],
            'hidden_columns':[i for i,v in sheet.column_dimensions.items() if v.hidden],
            'comments':[],'suspicious_style_cells':[],'text_findings':[],
            'formulas':0,'conditional_formatting_rules':len(sheet.conditional_formatting)}
        for row in sheet:
```

```
            for cell in row:
                if cell.value is None:continue
                value=str(cell.value);findings=scan_text(value)
                if findings['instruction_keyword_hits'] or
findings['special_characters']:sr['text_findings'].append({'cell':cell.coordinate,**findings})
                if cell.data_type=='f':sr['formulas']+=1
                if
cell.comment:sr['comments'].append({'cell':cell.coordinate,'text':cell.comment.text})
                color=cell.font.color;reasons=[]
                if cell.number_format.replace(' ','') in
[';;;',';;']:reasons.append('hidden_number_format')
                if color and color.type=='rgb' and str(color.rgb)[-
6:].upper()=='FFFFFF':reasons.append('white_font')
                if cell.font.sz is not None and cell.font.sz<3:reasons.append('tiny_font')
                if
reasons:sr['suspicious_style_cells'].append({'cell':cell.coordinate,'value':value[:300],'reasons':re
asons})
        rec['sheets'].append(sr)
    with zipfile.ZipFile(path) as z:
        for name in z.namelist():
            if any(t in name.lower() for t in
['comment','custom','vba','external','embedding','drawing']):rec['special_package_parts'].append(nam
e)
            if name.endswith(('.xml','.rels')):
                raw=z.read(name).decode('utf-8',errors='replace')
                try:visible=' '.join(ET.fromstring(raw).itertext())
                except ET.ParseError:visible=raw
                sf=scan_text(visible)
                if sf['instruction_keyword_hits'] or
sf['special_characters']:rec['package_text_findings'].append({'part':name,**sf})
    result['xlsx'].append(rec)

for path in (ADIR/'附件').glob('*.csv'):
    text=path.read_text(encoding='utf-8-sig');rows=list(csv.reader(text.splitlines()))
    result['csv'].append({'path':str(path),'sha256':h(path),'rows':len(rows)-
1,'columns':len(rows[0]),
        'row_length_variants':dict(Counter(len(r) for r in rows)),**scan_text(text)})

for path in (ADIR/'附件').glob('*.txt'):
    text=path.read_text(encoding='utf-8-sig')
    result['other_text'].append({'path':str(path),'sha256':h(path),**scan_text(text)})

seen=set();ns={'w':'http://schemas.openxmlformats.org/wordprocessingml/2006/main'}
for path in list(PACK.glob('*.docx'))+list(ROOT.glob('*.docx')):
    sha=h(path)
    if sha in seen:continue
    seen.add(sha);rec={'path':str(path),'sha256':sha,'hidden_or_white_runs':[],'package_findings':
[],'special_parts':[]}
    with zipfile.ZipFile(path) as z:
        for name in z.namelist():
            if any(t in name.lower() for t in
['comment','custom','vba','embedding']):rec['special_parts'].append(name)
            if not name.endswith(('.xml','.rels')):continue
            raw=z.read(name).decode('utf-8',errors='replace')
            try:tree=ET.fromstring(raw)
            except ET.ParseError:continue
            sf=scan_text(' '.join(tree.itertext()))
            if sf['instruction_keyword_hits'] or
sf['special_characters']:rec['package_findings'].append({'part':name,**sf})
            for run in tree.findall('.//w:r',ns):
                txt=''.join(run.itertext()).strip()
                if not txt:continue
                hidden=run.find('w:rPr/w:vanish',ns)
                col=run.find('w:rPr/w:color',ns)
                white=col is not None and col.get('{'+ns['w']+'}val','').upper()=='FFFFFF'
```

```
                if hidden is not None or
white:rec['hidden_or_white_runs'].append({'part':name,'text':txt,'hidden_property':hidden is not
None,'white':white})
    result['docx'].append(rec)

(OUT/'audit_details.json').write_text(json.dumps(result,ensure_ascii=False,indent=2,default=str),enc
oding='utf-8')
for kind,items in result.items():
    if isinstance(items,list):print(kind,len(items),flush=True)
print('Report data saved',flush=True)
```

analysis/q2/render_results.py

```
"""Render scientific figures from saved Q2 tables; no model training."""
from pathlib import Path
import argparse,json,textwrap
import numpy as np
import pandas as pd
import networkx as nx
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from matplotlib.font_manager import FontProperties

OUT=Path(__file__).resolve().parent
parser=argparse.ArgumentParser();parser.add_argument('--network-
only',action='store_true');args=parser.parse_args()
font=Path(r'C:\Windows\Fonts\msyh.ttc')
if font.exists():plt.rcParams['font.family']=FontProperties(fname=str(font)).get_name()
plt.rcParams.update({'axes.unicode_minus':False,'font.size':10,'axes.spines.top':False,'axes.spines.
right':False,'svg.fonttype':'none'})
mapping=pd.read_csv(OUT/'chemical_mapping_and_modules.csv').set_index('field')
edges=pd.read_csv(OUT/'network_edges.csv')
g=nx.Graph();g.add_nodes_from(mapping.index)
for row in edges.itertuples():g.add_edge(row.source,row.target,weight=abs(row.rho),rho=row.rho)
active=[n for n in g if g.degree(n)>0];sub=g.subgraph(active)
counts=mapping.groupby('module').size();nontriv=counts[counts>1].sort_values(ascending=False,kind='s
table')
colors=plt.get_cmap('tab20');mod_color={m:colors(i%20) for i,m in enumerate(nontriv.index)}
fig,(ax,bar)=plt.subplots(1,2,figsize=(13,7.4),gridspec_kw={'width_ratios':[2.1,1]})
pos=nx.spring_layout(sub,seed=20261025,weight='weight',iterations=100,k=.27)
positive=[(a,b) for a,b,d in sub.edges(data=True) if d['rho']>=0]
negative=[(a,b) for a,b,d in sub.edges(data=True) if d['rho']<0]
nx.draw_networkx_edges(sub,pos,edgelist=positive,ax=ax,alpha=.20,width=.7,edge_color='#65717c')
nx.draw_networkx_edges(sub,pos,edgelist=negative,ax=ax,alpha=.55,width=.9,edge_color='#bb4848',style
='dashed')
nx.draw_networkx_nodes(sub,pos,ax=ax,node_color=[mod_color[mapping.loc[n,'module']] for n in sub],
    node_size=[25+9*sub.degree(n) for n in sub],edgecolors='white',linewidths=.5,alpha=.9)
ax.set_title(f'相关网络：{len(active)}个非孤立节点、{g.number_of_edges()}条边',pad=12);ax.axis('off')
bar.barh(np.arange(len(nontriv)),nontriv.values,color=[mod_color[m] for m in
nontriv.index],alpha=.85)
bar.set_yticks(np.arange(len(nontriv)),nontriv.index);bar.invert_yaxis();bar.set_xlabel('模块内指标
数');bar.set_title('19个非单点模块',pad=12)
for i,n in enumerate(nontriv.values):bar.text(n+.25,i,str(n),va='center',fontsize=9)
bar.set_xlim(0,float(nontriv.max())+3);bar.grid(axis='x',alpha=.15)
fig.suptitle('化合物相关网络与固定模块划分',fontsize=16,y=.98)
fig.text(.5,.018,'附件2；保留 |Spearman ρ| ≥ 0.6；Louvain，固定随机种子；字段名称见配套映射表。\n101个孤立节点
未绘入左图，但保留在映射表和稳定性统计中；红虚线为负相关，相关边不表示因果或反应路
径。',ha='center',fontsize=9,color='#555555')
fig.tight_layout(rect=[0,.08,1,.93]);fig.savefig(OUT/'化合物网络与模
块.png',dpi=180,bbox_inches='tight');fig.savefig(OUT/'化合物网络与模
块.svg',bbox_inches='tight');plt.close(fig)
if args.network_only:raise SystemExit(0)

top=pd.read_csv(OUT/'top3_chemical_candidates.csv');targets=top.target.drop_duplicates().tolist()
chemical=top.pivot(index='target',columns='rank',values='top3_frequency').loc[targets].to_numpy()
modules=top.pivot(index='target',columns='rank',values='module_top3_frequency').loc[targets].to_nump
```

```
y()
fields=top.pivot(index='target',columns='rank',values='field').loc[targets]
module_names=top.pivot(index='target',columns='rank',values='module').loc[targets]
quality=top.groupby('target').Q1_R2.first()
fig,axes=plt.subplots(1,2,figsize=(13,17),gridspec_kw={'width_ratios':[1.5,1]})
for idx,(ax,data,title) in enumerate(zip(axes,[chemical,modules],['化合物 Top-3 入选频率','对应模块命中频
率'])):
    im=ax.imshow(data,vmin=0,vmax=1,cmap='Blues',aspect='auto')
    ax.set_xticks([0,1,2],['Top-1','Top-2','Top-
3']);ax.xaxis.tick_top();ax.set_title(title,pad=30,fontsize=13)
    if idx==0:ax.set_yticks(range(50),[t+(' †' if quality[t]<.1 else '') for t in
targets],fontsize=8)
    else:ax.set_yticks([])
    for row in range(50):
        for col in range(3):
            label=('\n'.join(textwrap.wrap(fields.iloc[row,col],24))+f'\n{data[row,col]:.0%}') if
idx==0 else f'{module_names.iloc[row,col]}  {data[row,col]:.0%}'
            ax.text(col,row,label,ha='center',va='center',fontsize=6.5 if idx==0 else 8,
                    color='white' if data[row,col]>.62 else '#17222e')

ax.set_yticks(np.arange(-.5,50,1),minor=True);ax.grid(which='minor',axis='y',color='white',linewidth
=.7);ax.tick_params(which='minor',left=False)
fig.suptitle('100 次分层 bootstrap：化合物与模块两级稳定性',fontsize=16,y=.99)
fig.text(.5,.013,'每次袋外置换 3 次；主名单按平均袋外 MAE 增量排序。† 表示 Q1 平均 R²<0.1，名单仅作探索。\n 模块每次最
多计一次命中；大模块更容易被命中；这些频率不是因果效应或显著性概率。',ha='center',fontsize=9,color='#555555')
fig.tight_layout(rect=[0,.045,1,.965]);fig.savefig(OUT/'Top3 与模块稳定
性.png',dpi=170,bbox_inches='tight');plt.close(fig)
print('Figures saved.')
```

analysis/q2/run_q2.py

```
"""Fixed Q2 statistical pipeline. Never executes downloaded reference code.
AI assistance: OpenAI Codex, 2026-10-25. See 方法与运行协议.md.
"""
from pathlib import Path
from time import perf_counter
import json,re,unicodedata,hashlib,argparse
import numpy as np
import pandas as pd
import networkx as nx
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import QuantileTransformer
from sklearn.cross_decomposition import PLSRegression
from sklearn.ensemble import ExtraTreesRegressor
from threadpoolctl import threadpool_limits

OUT=Path(__file__).resolve().parent
ROOT=OUT.parents[1]
DATA=next(ROOT.glob('2026 年第七届*/2026*赛道 A*'))/'附件'
parser=argparse.ArgumentParser();parser.add_argument('--limit',type=int,default=100)
args=parser.parse_args();N=100;R=3;SEED=20261025
assert 1<=args.limit<=N
threadpool_limits(limits=2)
x=pd.read_csv(DATA/'1.csv').set_index('beer')
y=pd.read_csv(DATA/'4.csv').set_index('beer').loc[x.index]
X=x.drop(columns=['beer_id','tasting_category_fine']);Y=y.drop(columns=['beer_id','tasting_category_
fine'])
fields=list(X);targets=list(Y);A=X.to_numpy(float);YY=Y.to_numpy(float)
threshold=pd.read_excel(DATA/'8.xlsx',sheet_name='Sensory thresholds')
provided=pd.read_csv(DATA/'2.csv',index_col=0)

def repaired(s):
    s=str(s)
    if any(c in s for c in ['Î','Ã','Â']):
        try:s=s.encode('latin1').decode('utf-8')
        except (UnicodeError,LookupError):pass
    return unicodedata.normalize('NFKC',s).strip()
```

```
def normalized(s):
    s=repaired(s).lower()
    for a,b in [('α','alpha'),('β','beta'),('γ','gamma'),('δ','delta')]:s=s.replace(a,b)
    return re.sub(r'[^a-z0-9]','',s)
name_index={}
for lab in provided.index:name_index.setdefault(normalized(lab),[]).append(lab)
mapping=[]
for field in fields:
    rr=threshold[threshold.field_name_R==field]
    assert len(rr)==1,(field,len(rr))
    rr=rr.iloc[0]
    candidates=set()
    for key in ['chemical_name','chemical_name_short']:
        candidates.update(name_index.get(normalized(rr[key]),[]))
    assert len(candidates)==1,(field,candidates)
    name=next(iter(candidates))
    mapping.append({'field':field,'matrix_name':name,'reference_name':repaired(rr.chemical_name),
        'origin':rr.origin_paper,'unit':None if pd.isna(rr.unit) else rr.unit,
        'mapping_method':'reference_names_with_punctuation_and_Greek_encoding_normalization'})
assert len(set(r['matrix_name'] for r in mapping))==231
C=provided.loc[[r['matrix_name'] for r in mapping],[r['matrix_name'] for r in
mapping]].to_numpy(float)
Craw=X.corr(method='spearman').to_numpy()
maxerr=float(np.nanmax(abs(C-Craw)))
assert maxerr<1e-12,maxerr
G=nx.Graph();G.add_nodes_from(range(231))
for i in range(231):
    for j in range(i+1,231):
        if np.isfinite(C[i,j]) and abs(C[i,j])>=.6:
            G.add_edge(i,j,weight=float(abs(C[i,j])),rho=float(C[i,j]))
communities=nx.community.louvain_communities(G,weight='weight',resolution=1,seed=SEED)
communities=sorted(communities,key=lambda s:(-len(s),min(fields[i] for i in s)))
module=np.empty(231,dtype=int)
for k,comm in enumerate(communities):
    for j in comm:module[j]=k
exclude={'pH':'physical_parameter','color.EBC.':'physical_parameter','bitterness':'physical_paramete
r',
    'CO2.PSI.':'pressure_measurement','GFS':'mixture','unfermentables':'mixture',
    'protein.g.L.':'mixture','betaglucan.mg.L.':'polymer_class','iron':'element_metric',

'kcalperc':'derived','esters_sum':'derived','aroma_hops_sum':'derived','acids_sum':'derived','sulfur
_sum':'derived'}
eligible=np.array([j for j,f in enumerate(fields) if f not in exclude],dtype=int)
for j,row in enumerate(mapping):
    row.update({'module':f'M{module[j]+1:02d}','degree':G.degree(j),
                'eligible_primary_top3':fields[j] not in
exclude,'exclusion_reason':exclude.get(fields[j],'')})
pd.DataFrame(mapping).to_csv(OUT/'chemical_mapping_and_modules.csv',index=False,encoding='utf-8-
sig')
edge_rows=[{'source':fields[i],'target':fields[j],'rho':v['rho'],'weight':v['weight']} for i,j,v in
G.edges(data=True)]
pd.DataFrame(edge_rows).to_csv(OUT/'network_edges.csv',index=False,encoding='utf-8-sig')
graph_info={'n_nodes':231,'n_edges':G.number_of_edges(),'n_modules':len(communities),
            'n_singletons':sum(len(c)==1 for c in communities),'module_sizes':[len(c) for c in
communities],
            'name_mapping_count':len(mapping),'matrix_spearman_max_error':maxerr,'rho_threshold':.6,
            'eligible_compounds':len(eligible)}
(OUT/'graph_summary.json').write_text(json.dumps(graph_info,ensure_ascii=False,indent=2),encoding='u
tf-8')
print('GRAPH '+json.dumps(graph_info,ensure_ascii=False),flush=True)

signature=hashlib.sha256((OUT/'方法与运行协议.md').read_bytes()).hexdigest()
checkpoint=OUT/'bootstrap_checkpoint.npz'
shape=(N,50,231)
if checkpoint.exists():
```

```
    old=np.load(checkpoint,allow_pickle=False)
    assert str(old['protocol_sha'])==signature

imp_et=old['extra_trees'];imp_pls=old['pls'];completed=int(old['completed']);elapsed=float(old['elap
sed_seconds'])
    logs=json.loads((OUT/'bootstrap_samples.json').read_text(encoding='utf-8'))
else:
    imp_et=np.full(shape,np.nan,dtype=np.float32);imp_pls=np.full(shape,np.nan,dtype=np.float32)
    completed=0;elapsed=0.;logs=[]
groups=[np.flatnonzero(x.tasting_category_fine.to_numpy()==s) for s in
sorted(x.tasting_category_fine.unique())]
start=perf_counter()
for b in range(completed,args.limit):
    rng=np.random.default_rng(SEED+1000+b)
    train=np.concatenate([rng.choice(g,size=len(g),replace=True) for g in groups])
    oob=np.setdiff1d(np.arange(len(X)),np.unique(train))
    assert len(oob)>0 and not set(train)&set(oob)

imputer=SimpleImputer(strategy='median');at=imputer.fit_transform(A[train]);bt=imputer.transform(A[o
ob])

et=ExtraTreesRegressor(n_estimators=160,min_samples_leaf=3,max_features=.7,random_state=SEED+b,n_job
s=2)
    et.fit(at,YY[train]);ep=et.predict(bt);eb=np.mean(abs(ep-YY[oob]),axis=0)
    qt=QuantileTransformer(n_quantiles=100,output_distribution='normal',random_state=SEED+b)
    pat=qt.fit_transform(at);pbt=qt.transform(bt)
    pls=PLSRegression(n_components=8,scale=False,max_iter=1000)
    pls.fit(pat,YY[train]);pp=pls.predict(pbt);pb=np.mean(abs(pp-YY[oob]),axis=0)
    coef=pls.coef_
    if coef.shape==(231,50):coef=coef.T
    assert coef.shape==(50,231)
    n=len(oob)
    for first in range(0,231,12):
        cols=list(range(first,min(first+12,231)))
        batch=np.tile(bt,(len(cols)*R,1));pi=np.empty((len(cols),R,50))
        for u,j in enumerate(cols):
            for r in range(R):
                perm=rng.permutation(n);sl=slice((u*R+r)*n,(u*R+r+1)*n)
                batch[sl,j]=bt[perm,j]
                diff=pbt[perm,j]-pbt[:,j]
                predicted=pp+diff[:,None]*coef[:,j][None,:]
                pi[u,r]=np.mean(abs(predicted-YY[oob]),axis=0)-pb
                if b==0 and first==0 and u==0 and r==0:
                    check=pbt.copy();check[:,j]=pbt[perm,j]
                    assert np.max(abs(pls.predict(check)-predicted))<1e-8
        perturbed=et.predict(batch).reshape(len(cols),R,n,50)
        delta=np.mean(abs(perturbed-YY[oob][None,None,:,:]),axis=(1,2))-eb[None,:]
        imp_et[b][:,cols]=delta.T
        imp_pls[b][:,cols]=pi.mean(axis=1).T
    assert np.isfinite(imp_et[b]).all() and np.isfinite(imp_pls[b]).all()
    logs.append({'bootstrap':b,'train_beer_with_replacement':X.index[train].tolist(),
                 'oob_beer':X.index[oob].tolist(),'n_oob':n,
                 'ET_oob_macro_MAE':float(eb.mean()),'PLS_oob_macro_MAE':float(pb.mean())})
    completed=b+1
    if completed%5==0 or completed==args.limit:
        total=elapsed+perf_counter()-start
        print(f'BOOTSTRAP {completed}/100; elapsed={total:.1f}s; mean OOB count={np.mean([a["n_oob"]
for a in logs]):.1f}',flush=True)
    if completed%10==0 or completed==args.limit:
        np.savez_compressed(checkpoint,extra_trees=imp_et,pls=imp_pls,completed=completed,
                            elapsed_seconds=elapsed+perf_counter()-
start,protocol_sha=np.array(signature))

(OUT/'bootstrap_samples.json').write_text(json.dumps(logs,ensure_ascii=False,indent=2),encoding='utf
-8')
```

```
if completed<100:
    print('Checkpoint saved; resume with --limit 100.',flush=True)
    raise SystemExit(0)

def top_indices(values):return eligible[np.argsort(-values[:,eligible],axis=1,kind='stable')[:,:3]]
tops_et=np.stack([top_indices(v) for v in imp_et]);tops_pls=np.stack([top_indices(v) for v in
imp_pls])
mean_et=imp_et.mean(axis=0);mean_pls=imp_pls.mean(axis=0)
final_et=top_indices(mean_et);final_pls=top_indices(mean_pls)
freq=np.zeros((50,231));modulefreq=np.zeros((50,len(communities)))
for b in range(100):
    for t in range(50):
        freq[t,tops_et[b,t]]+=.01
        modulefreq[t,np.unique(module[tops_et[b,t]])]+=.01
quality=pd.read_csv(ROOT/'analysis/A_feasibility/q1_metrics/ExtraTrees_dimension_summary.csv',index_
col=0).loc[targets]
observed=pd.concat([X,Y.add_prefix('target__')],axis=1).corr(method='spearman').iloc[:231,231:].to_n
umpy()
rows=[];target_rows=[]
for t,target in enumerate(targets):
    jaccards=[]
    for b in range(100):
        a,c=set(tops_et[b,t]),set(tops_pls[b,t]);jaccards.append(len(a&c)/len(a|c))
    a,c=set(final_et[t]),set(final_pls[t]);jaccard=len(a&c)/len(a|c)
    target_rows.append({'target':target,'Q1_R2':float(quality.loc[target,'R2']),
        'Q1_Spearman':float(quality.loc[target,'Spearman']),
        'ET_top3':' | '.join(fields[j] for j in final_et[t]),
        'PLS_top3':' | '.join(fields[j] for j in final_pls[t]),
        'Jaccard_final':jaccard,'Jaccard_bootstrap_mean':float(np.mean(jaccards)),
        'min_top3_frequency':float(min(freq[t,final_et[t]])),
        'exploratory_Q1_R2_below_0_1':bool(quality.loc[target,'R2']<.1)})
    for rank,j in enumerate(final_et[t],1):
        vals=imp_et[:,t,j];info=mapping[j]

rows.append({'target':target,'rank':rank,'field':fields[j],'chemical_name':info['matrix_name'],
            'origin':info['origin'],'unit':info['unit'],'module':info['module'],
            'mean_OOB_permutation_MAE_increase':float(vals.mean()),
            'bootstrap_percentile_2_5':float(np.quantile(vals,.025)),
            'bootstrap_percentile_97_5':float(np.quantile(vals,.975)),

'top3_frequency':float(freq[t,j]),'module_top3_frequency':float(modulefreq[t,module[j]]),
            'observed_spearman_descriptive':float(observed[j,t]),
            'Q1_R2':float(quality.loc[target,'R2']),
            'flag':'exploratory_weak_predictor' if quality.loc[target,'R2']<.1 else
('low_selection_stability' if freq[t,j]<.5 else 'candidate')})
pd.DataFrame(rows).to_csv(OUT/'top3_chemical_candidates.csv',index=False,encoding='utf-8-sig')
td=pd.DataFrame(target_rows);td.to_csv(OUT/'target_stability_and_jaccard.csv',index=False,encoding='
utf-8-sig')
pd.DataFrame(freq,index=targets,columns=fields).to_csv(OUT/'chemical_top3_frequencies.csv',encoding=
'utf-8-sig')
pd.DataFrame(modulefreq,index=targets,columns=[f'M{k+1:02d}' for k in
range(len(communities))]).to_csv(OUT/'module_top3_frequencies.csv',encoding='utf-8-sig')
pd.DataFrame(mean_et,index=targets,columns=fields).to_csv(OUT/'all_factor_mean_permutation_importanc
e.csv',encoding='utf-8-sig')
summary={'bootstrap_completed':100,'permutation_repeats':3,'graph':graph_info,
         'elapsed_seconds':elapsed+perf_counter()-start,
         'median_final_Jaccard':float(td.Jaccard_final.median()),
         'mean_final_Jaccard':float(td.Jaccard_final.mean()),
         'targets_with_all_top3_frequency_at_least_0_5':int((td.min_top3_frequency>=.5).sum()),
         'targets_with_all_top3_frequency_at_least_0_6':int((td.min_top3_frequency>=.6).sum()),
         'targets_Q1_R2_below_0_1':int(td.exploratory_Q1_R2_below_0_1.sum()),
         'protocol_sha256':signature,'note':'Statistical candidates, not causal drivers; source
graph is descriptive only.'}
(OUT/'summary.json').write_text(json.dumps(summary,ensure_ascii=False,indent=2),encoding='utf-8')
print('SUMMARY '+json.dumps(summary,ensure_ascii=False),flush=True)
```

analysis/q2/summarize_results.py

```
"""Assemble a readable Q2 statistical report from completed saved results."""
from pathlib import Path
import json
import numpy as np
import pandas as pd

OUT=Path(__file__).resolve().parent
s=json.loads((OUT/'summary.json').read_text(encoding='utf-8'))
assert s['bootstrap_completed']==100
top=pd.read_csv(OUT/'top3_chemical_candidates.csv')
target=pd.read_csv(OUT/'target_stability_and_jaccard.csv').set_index('target')
mapping=pd.read_csv(OUT/'chemical_mapping_and_modules.csv').set_index('field')
imps=pd.read_csv(OUT/'all_factor_mean_permutation_importance.csv',index_col=0)
freq=pd.read_csv(OUT/'chemical_top3_frequencies.csv',index_col=0)
mfreq=pd.read_csv(OUT/'module_top3_frequencies.csv',index_col=0)
assert len(top)==150 and top.groupby('target')['rank'].apply(lambda v:sorted(v)==[1,2,3]).all()
assert top.field.isin(mapping[mapping.eligible_primary_top3].index).all()
assert freq.to_numpy().min()>=0 and freq.to_numpy().max()<=1+1e-9
assert np.allclose(freq.sum(axis=1),3)
assert mfreq.to_numpy().min()>=0 and mfreq.to_numpy().max()<=1+1e-9
assert all(mfreq.loc[r.target,r.module]+1e-9>=r.top3_frequency for r in top.itertuples())
aux=[]
for field,row in imps.iterrows():
    for rank,compound in enumerate(row.sort_values(ascending=False,kind='stable').index[:3],1):
        aux.append({'target':field,'rank':rank,'field':compound,'importance':float(row[compound]),
                    'eligible_primary_top3':bool(mapping.loc[compound,'eligible_primary_top3'])})
pd.DataFrame(aux).to_csv(OUT/'supplementary_all_factor_top3.csv',index=False,encoding='utf-8-sig')

lines=['# Q2 网络、关键化合物与稳定性：统计主流程结果','',
       '日期：2026-10-25。已完成 100 次分层 bootstrap、每列 3 次袋外置换、Extra Trees 与稳健 PLS 对照。本文给出统计候选和稳定性，尚不能替代逐项化学机理核查。','',
       '## 网络与名称核验','',
       f'231 个指标全部通过附件 8 名称映射，修复希腊字母的可逆乱码后，与附件 2 逐项对齐。对齐矩阵与附件 1 重算 Spearman 的最大差异为{s["graph"]["matrix_spearman_max_error"]:.2g}。没有使用附件 6 标签相关性筛选变量。','',
       '固定|rho|≥0.6 阈值，得到 231 个节点、266 条边（3 条负相关边），Louvain 划分为 19 个非单点模块与 101 个单点模块。101 个孤立节点仍参与模型和统计，只为图的可读性未在左图绘制。阈值没有按输出结果调整。','',
       '![化合物网络与模块](化合物网络与模块.png)','',
       '网络仅用于描述和把每次名单映射到固定模块，不进入预测拟合。模块频率是固定划分下的名单命中率，并非模块划分本身在重采样中的稳定性。','',
       '## 评价方法','',
       '每次按 22 种风格分层有放回重采样 250 条训练记录，训练折中位数插补，未抽中的样本作为袋外评价集。两个模型使用同一重采样及同一置换顺序；置换重要性为 MAE 增加量，逐目标计算。保存全部训练/袋外编号，训练与袋外编号不交叉。','',
       'Extra Trees 参数与 Q1 保持一致；对照为已有 8 成分稳健 PLS。PLS 是固定线性预测器，其置换预测以系数精确计算，并用显式置换预测校验；没有以系数大小冒充验证集置换重要性。','',
       '模型输入保留全部 231 列。主名单在 217 个单体化学候选中排序，排除 14 个派生、物理、混合或元素指标；完整因子 Top-3 另存 supplementary_all_factor_top3.csv，不隐去模型对这些指标的依赖。','',
       '主名单按 100 次平均袋外置换 MAE 增量排序；入选频率是每次重拟合后进入前三的比例。模块每次最多命中一次。同模块可出现多个化合物，不强制分散；大模块命中机会较大。','',
       '逐列置换仍受共线性及不真实化学组合的影响，重采样不能自动消除这种偏差。重要性可以为负，不截断成零。重要性的 2.5% 和 97.5%数值是重采样分布百分位，不称为因果效应置信区间。','',
       '## 稳定性概况','',
       f'- 50 个目标均输出 3 个候选，共 150 条主结果。',
       f'- 三个候选入选频率均不低于 50%的目标：{s["targets_with_all_top3_frequency_at_least_0_5"]}/50。',
       f'- 三个候选入选频率均不低于 60%的目标：{s["targets_with_all_top3_frequency_at_least_0_6"]}/50。',
       f'- 两模型最终 Top-3 的 Jaccard 中位数：{s["median_final_Jaccard"]:.3f}；均值：{s["mean_final_Jaccard"]:.3f}。',
       f'- Q1 平均 $R^2$ 低于 0.1 的目标：{s["targets_Q1_R2_below_0_1"]}/50，清单明确标记为探索性。','',
       '50%和 60%仅作稳定性描述，不是显著性阈值。跨方法一致性与重采样稳定性不是同一件事，不因某个候选重复出现就把它称为真实驱动。','',
       '![Top3 与模块稳定性](Top3 与模块稳定性.png)','',
       '## 代表性目标','']
selected=['acidity','bitternes','alcohol','A_hops_citrus','A_esters_isoaa','diacetyl','dms','X4vg','t2n','stale_hops']
for name in selected:
```

```
    g=top[top.target==name].sort_values('rank');tt=target.loc[name]
    lines += [f'### {name}：Q1 R²={tt.Q1_R2:.3f}，跨方法 Jaccard={tt.Jaccard_final:.2f}','',
              '| 排名 | 化合物 | 袋外 MAE 增量 | 入选频率 | 模块 | 模块命中频率 |','|---|---|---:|---:|---|---:|']
    for r in g.itertuples():
        lines.append(f'| {r.rank} | {r.chemical_name} | {r.mean_OOB_permutation_MAE_increase:.5f} | {r.top3_frequency:.0%} | {r.module} | {r.module_top3_frequency:.0%} |')
    lines += ['', '上表是模型关联候选，不能仅凭化合物名称判定直接感官机制。低预测目标尤其不能据此作干预保证。','']
lines += ['## 全 50 维主名单','',
          '| 感官目标 | Q1 R² | Top-1（入选率） | Top-2（入选率） | Top-3（入选率） | 最终 Jaccard |',
          '|---|---:|---|---|---|---:|']
for name in target.index:
    g=top[top.target==name].sort_values('rank');cells=[f'{r.field}（{r.top3_frequency:.0%}）' for r in g.itertuples()]
    lines.append(f'| {name} | {target.loc[name,"Q1_R2"]:.3f} | '+ ' | '.join(cells)+f' | {target.loc[name,"Jaccard_final"]:.2f} |')
lines += ['', '## 对下一阶段的约束','',
          '保留低稳定性或低预测可靠性目标，明确其探索性，不为得到更好数字追加模型、重选图阈值或重抽随机种子。未做的组置换、SHAP 和图阈值敏感性不写成已经验证。',
          '第三问只能把稳定名单作为候选化学关系与模块约束的辅助，不能把置换重要性当成因果导数。DMS 等缺陷仍优先使用真实单位浓度阈值，Stale hops 的弱感官代理不能构成严格保证。','',
          '本次统计完成后，下一步按较高预测可靠性目标和实际 Top-3 逐项核对化学作用、单位及阈值；尚未完成的机理文献核查不得写作已证实。']
(OUT/'Q2 统计主流程结果.md').write_text('\n'.join(lines)+'\n',encoding='utf-8')
print(json.dumps(s,ensure_ascii=False,indent=2))
print(top[top.target.isin(['acidity','bitternes','alcohol','dms','stale_hops'])]
[['target','field','top3_frequency','module_top3_frequency']].to_string(index=False))
```

analysis/q2/supplement_reliability.py

```
"""Reorganize saved Q1/Q2 outputs; no predictive refitting or new bootstrap.
AI assistance: OpenAI Codex, 2026-10-26.
"""
from pathlib import Path
import json
import numpy as np
import pandas as pd
from scipy.stats import spearmanr
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

OUT=Path(__file__).resolve().parent
ROOT=OUT.parents[1]
q=pd.read_csv(ROOT/'analysis/A_feasibility/q1_metrics/ExtraTrees_dimension_summary.csv',index_col=0)
top=pd.read_csv(OUT/'top3_chemical_candidates.csv')
mapping=pd.read_csv(OUT/'chemical_mapping_and_modules.csv').set_index('field')
counts=mapping.groupby('module').size()
fields=list(mapping.index);eligible=np.flatnonzero(mapping.eligible_primary_top3.to_numpy())
targets=list(pd.read_csv(OUT/'all_factor_mean_permutation_importance.csv',index_col=0).index)
q=q.loc[targets]

def wilson(p,n=100):
    z=1.959963984540054;den=1+z*z/n
    center=(p+z*z/(2*n))/den
    width=z*np.sqrt(p*(1-p)/n+z*z/(4*n*n))/den
    return center-width,center+width

rows=[]
for name in targets:
    a=top[top.target==name].sort_values('rank')
    probs=a.top3_frequency.to_numpy();lo,hi=wilson(probs)
    modules=a[['module','module_top3_frequency']].drop_duplicates('module')
    nonsingle=modules[modules.module.map(counts)>1]
    maxmodule=float(nonsingle.module_top3_frequency.max()) if len(nonsingle) else 0.
    skill=float(q.loc[name,'MSE_skill_vs_style'])
    A=(skill>0 and probs.min()>=.5-1e-10 and (a.mean_OOB_permutation_MAE_increase>0).all())
```

```
    B=(not A and skill>0 and maxmodule>=.7-1e-10)
    tier='A' if A else ('B' if B else 'C')
    reason=('相对风格均值有 MSE 提升；三个候选均有至少 50%入选率' if A else
            ('相对风格均值有 MSE 提升；至少一个非单点模块命中率≥70%' if B else
             ('未优于风格均值' if skill<=0 else '单体及非单点模块未达到预设描述门槛')))
    rows.append({'target':name,'tier':tier,'Q1_R2':float(q.loc[name,'R2']),

'Q1_MSE_skill_vs_style':skill,'Q1_relative_MAE_gain_vs_style':float(q.loc[name,'relative_MAE_gain_vs
_style']),

'mean_final_top3_selection_frequency':float(probs.mean()),'min_final_top3_selection_frequency':float
(probs.min()),
        'max_non_singleton_module_frequency':maxmodule,
        'n_top3_Wilson_intervals_covering_0_5':int(((lo<=.5)&(hi>=.5)).sum()),'reason':reason})
    for j,(idx,r) in enumerate(a.iterrows()):
        top.loc[idx,'frequency_MC_Wilson_lower']=lo[j]
        top.loc[idx,'frequency_MC_Wilson_upper']=hi[j]
        top.loc[idx,'tier']=tier
df=pd.DataFrame(rows)
df.to_csv(OUT/'reliability_tiers.csv',index=False,encoding='utf-8-sig')
top.to_csv(OUT/'top3_with_MC_frequency_intervals.csv',index=False,encoding='utf-8-sig')

cp=np.load(OUT/'bootstrap_checkpoint.npz')
imps={key:cp[key].mean(0) for key in ['extra_trees','pls']}
overlap=[]
def jac(a,b):return len(a&b)/len(a|b)
for k in [3,5,10]:
    ranks={key:eligible[np.argsort(-v[:,eligible],axis=1,kind='stable')[:,:k]] for key,v in
imps.items()}
    for j,name in enumerate(targets):
        a=set(ranks['extra_trees'][j]);b=set(ranks['pls'][j])
        ma=set(mapping.iloc[list(a)].module);mb=set(mapping.iloc[list(b)].module)

overlap.append({'target':name,'top_k':k,'chemical_Jaccard':jac(a,b),'module_Jaccard':jac(ma,mb),
                        'ET_unique_modules':len(ma),'PLS_unique_modules':len(mb)})
ov=pd.DataFrame(overlap);ov.to_csv(OUT/'relaxed_cross_method_jaccard.csv',index=False,encoding='utf-
8-sig')
assoc={name:float(spearmanr(df[col],df.mean_final_top3_selection_frequency).statistic)
       for name,col in [('skill_vs_selection','Q1_MSE_skill_vs_style'),('R2_vs_selection','Q1_R2')]}
summary={'date':'2026-10-26','n_dimensions':len(df),'tier_counts':df.tier.value_counts().to_dict(),
    'descriptive_Spearman_across_dimensions':assoc,
    'jaccard_medians':ov.groupby('top_k')
[['chemical_Jaccard','module_Jaccard']].median().to_dict('index'),
    'jaccard_means':ov.groupby('top_k')
[['chemical_Jaccard','module_Jaccard']].mean().to_dict('index'),
    'retrained_models':0,'style_confounding_sensitivity_completed':False,
    'tier_rule':'A: skill>0, min chemical frequency>=.5, positive average importance for all 3; B:
not A, skill>0, at least one non-singleton module frequency>=.7; C: remaining',
    'qualification':'Post-analysis descriptive prioritization, not independent validation or causal
evidence; Wilson intervals quantify 100-run conditional Monte Carlo precision only.'}
(OUT/'reliability_supplement_summary.json').write_text(json.dumps(summary,ensure_ascii=False,indent=
2),encoding='utf-8')
plt.rcParams.update({'font.family':'Microsoft YaHei','axes.unicode_minus':False,'font.size':10})
colors={'A':'#168377','B':'#d28b20','C':'#89939c'}
fig,axes=plt.subplots(1,2,figsize=(12.2,5.4),sharey=True)
for ax,col,label in zip(axes,['Q1_MSE_skill_vs_style','Q1_R2'],['Q1 相对风格均值的 MSE skill','Q1 逐维
R²']):
    for tier in ['A','B','C']:
        a=df[df.tier==tier]

ax.scatter(a[col],a.mean_final_top3_selection_frequency,c=colors[tier],s=42,alpha=.85,label=f'{tier}
档（{len(a)}维）',edgecolors='white',linewidth=.5)
    ax.axvline(0,color='#b7bdc4',lw=.8);ax.set_xlabel(label);ax.set_ylim(0,.9);ax.grid(alpha=.16)
    for name in ['acidity','alcohol','stale_hops']:
        a=df[df.target==name].iloc[0]
```

```
        ax.annotate(name,(a[col],a.mean_final_top3_selection_frequency),
                    xytext=(-6,6) if name=='acidity' else (5,6),
                    ha='right' if name=='acidity' else 'left',textcoords='offset points',fontsize=8)
    rho=assoc['skill_vs_selection' if col.startswith('Q1_MSE') else 'R2_vs_selection']
    ax.set_title(f'跨 50 个维度的描述性 Spearman ρ={rho:.3f}',fontsize=11)
axes[0].set_ylabel('最终 Top-3 的平均 bootstrap 入选率')
axes[1].legend(frameon=False,loc='upper left')
fig.suptitle('可预测性与关键因素稳定性：使用已有折外及重采样结果',fontsize=13)
fig.text(.5,.025,'分档用于安排解释优先级；没有排除风格混杂，也不表示因果可信等
级。',ha='center',fontsize=9,color='#59636d')
fig.tight_layout(rect=(0,.06,1,.94));fig.savefig(OUT/'可预测性与Top3稳定
性.png',dpi=190);fig.savefig(OUT/'可预测性与Top3稳定性.svg');plt.close(fig)

lines=['# Q2 可靠性分档与已有结果补充','', '日期：2026-10-26。本次重训模型 0 个；没有重做 bootstrap 或风格残差
化。','',
'## 描述性分档规则','',
'分档在已看到总体稳定性结果后制定，用于论文解释和人工核查的优先级，不称为预注册检验、显著性或因果证据。',
'', '- A：Q1 相对风格均值 MSE skill>0；最终三个候选平均置换重要性均为正；三个入选率均≥50%。可以重点报告稳定的模型关
联，仍需化学核查。',
'- B：未入 A；skill>0；最终候选涉及的至少一个非单点模块命中率≥70%。报告模块线索，不把整个 Top-3 当作稳定发现。',
'- C：其余目标。保留候选清单，但不据此下关键化学因素结论。',
'', '这些是描述门槛，不是客观置信度。大模块命中机会更高，模块命中不等于社区划分稳定；B 不能解释为已经验证共线替换。','',
f'本次分档：'+ '、'.join(f'{t}档{int((df.tier==t).sum())}维' for t in ['A','B','C'])+'。','',
'![可预测性与稳定性](可预测性与Top3稳定性.png)','',
f'跨 50 维的描述性 Spearman：MSE skill 对入选率={assoc["skill_vs_selection"]:.3f}，R² 对入选率=
{assoc["R2_vs_selection"]:.3f}。维度彼此相关，不作 50 个独立重复的显著性检验；正相关也不能证明弱预测是低稳定性的唯
一原因。','',
'## 跨模型一致性','', '| Top-k | 化合物 Jaccard 中位数 | 模块 Jaccard 中位数 | 化合物均值 | 模块均值 |','|---|---:
|---:|---:|---:|']
for k,g in ov.groupby('top_k'):
    lines.append(f'| {k} | {g.chemical_Jaccard.median():.3f} | {g.module_Jaccard.median():.3f} |
{g.chemical_Jaccard.mean():.3f} | {g.module_Jaccard.mean():.3f} |')
lines += ['', '比较对象为 Extra Trees 与 8 成分 PLS，均按已有袋外置换 MAE 增量排序，在同一 217 个单体化学候选内截取
Top-k。模块 Jaccard 先对各名单去重映射到固定模块，再求交并比。模块压缩与大模块本身会提高重叠，不能单独证实共线替
换。','',
'## 入选率的计算精度','',
'100 次重采样下，p=50%的二项 Monte Carlo 标准误约 5 个百分点。表中增加 Wilson 区间，仅近似描述给定数据和协议下重复 100
次的计算精度；不是未知真实化合物效应或总体入选率的置信区间。50%边界附近的 A 档应降格措辞。','',
'## 逐维清单','', '| 目标 | 档 | Q1 skill | 平均入选率 | 最低入选率 | 非单点模块最高命中率 |','|---|---|---:|---:
|---:|---:|']
for a in
df.sort_values(['tier','mean_final_top3_selection_frequency'],ascending=[True,False]).itertuples():
    lines.append(f'| {a.target} | {a.tier} | {a.Q1_MSE_skill_vs_style:.3f} |
{a.mean_final_top3_selection_frequency:.1%} | {a.min_final_top3_selection_frequency:.1%} |
{a.max_non_singleton_module_frequency:.1%} |')
lines += ['', '## 尚未解决的混杂','',
'尚未进行风格内置换、风格残差化或模块整体置换。酸度的乳酸候选与酚类候选须分开解释；酚类可能是发酵/风格共现标志物。A、B
档也不能因此写成已经排除风格代理效应。酒精感虽 R² 高，但相对风格均值 skill 为负，按本规则不列入可强调的增量解释结论。']
(OUT/'Q2 可靠性补充.md').write_text('\n'.join(lines)+'\n',encoding='utf-8')
print(json.dumps(summary,ensure_ascii=False,indent=2))
```

analysis/q3/build_threshold_spec.py

```
"""Prepare auditable defect units, references and real-data compatibility checks."""
from pathlib import Path
import json
import numpy as np
import pandas as pd

OUT=Path(__file__).resolve().parent
ROOT=OUT.parents[1]
DATA=next(ROOT.glob('2026 年第七届*/2026*赛道 A*'))/'附件'
x=pd.read_csv(DATA/'1.csv').set_index('beer')
c=pd.read_excel(DATA/'8.xlsx',sheet_name='Calibrated chemical
dataset').set_index('beer').loc[x.index]
t=pd.read_excel(DATA/'8.xlsx',sheet_name='Sensory thresholds').set_index('field_name_R')
fields={'DMS':'DMS','Diacetyl':'X2.3.butanedione','4VG':'X4VG','t2n':'X.E..2.nonenal'}
```

```
spec={}
for name,field in fields.items():
    row=t.loc[field]
    if name=='t2n':
        threshold=.0001;alternatives=[.00003,.0001];source='Santos et al. 2003 reference 0.1 ug/L;
Saison et al. 2009 Table 2 sensitivity 0.03 ppb'
    else:
        factor=.001 if row.unit=='ug/L' else 1.
        threshold=float(row.threshold_median)*factor
        alternatives=sorted(set([threshold,float(row.threshold_mean)*factor]));source='8.xlsx
Sensory thresholds; median main, mean alternative'
    spec[name]={'field':field,'declared_raw_unit':row.unit,'threshold_mg_L':threshold,
                'sensitivity_thresholds_mg_L':alternatives,'threshold_source':source,
                'table1_mapping':'log10(raw value in declared raw unit)' if name in ['DMS','4VG']
else 'mostly identity, exceptional rows require review',
                'qualification':'t2n absolute scale flagged by near-universal exceedance' if
name=='t2n' else 'conditional on calibration and detection processing'}
config={'analysis_date':'2026-10-25','canonical_unit':'mg/L','measured_defects':spec,
        'recommended_primary_transfer_rule':'concentration_nonincrease_on_same_measurement_scale',
        'also_report':['excess_nonworsening','below_threshold'],
        'Stale_hops':{'measured_chemical':'absent','candidate_external_marker':'isovaleric acid / 3-
methylbutanoic acid',
                      'not_equivalent_to':'ethyl_isovalerate / ethyl 3-methylbutyrate',
                      'status':'not_chemically_verifiable; weak sensory proxy only'},
        'sources':[
          {'id':'Santos2003','doi':'10.1016/S0021-9673(02)01396-1','url':'https://
pubmed.ncbi.nlm.nih.gov/12580508/'},
          {'id':'Saison2009','doi':'10.1016/j.foodchem.2008.10.078','url':'https://
www.agraria.com.br/extranet_2016/uploads/AgromalteArquivo/
contribution_of_staling_compounds_to_the_aged_flavour_1601494914447.pdf','locator':'Table 2, journal
p.1209; thresholds are indicative'},
          {'id':'BrewersAssociation2025','url':'https://cdn.brewersassociation.org/wp-content/
uploads/2025/07/14095250/BAtech25-Off-Flavor-Isovaleric-201.pdf','locator':'p.1; isovaleric acid,
hop degradation and matrix-dependent thresholds'}],
        'important':'No automatic 1000x rescaling; no missing-as-zero; no all-five sensory
guarantee.'}
(OUT/'defect_thresholds.json').write_text(json.dumps(config,ensure_ascii=False,indent=2),encoding='u
tf-8')
rows=[];summ={}
for name,field in fields.items():
    raw=c[field];obs=x[field]
    threshold=spec[name]['threshold_mg_L'];factor=.001 if t.loc[field,'unit']=='ug/L' else 1.
    for beer in x.index:
        r=raw.loc[beer];o=obs.loc[beer]
        flags=[]
        if pd.isna(r):flags.append('calibrated_value_missing')
        if pd.isna(o):flags.append('table1_value_missing_do_not_treat_as_zero')
        if pd.notna(r) and r==0:flags.append('calibrated_zero_is_not_proof_of_true_zero')
        consistent=None
        if pd.notna(r) and pd.notna(o) and r>0:
            expected=np.log10(r) if name in ['DMS','4VG'] else r
            consistent=bool(abs(o-expected)<1e-7)
            if not consistent:flags.append('table1_calibrated_discrepancy')

rows.append({'beer':int(beer),'style':x.loc[beer,'tasting_category_fine'],'defect':name,'field':fiel
d,
                     'table1_value':None if pd.isna(o) else float(o),'calibrated_raw_value':None if
pd.isna(r) else float(r),
                     'calibrated_mg_L':None if pd.isna(r) else
float(r*factor),'threshold_mg_L':threshold,
                     'mapping_consistent_positive_pair':consistent,
                     'flags':' | '.join(flags)})
    mg=raw*factor
    summ[name]={'valid_calibrated':int(raw.notna().sum()),'table1_missing':int(obs.isna().sum()),
```

```
'calibrated_zero':int(raw.eq(0).sum()),'above_main_threshold_under_declared_units':int((mg>threshold
).sum()),
                'median_threshold_ratio':float(mg.median()/threshold)}
audit=pd.DataFrame(rows);audit.to_csv(OUT/'defect_concentration_audit.csv',index=False,encoding='utf
-8-sig')
extra={'summary':summ,
       'specific_discrepancies':audit[audit['flags'].str.contains('discrepancy|
missing')].replace({np.nan:None}).to_dict('records'),
       'isovaleric_acid_exact_measured_field_found':False,
       't2n_low_threshold_exceedance':int((c['X.E..2.nonenal']>.00003).sum())}
(OUT/'threshold_audit_summary.json').write_text(json.dumps(extra,ensure_ascii=False,indent=2),encodi
ng='utf-8')
print(json.dumps(summ,ensure_ascii=False,indent=2))
```

analysis/q3/defect_constraints.py

```
"""Numerical checks on explicitly supplied mg/L coordinates, not sensory certification.
AI assistance: OpenAI Codex, 2026-10-25.
"""
from pathlib import Path
import math,json

CONFIG_PATH=Path(__file__).with_name('defect_thresholds.json')

def check_transfer(source_mg_L,candidate_mg_L,config=None):
    """Never turns missing chemical/sensory information into a passed guarantee."""
    config=config or json.loads(CONFIG_PATH.read_text(encoding='utf-8'))
    items={}
    for name,spec in config['measured_defects'].items():
        a=source_mg_L.get(name);b=candidate_mg_L.get(name);threshold=spec['threshold_mg_L']
        valid=(isinstance(a,(int,float)) and isinstance(b,(int,float)) and
               math.isfinite(a) and math.isfinite(b) and a>=0 and b>=0)
        if not valid:

items[name]={'status':'not_evaluable','reason':'missing_nonfinite_or_negative_concentration'}
            continue
        es=max(a/threshold-1,0);ec=max(b/threshold-1,0)
        items[name]={'status':'evaluated_under_declared_units','source_mg_L':a,'candidate_mg_L':b,
                     'threshold_mg_L':threshold,'source_excess':es,'candidate_excess':ec,
                     'concentration_nonincrease':bool(b<=a),
                     'excess_nonworsening':bool(ec<=es),
                     'candidate_below_threshold':bool(b<=threshold)}
    complete=all(v['status']=='evaluated_under_declared_units' for v in items.values())
    return {'items':items,
            'all_four_evaluable':complete,
            'all_four_concentration_nonincrease':complete and all(v['concentration_nonincrease'] for
v in items.values()),
            'all_four_excess_nonworsening':complete and all(v['excess_nonworsening'] for v in
items.values()),
            'all_four_below_threshold':complete and all(v['candidate_below_threshold'] for v in
items.values()),
            'Stale_hops':
{'status':'not_chemically_verifiable','reason':'isovaleric_acid_not_measured;
ethyl_isovalerate_is_not_isovaleric_acid'},
            'all_five_chemically_certified':False,
            'qualification':'Numerical feasibility under supplied coordinates and thresholds; not a
physical or sensory guarantee.'}

if __name__=='__main__':
    base={'DMS':.01,'Diacetyl':.05,'4VG':.1,'t2n':.00005}
    assert check_transfer(base,base)['all_four_concentration_nonincrease']
    small={k:v/2 for k,v in base.items()}
    assert check_transfer(base,small)['all_four_concentration_nonincrease']
    under=base.copy();under['DMS']=.015
    got=check_transfer(base,under)
    assert not got['all_four_concentration_nonincrease'] and got['all_four_excess_nonworsening']
    high=base.copy();high['DMS']=.03
```

```
    lower=high.copy();lower['DMS']=.025
    got=check_transfer(high,lower)
    assert got['all_four_concentration_nonincrease'] and not got['all_four_below_threshold']
    for invalid in [None,float('nan'),-1]:
        changed=base.copy();changed['t2n']=invalid
        got=check_transfer(base,changed)
        assert not got['all_four_evaluable'] and not got['all_four_excess_nonworsening']
    assert not check_transfer(base,base)['all_five_chemically_certified']
    tiny=base.copy();tiny['DMS']=math.nextafter(base['DMS'],math.inf)
    assert not check_transfer(base,tiny)['all_four_concentration_nonincrease']
    print('Constraint checker passed: unchanged/decreased values, below-threshold increases,
existing exceedance, missing/nonfinite/negative values, and unmeasured Stale hops.')
```

analysis/q3/render_baseline_report.py

```
"""Scientific figures and report from saved first-version results."""
from pathlib import Path
import json
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from matplotlib.colors import TwoSlopeNorm
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler

OUT=Path(__file__).resolve().parent;ROOT=OUT.parents[1]
DATA=next(ROOT.glob('2026年第七届*/2026*赛道A*'))/'附件'
s=json.loads((OUT/'gaussian_baseline_summary.json').read_text(encoding='utf-8'))
m=pd.read_csv(OUT/'migration_metrics_all_weights.csv');d=pd.read_csv(OUT/'migration_defect_checks.cs
v')
g=pd.read_csv(OUT/'generator_final_descriptive_metrics.csv');cv=pd.read_csv(OUT/'generator_holdout_m
etrics.csv')
cvcor=pd.read_csv(OUT/'generator_holdout_correlation.csv')
quality=pd.read_csv(ROOT/'analysis/A_feasibility/q1_metrics/ExtraTrees_dimension_summary.csv',index_
col=0)
plt.rcParams.update({'font.family':'Microsoft YaHei','axes.unicode_minus':False,'font.size':10})
palette=['#247e8a','#c48b35','#8063a5']

# Source/target preservation and defect control must be visible together.
group=m.groupby('alpha').mean(numeric_only=True).reset_index()
fig,axes=plt.subplots(1,2,figsize=(12.6,5.2))
axes[0].scatter(100*m.nearest_real_Tripel_relative_reduction,m.mean_defect_change_standardized,
                c=m.alpha,cmap='viridis',alpha=.24,s=23)
axes[0].plot(100*group.nearest_real_Tripel_relative_reduction,group.mean_defect_change_standardized,
             '-o',color='#247e8a',lw=1.8,ms=6)
for a in group.itertuples():
    axes[0].annotate(f'α={a.alpha:g}',
(100*a.nearest_real_Tripel_relative_reduction,a.mean_defect_change_standardized),
                     xytext=(5,-13 if a.alpha in [1,10] else 6),textcoords='offset
points',fontsize=9)
axes[0].axhline(0,color='#9aa1a7',lw=.8)
axes[0].set_xlabel('到真实 Tripel 最近邻距离的平均下降（%）')
axes[0].set_ylabel('已观测缺陷坐标平均变化（各列训练标准差单位）')
axes[0].set_title('风格接近与缺陷变化同时报告')
axes[1].plot(group.source_change_RMS_standardized,100*group.nearest_real_Tripel_relative_reduction,'
-o',color='#8063a5')
for a in group.itertuples():
    axes[1].annotate(f'{a.alpha:g}',
(a.source_change_RMS_standardized,100*a.nearest_real_Tripel_relative_reduction),
                     xytext=(6,-12 if a.alpha==10 else 5),textcoords='offset points',fontsize=9)
axes[1].set_xlabel('相对源谱改变量（231维标准化RMS）')
axes[1].set_ylabel('到真实 Tripel 最近邻距离的平均下降（%）')
axes[1].set_title('向目标接近，需要改变多少源谱')
for ax in axes:ax.grid(alpha=.17)
fig.suptitle('Pils/Lager → Tripel：固定六档权重，主结果 α=1',fontsize=13)
```

```
fig.text(.5,.025,'实点为均值，淡点为单款。11 款源谱中 2 款缺双乙酰，缺陷均值只含已观测项；距离下降不等于独立品评验证。',ha='center',fontsize=9,color='#59636d')
fig.tight_layout(rect=(0,.06,1,.94));fig.savefig(OUT/'迁移风格接近与缺陷权衡.png',dpi=190);fig.savefig(OUT/'迁移风格接近与缺陷权衡.svg');plt.close(fig)

# Keep the failed floor-mass reproduction prominent.
fig,axes=plt.subplots(1,2,figsize=(11.8,4.8));pos=np.arange(len(g));w=.34
axes[0].bar(pos-w/2,100*g.real_4VG_floor_fraction,width=w,label='实测记录',color='#5a7185')
axes[0].bar(pos+w/2,100*g.synthetic_4VG_floor_fraction,width=w,label='生成候选',color='#cf994a')
axes[0].set_xticks(pos,g['style']);axes[0].set_ylim(0,112);axes[0].set_ylabel('4VG 下限状态占比（%）');
axes[0].legend(frameon=False)
for i in range(len(g)):
    axes[0].text(i-w/2,100*g.real_4VG_floor_fraction.iloc[i]
+2,f'{g.real_4VG_floor_fraction.iloc[i]:.1%}',ha='center',fontsize=9)
    axes[0].text(i+w/2,100*g.synthetic_4VG_floor_fraction.iloc[i]
+2,f'{g.synthetic_4VG_floor_fraction.iloc[i]:.1%}',ha='center',fontsize=9)
axes[0].set_title('首版明确短板：下限点质量未保真')
methods=['style_conditional','unconditional'];agg=cv.groupby('method')
for i,method in enumerate(methods):
    data=cv[cv.method==method].mean_coordinate_Wasserstein_standardized
    axes[1].scatter(np.full(len(data),i)
+np.linspace(-.08,.08,len(data)),data,s=28,alpha=.6,color=['#247e8a','#9a8aab'][i])
    axes[1].plot([i-.17,i+.17],[data.median()]*2,color='#263b49',lw=2)
axes[1].set_xticks([0,1],['风格条件高斯','无条件高斯']);axes[1].set_ylabel('留出平均单变量 Wasserstein 距离')
axes[1].set_title('三折 × 三风格：保留异常值影响');axes[1].grid(axis='y',alpha=.15)
fig.text(.5,.015,'左图为全数据设计拟合的描述比较；右图为训练折内预处理的留出结果。横线为中位数。',ha='center',fontsize=9,color='#59636d')
fig.tight_layout(rect=(0,.05,1,1));fig.savefig(OUT/'首版生成保真与下限偏差.png',dpi=190);plt.close(fig)

sens=pd.read_csv(OUT/'generated_sensory_summary_with_reliability.csv')
numeric=quality[quality.type=='numeric'].index.tolist()
matrix=sens[sens.type=='numeric'].pivot(index='style',columns='target',values='prediction_mean').loc[['Tripel','Kriek','Lambic'],numeric]
reliable=sens[sens.type=='numeric'].drop_duplicates('target').set_index('target').reliable_for_emphasis
ticklabels=[t+(' †' if not reliable[t] else '') for t in numeric]
bound=max(abs(matrix.to_numpy()).max(),.1)
fig,ax=plt.subplots(figsize=(15,4.8));im=ax.imshow(matrix.to_numpy(),aspect='auto',cmap='RdBu_r',norm=TwoSlopeNorm(0,-bound,bound))
ax.set_yticks(range(3),matrix.index);ax.set_xticks(range(len(numeric)),ticklabels,rotation=55,ha='right',fontsize=8)
for i in range(3):
    for j in range(len(numeric)):
        if not reliable[numeric[j]]:

ax.add_patch(plt.Rectangle((j-.5,i-.5),1,1,facecolor='#dddddd',alpha=.65,edgecolor='none'))
ax.set_title('生成候选的感官画像：Q1 前向模型预测，保留逐维可靠性标记',pad=12)
fig.colorbar(im,ax=ax,fraction=.025,pad=.02,label='标准化感官预测均值')
fig.text(.5,.02,'†及灰色覆盖：Q1 $R^2$<0.1 或相对风格均值 MSE skill≤0。图中数值是模型输出，不是原始 0–6 分或品评验证。',ha='center',fontsize=9)
fig.tight_layout(rect=(0,.08,1,1));fig.savefig(OUT/'生成感官画像与可靠性.png',dpi=190);plt.close(fig)

# Scaling diagnostics retained as evidence, never used to delete test observations.
x=pd.read_csv(DATA/'1.csv').set_index('beer');X=x.drop(columns=['beer_id','tasting_category_fine'])
logs=json.loads((OUT/'generator_holdout_splits.json').read_text(encoding='utf-8'));diag=[]
for log in logs:

ii=SimpleImputer(strategy='median');sc=StandardScaler().fit(ii.fit_transform(X.loc[log['train_beer']]))
    z=sc.transform(ii.transform(X.loc[log['test_beer']]));maxabs=abs(z).max(0)
    for j in np.argsort(-maxabs)[:5]:
        i=int(np.argmax(abs(z[:,j])));diag.append({'fold':log['fold'],'field':X.columns[j],
            'test_beer':log['test_beer'][i],'max_abs_train_scaled_holdout':float(maxabs[j]),
            'training_scale':float(sc.scale_[j]),'holdout_stored_value':float(X.loc[log['test_beer']
```

```
[i],X.columns[j]])})
pd.DataFrame(diag).to_csv(OUT/'holdout_scaling_diagnostics.csv',index=False,encoding='utf-8-sig')

pc=m[m.alpha==1];cvmean=cv.groupby('method').mean(numeric_only=True)
maegain=1-cvmean.loc['style_conditional','mean_coordinate_MAE_standardized']/
cvmean.loc['unconditional','mean_coordinate_MAE_standardized']
wgain=1-cvmean.loc['style_conditional','mean_coordinate_Wasserstein_standardized']/
cvmean.loc['unconditional','mean_coordinate_Wasserstein_standardized']
lines=['# Q3 收缩高斯与约束迁移：首版结果','',
'日期：2026-10-26。已跑通生成、留出保真评价、跨风格迁移、缺陷验收及感官画像。首版约束可运行，但生成分布仍有明显不足，不能宣称所有保真检验通过。','',
'## 已实现方法与输出','',
'231 维储存坐标训练集内中位数插补、标准化，按风格拟合 Ledoit–Wolf 收缩高斯；样本投影到训练观测范围。
Tripel、Kriek、Lambic 各 300 个，共 900 个候选。另输出 217 个单体化学候选字段，去除 Q2 定义的 14 个理化/派生/混合指标。',
'', '它是固定参数的最小基线，没有潜变量维数选择、GAN 或 CVAE。完整 231 维包含派生指标，但未施加派生指标恒等式、化学计量或酿造工艺约束，因此交付是候选特征谱，不能称为生产配方。','',
'## 阈值与下限补充','',
'| 缺陷 | 按附件 8 主阈值超出/有效数 | 比例 |', '|---|---:|---:|',
'| DMS | 156/250 | 62.4% |','| Diacetyl | 172/250 | 68.8% |','| 4VG | 18/250 | 7.2% |','| t2n |
248/249 | 99.6% |','',
'这些是附件记录和标注单位下的条件性计数。双乙酰校准零值不等于真实零；t2n 尺度疑点未解决，不擅自作 1000 倍修正。附件阈值只提供一行均值/中位数汇总，底层记录数量和介质不详，不编造为“n 次啤酒试验的中位数”。',
'', '4VG 下限状态 168/250=67.2%，CS2 最小值 209/250=83.6%。迁移中 9/11 款源谱 4VG 处于下限，候选强制不高于相同替代下限，不能解释为精确浓度不增加。没有测量误差依据，不额外添加测量容差。','',
'## 风格条件生成的留出保真','',
'三折风格分层；所有预处理、观测范围和高斯参数只来自训练折。对三个目标风格分别评价，以 9 个“折×风格”单元等权汇总。无条件高斯作为比较，共用标准正态随机数。','',
'| 指标（越小越好） | 风格条件高斯 | 无条件高斯 |', '|---|---:|---:|',
f'| 标准化逐坐标均值误差 | {cvmean.loc["style_conditional","mean_coordinate_MAE_standardized"]:.4f} |
{cvmean.loc["unconditional","mean_coordinate_MAE_standardized"]:.4f} |',
f'| 平均单变量 Wasserstein |
{cvmean.loc["style_conditional","mean_coordinate_Wasserstein_standardized"]:.4f} |
{cvmean.loc["unconditional","mean_coordinate_Wasserstein_standardized"]:.4f} |',
f'| 去风格中心相关矩阵 RMSE | {s["holdout_mean_correlation"]["style_conditional"]:.4f} |
{s["holdout_mean_correlation"]["unconditional"]:.4f} |','',
f'风格条件化使前两项分别下降{maegain:.1%}和{wgain:.1%}，但相关结构没有优于无条件基线。因此只能说风格中心与边际分布有所改善，不能说联合结构已得到充分保真。小风格留出只有 3–4 条，相关指标本身也很不稳定。',
'', '第四项为到同风格训练样本的最近邻距离。保存了生成/真实留出各自距离及比值，不设置事后通过线。生成训练样本精确复制数为 0，仅能排除精确复制，不能证明没有近邻依赖或模式收缩。',
'', '训练折尺度诊断发现个别留出值可达训练标准差的约 475 倍或 3330 倍，宏观均值会受其影响。没有删掉这些样本或按测试表现改缩放；同时保留均值、中位数、逐折表及诊断字段。',
'', '![生成保真与下限偏差](首版生成保真与下限偏差.png)','',
'| 风格 | 实测 4VG 下限占比 | 生成 4VG 下限占比 | 范围投影的坐标占比 |', '|---|---:|---:|---:|']
for a in g.itertuples():
    lines.append(f'| {a.style} | {a.real_4VG_floor_fraction:.1%} |
{a.synthetic_4VG_floor_fraction:.1%} | {a.range_projection_fraction:.1%} |')
lines += ['', '高斯模型不能重现大量重复下限形成的点质量。这里的实测与生成占比是全数据拟合的描述比较，仍足以暴露问题。范围投影较多也说明矩形范围约束不足以恢复真实联合分布。','',
'## Pils/Lager → Tripel 迁移','',
'在标准化坐标 z 中，最小化 0.5||z−z₀||²+0.5α(z−μ_T)ᵀΣ_T⁻¹(z−μ_T)，同时限制训练观测范围及四项缺陷上界。采用 L-BFGS-B 求解此有界凸二次问题，再在原储存坐标最终投影。α 固定为 0、0.1、0.3、1、3、10；主结果预先指定 α=1，不按效果挑权重。',
'', f'共 11 款源谱×6 档权重=66 个结果，优化均成功。主权重下，到目标高斯的 Mahalanobis 距离平均下降
{s["primary_mean_target_Mahalanobis_reduction"]:.1%}；到真实 Tripel 的最近邻标准化距离平均下降
{s["primary_mean_real_Tripel_NN_reduction"]:.1%}。前一距离直接参与目标，后一距离虽不是优化项，仍只是在同一数据几何空间内的接近度，不能替代独立风格品评。','',
f'- 9/11 款源谱四项字段齐全，9/9 在主权重下通过四项同尺度非增；涉及下限的项目另标记为下限状态保持。',
'- beer 119、28 缺少源双乙酰，2 款只验证其余已观测项，四项全部通过状态为不可完整验收。',
f'- 主权重下四项同时低于主工作阈值：{s["primary_four_below_threshold"]}/9。浓度非增不等于达标；t2n 阈值情景单独保存。',
'- 所有 66 个候选已观测缺陷都不增加；9 个源 4VG 下限状态在全部权重下均保持。',
'- 没有任何候选被认证为五项缺陷感官不恶化；Stale hops 没有异戊酸测量，现有乳酸/乙酸不新增为特异代理。','',
'![风格与缺陷权衡](迁移风格接近与缺陷权衡.png)','',
'图同时展示风格接近、缺陷变化和源谱改变量，因此不能通过把缺陷压低而忽略目标风格。它是设计权重扫描，不称为已求得全局 Pareto 前沿。','',
```

```
'| α | 到真实 Tripel 最近邻距离平均下降 | 源谱改变量 RMS | 已观测缺陷坐标平均变化 |', '|---|---:|---:|---:|---:|']
for a in group.itertuples():
    lines.append(f'| {a.alpha:g} | {a.nearest_real_Tripel_relative_reduction:.1%} |
{a.source_change_RMS_standardized:.3f} | {a.mean_defect_change_standardized:.4f} |')
lines += ['', '缺陷平均变化以各列训练标准差归一化，缺失项不纳入；不是把不同 mg/L 量纲直接相加，也不是新的缺陷感官总分。逐项原值/替代浓度与质量标志见验收表。','',
'## 感官画像与可靠性','',
'固定 Q1 Extra Trees 参数在全体实测样本上拟合最终前向模型，给生成与迁移谱输出 50 维标准化感官预测。逐项附原 Q1 折外 R² 和相对风格均值 MSE skill；候选间分位数不称为预测置信区间。','',
'![感官画像可靠性](生成感官画像与可靠性.png)','',
'酒精感虽 R² 高但相对风格均值 skill<0，灰色标记；Stale hops 也保留低可靠性标记，不能参与严格缺陷保证。这次不将新候选的标准化模型输出伪称为真实 0–6 评分。','',
'## 首版判断与下一步取舍','',
'首版完整跑通，可供论文报告方法、约束与权衡。保真仍存在明确问题：重复下限的点质量、训练尺度对稀有极端值敏感、相关结构未胜无条件基线、派生量恒等式未施加。不能把 900 条输出直接称为 900 条验证通过的合理化学谱。',
'', '暂不升级到 CVAE、GAN 或更复杂潜变量模型。若继续改进，优先考虑保留重复下限质量的边际映射及已知派生关系，属于针对数据结构的修正；需要另记协议并与首版同折比较，不回写本次预设协议。',
'', 'Q2 可靠性分档、可预测性关联图和宽松 Jaccard 已用保存结果完成，没有重训主模型。风格混杂仍未排除，不把相关性写成因果。','',
'## 复现与文件','',
'依次运行 supplement_threshold_audit.py、run_gaussian_baseline.py、render_baseline_report.py；Q2 补充运行 supplement_reliability.py。建模运行环境 Python3.9、NumPy、Pandas、SciPy、scikit-
learn、Matplotlib、joblib，具体版本另存 environment_versions.json。CSV 为数据与统计中间输出，原始附件未改动。']
(OUT/'Q3 首版生成与迁移结果.md').write_text('\n'.join(lines)+'\n',encoding='utf-8')
import sys,scipy,sklearn,joblib
versions={'python':sys.version,'numpy':np.__version__,'pandas':pd.__version__,'scipy':scipy.__versio
n__,
          'scikit-
learn':sklearn.__version__,'matplotlib':matplotlib.__version__,'joblib':joblib.__version__}
(OUT/'environment_versions.json').write_text(json.dumps(versions,ensure_ascii=False,indent=2),encodi
ng='utf-8')
print('Report and three scientific figures completed.')
```

analysis/q3/run_gaussian_baseline.py

```
"""Style shrinkage Gaussian baseline and conservative bounded migration.
AI assistance: OpenAI Codex, 2026-10-26. See 首版运行协议.md.
"""
from pathlib import Path
import json
import warnings
import numpy as np
import pandas as pd
from scipy.optimize import minimize
from scipy.spatial.distance import cdist
from scipy.stats import wasserstein_distance
from sklearn.covariance import LedoitWolf
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import StratifiedKFold
from sklearn.ensemble import ExtraTreesRegressor
from threadpoolctl import threadpool_limits
import joblib

OUT=Path(__file__).resolve().parent
ROOT=OUT.parents[1]
DATA=next(ROOT.glob('2026年第七届*/2026*赛道A*'))/'附件'
SEED=20261026;NGEN=300;STYLES=['Tripel','Kriek','Lambic'];ALPHAS=[0,.1,.3,1,3,10]
threadpool_limits(2)
x=pd.read_csv(DATA/'1.csv').set_index('beer')
y=pd.read_csv(DATA/'4.csv').set_index('beer').loc[x.index]
X=x.drop(columns=['beer_id','tasting_category_fine']);Y=y.drop(columns=['beer_id','tasting_category_
fine'])
fields=list(X);targets=list(Y);labels=x.tasting_category_fine.to_numpy()
config=json.loads((OUT/'defect_thresholds.json').read_text(encoding='utf-8'))
floorinfo=pd.read_csv(OUT/'threshold_and_floor_prevalence.csv').set_index('defect')
quality=pd.read_csv(ROOT/'analysis/A_feasibility/q1_metrics/ExtraTrees_dimension_summary.csv',index_
```

```
col=0).loc[targets]
meta=pd.read_csv(ROOT/'analysis/A_feasibility/day1/sensory_metadata.csv',index_col=0).loc[targets]
eligible=pd.read_csv(ROOT/'analysis/q2/chemical_mapping_and_modules.csv')
chemical_fields=eligible.loc[eligible.eligible_primary_top3,'field'].tolist()

class GaussianStyles:
    def fit(self,A,styles):

self.imp=SimpleImputer(strategy='median');B=self.imp.fit_transform(np.asarray(A,dtype=float))
        self.scaler=StandardScaler().fit(B);self.Z=self.scaler.transform(B)
        self.low=B.min(axis=0);self.high=B.max(axis=0)
        self.models={'__all__':LedoitWolf().fit(self.Z)}
        for style in np.unique(styles):
            self.models[style]=LedoitWolf().fit(self.Z[styles==style])
        self.roots={}
        for key,m in self.models.items():
            v,Q=np.linalg.eigh(m.covariance_)
            self.roots[key]=Q*np.sqrt(np.maximum(v,0))[None,:]
        return self
    def sample(self,key,normals):
        m=self.models[key]
        B=self.scaler.inverse_transform(normals@self.roots[key].T+m.location_)
        flag=(B<self.low)|(B>self.high)
        return np.clip(B,self.low,self.high),float(flag.mean())
    def transform(self,A):return
self.scaler.transform(self.imp.transform(np.asarray(A,dtype=float)))

def corr(A):
    B=A-A.mean(0);den=np.sqrt((B*B).sum(0))
    C=(B.T@B)/np.maximum(np.outer(den,den),1e-15)
    return C,den>1e-12

# The split definition is fixed and independent of output quality.
foldrows=[];foldlogs=[];centered={};cvgen=[]
for fold,(train,test) in
enumerate(StratifiedKFold(3,shuffle=True,random_state=SEED).split(X,labels)):
    fit=GaussianStyles().fit(X.iloc[train],labels[train]);rng=np.random.default_rng(SEED+fold)

foldlogs.append({'fold':fold,'train_beer':X.index[train].tolist(),'test_beer':X.index[test].tolist()
})
    for method in ['style_conditional','unconditional']:centered[method]=[[],[]]
    for style in STYLES:
        at=test[labels[test]==style];testz=fit.transform(X.iloc[at])
        trainz=fit.Z[labels[train]==style];noise=rng.normal(size=(NGEN,len(fields)))
        realnn=cdist(testz,trainz).min(axis=1).mean()/np.sqrt(len(fields))
        for method,key in [('style_conditional',style),('unconditional','__all__')]:
            B,cliprate=fit.sample(key,noise);Z=fit.transform(B)
            dnn=cdist(Z,trainz).min(axis=1).mean()/np.sqrt(len(fields))
            floor=float(X.iloc[train]['X4VG'].min())

foldrows.append({'fold':fold,'style':style,'method':method,'n_train_style':len(trainz),'n_test_style
':len(at),
                'mean_coordinate_MAE_standardized':float(abs(Z.mean(0)-testz.mean(0)).mean()),

'mean_coordinate_Wasserstein_standardized':float(np.mean([wasserstein_distance(Z[:,j],testz[:,j])
for j in range(len(fields))])),

'synthetic_nearest_train_distance':float(dnn),'real_holdout_nearest_train_distance':float(realnn),
                'synthetic_to_real_NN_ratio':float(dnn/realnn),'range_projection_fraction':cliprate,
                'synthetic_4VG_floor_fraction':float((B[:,fields.index('X4VG')]==floor).mean()),
                'holdout_4VG_floor_fraction':float(X.iloc[at]['X4VG'].eq(floor).mean())})
            centered[method][0].append(Z-Z.mean(0));centered[method][1].append(testz-testz.mean(0))
    for method,(aa,bb) in centered.items():
        Cg,vg=corr(np.vstack(aa));Cr,vr=corr(np.vstack(bb));valid=vg&vr
        ids=np.where(valid)[0];i,j=np.triu_indices(len(ids),1)
```

```
        rmse=float(np.sqrt(np.mean((Cg[np.ix_(ids,ids)][i,j]-Cr[np.ix_(ids,ids)][i,j])**2)))
        cvgen.append({'fold':fold,'method':method,'pooled_within_style_correlation_RMSE':rmse,
                      'valid_coordinates':int(valid.sum()),'n_holdout':sum(len(v) for v in bb)})
    print(f'Completed generator holdout fold {fold+1}/3',flush=True)
pd.DataFrame(foldrows).to_csv(OUT/'generator_holdout_metrics.csv',index=False,encoding='utf-8-sig')
pd.DataFrame(cvgen).to_csv(OUT/'generator_holdout_correlation.csv',index=False,encoding='utf-8-sig')
(OUT/'generator_holdout_splits.json').write_text(json.dumps(foldlogs,ensure_ascii=False,indent=2),en
coding='utf-8')

# Final design fit, not an out-of-fold predictor evaluation.
fit=GaussianStyles().fit(X,labels);rng=np.random.default_rng(SEED+100)
generated=[];descriptive=[]
for style in STYLES:
    B,cliprate=fit.sample(style,rng.normal(size=(NGEN,len(fields))))
    df=pd.DataFrame(B,columns=fields);df.insert(0,'style',style)
    df.insert(0,'candidate_id',[f'{style}_{i+1:03}' for i in range(NGEN)])
    generated.append(df)
    real=fit.Z[labels==style];syn=fit.transform(B)
    c1,v1=corr(real);c2,v2=corr(syn);idx=np.where(v1&v2)[0];i,j=np.triu_indices(len(idx),1)
    nn=cdist(syn,real).min(1)/np.sqrt(len(fields))
    descriptive.append({'style':style,'n_real':len(real),'n_generated':NGEN,
        'LedoitWolf_shrinkage':float(fit.models[style].shrinkage_),

'mean_coordinate_Wasserstein_standardized':float(np.mean([wasserstein_distance(syn[:,j],real[:,j])
for j in range(len(fields))])),
        'correlation_RMSE_in_sample':float(np.sqrt(np.mean((c1[np.ix_(idx,idx)][i,j]-
c2[np.ix_(idx,idx)][i,j])**2))),

'mean_nearest_real_distance_standardized':float(nn.mean()),'exact_training_copy_count':int((nn<1e-
12).sum()),
        'range_projection_fraction':cliprate,
        'real_4VG_floor_fraction':float(X.loc[labels==style,'X4VG'].eq(X.X4VG.min()).mean()),
        'synthetic_4VG_floor_fraction':float((B[:,fields.index('X4VG')]==X.X4VG.min()).mean())})
gen=pd.concat(generated,ignore_index=True)
gen.to_csv(OUT/'generated_231_coordinates.csv',index=False,encoding='utf-8-sig')
gen[['candidate_id','style']
+chemical_fields].to_csv(OUT/'generated_217_chemical_coordinates.csv',index=False,encoding='utf-8-
sig')
pd.DataFrame(descriptive).to_csv(OUT/'generator_final_descriptive_metrics.csv',index=False,encoding=
'utf-8-sig')
joblib.dump({'imputer':fit.imp,'scaler':fit.scaler,'low':fit.low,'high':fit.high,
             'style_models':fit.models,'fields':fields},OUT/'style_gaussians.joblib')

def mg(name,value):
    if not np.isfinite(value):return np.nan
    return float(10.**value*(.001 if name=='DMS' else 1)) if name in ['DMS','4VG'] else float(value)

audit=pd.read_csv(OUT/'defect_concentration_audit.csv').set_index(['beer','defect'])
targetmodel=fit.models['Tripel'];P=targetmodel.precision_;mu=targetmodel.location_;p=len(fields)
low=(fit.low-fit.scaler.mean_)/fit.scaler.scale_
high=(fit.high-fit.scaler.mean_)/fit.scaler.scale_
sources=X.loc[labels=='Pils/Lager'];srcz=fit.transform(sources)
realtripel=fit.Z[labels=='Tripel']
rows=[];defrows=[];migrations=[]
for bi,(beer,source) in enumerate(sources.iterrows()):
    z0=srcz[bi];hi=high.copy();active=[];known=[]
    for name,spec in config['measured_defects'].items():
        field=spec['field'];j=fields.index(field);original=source[field]
        if pd.notna(original):
            hi[j]=min(hi[j],(original-fit.scaler.mean_[j])/fit.scaler.scale_[j]);known.append(name)
        if bool(floorinfo.loc[name,'floor_is_supported_by_zero_pairing']) and
original==floorinfo.loc[name,'table1_floor']:
            active.append(name)
    source_md=float(np.sqrt(max((z0-mu)@P@(z0-mu),0)/p))
    source_nn=float(cdist(z0[None],realtripel).min()/np.sqrt(p))
```

```
    for alpha in ALPHAS:
        if alpha==0:
            z=z0.copy();success=True;nit=0
        else:
            def obj(z):
                diff=z-z0;delta=z-mu
                return .5*(diff@diff+alpha*(delta@P@delta)),diff+alpha*(P@delta)
            opt=minimize(obj,np.clip(z0,low,hi),method='L-BFGS-B',jac=True,
                         bounds=list(zip(low,hi)),options={'maxiter':1200,'ftol':1e-12,'gtol':1e-
8,'maxls':40})
            z=np.clip(opt.x,low,hi);success=bool(opt.success);nit=int(opt.nit)
        B=np.clip(fit.scaler.inverse_transform(z[None])[0],fit.low,fit.high)
        # Projection in the original stored coordinate eliminates conversion roundoff.
        for name in known:
            field=config['measured_defects'][name]['field'];j=fields.index(field)
            B[j]=min(B[j],source[field])
        z=fit.transform(B[None])[0]
        delta=z-mu;md=float(np.sqrt(max(delta@P@delta,0)/p))
        nn=float(cdist(z[None],realtripel).min()/np.sqrt(p))
        changes=[];passes=[];below=[];excess=[];flags=[]
        for name,spec in config['measured_defects'].items():
            f=spec['field'];j=fields.index(f);a=source[f];b=B[j]
            aa=mg(name,a);bb=mg(name,b);threshold=spec['threshold_mg_L']
            valid=bool(np.isfinite(aa) and aa>=0)
            isfloor=name in active
            passed=bool(b<=a) if valid else None
            if valid:
                passes.append(passed);below.append(bool(bb<=threshold))
                excess.append(bool(max(bb/threshold-1,0)<=max(aa/threshold-1,0)))
                changes.append(float((b-a)/fit.scaler.scale_[j]))
            flag=str(audit.loc[(beer,name),'flags'])
            if flag!='nan':flags.append(name+':'+flag)
            defrows.append({'beer':int(beer),'alpha':alpha,'defect':name,
                'source_table1':None if pd.isna(a) else float(a),'candidate_table1':float(b),
                'source_mg_L_or_floor_surrogate':aa,'candidate_mg_L_or_floor_surrogate':bb,
                'threshold_mg_L':threshold,'concentration_nonincrease':passed,
                'threshold_pass':bool(bb<=threshold) if valid else None,
                'source_at_supported_floor':isfloor,'floor_status_retained':bool(b<=a) if isfloor
else None,
                'interpretation':'floor_surrogate_cap_not_exact_measured_concentration' if isfloor
else ('same_table1_coordinate' if valid else 'missing_source_not_certifiable'),
                'source_quality_flags':flag if flag!='nan' else '',
                't2n_low_scenario_pass':bool(bb<=.00003) if name=='t2n' and valid else None})
        complete=len(known)==4

rows.append({'beer':int(beer),'alpha':alpha,'primary_alpha':alpha==1,'optimizer_success':success,'op
timizer_iterations':nit,
            'source_target_Mahalanobis':source_md,'target_Mahalanobis':md,
            'target_Mahalanobis_relative_reduction':1-md/source_md,
            'source_nearest_real_Tripel_distance':source_nn,'nearest_real_Tripel_distance':nn,
            'nearest_real_Tripel_relative_reduction':1-nn/source_nn,
            'source_change_RMS_standardized':float(np.sqrt(np.mean((z-z0)**2))),
            'mean_defect_change_standardized':float(np.mean(changes)),
            'maximum_defect_change_standardized':float(max(changes)),
            'four_defects_evaluable_on_table1':complete,
            'all_observed_defects_nonincrease':bool(all(passes)),
            'all_four_nonincrease':bool(complete and all(passes)),
            'all_four_excess_nonworsening':bool(complete and all(excess)),
            'all_four_below_threshold':bool(complete and all(below)),
            'n_floor_defects':len(active),'source_quality_flags':' | '.join(flags),
            'Stale_hops_chemically_verified':False})

record=dict(zip(fields,B));record.update({'beer':int(beer),'alpha':alpha,'source_style':'Pils/
Lager','target_style':'Tripel'})
        migrations.append(record)
```

```
    print(f'Migration source {bi+1}/{len(sources)} complete, beer {beer}',flush=True)
mr=pd.DataFrame(rows);dr=pd.DataFrame(defrows);mdf=pd.DataFrame(migrations)
mr.to_csv(OUT/'migration_metrics_all_weights.csv',index=False,encoding='utf-8-sig')
dr.to_csv(OUT/'migration_defect_checks.csv',index=False,encoding='utf-8-sig')
mdf.to_csv(OUT/'migration_231_coordinates.csv',index=False,encoding='utf-8-sig')
mdf[mdf.alpha==1].to_csv(OUT/'migration_primary_alpha1.csv',index=False,encoding='utf-8-sig')
assert mr.optimizer_success.all(),mr[~mr.optimizer_success]
assert mr.all_observed_defects_nonincrease.all()
assert dr[dr.source_at_supported_floor].floor_status_retained.all()
assert not mr.Stale_hops_chemically_verified.any()
assert gen[fields].notna().all().all()
assert (gen[fields].to_numpy()>=fit.low).all() and (gen[fields].to_numpy()<=fit.high).all()

imputer=SimpleImputer(strategy='median');XX=imputer.fit_transform(X)
front=ExtraTreesRegressor(n_estimators=160,min_samples_leaf=3,max_features=.7,random_state=20261024,
n_jobs=2).fit(XX,Y)
joblib.dump({'imputer':imputer,'model':front,'fields':fields,'targets':targets},OUT/'sensory_forward
_model.joblib')
out=gen[['candidate_id','style']].copy()
prediction=front.predict(imputer.transform(gen[fields]))
out=pd.concat([out,pd.DataFrame(prediction,columns=targets)],axis=1)
out.to_csv(OUT/'generated_sensory_standardized.csv',index=False,encoding='utf-8-sig')
mp=front.predict(imputer.transform(mdf[fields]));sens=mdf[['beer','alpha']].copy()
sens=pd.concat([sens,pd.DataFrame(mp,columns=targets)],axis=1)
sens.to_csv(OUT/'migration_sensory_standardized.csv',index=False,encoding='utf-8-sig')
sensrows=[]
for style in STYLES:
    a=prediction[gen['style'].eq(style)]
    for j,t in enumerate(targets):
        sensrows.append({'style':style,'target':t,'type':meta.loc[t,'DataType'],
            'prediction_mean':float(a[:,j].mean()),'candidate_q10':float(np.quantile(a[:,j],.1)),
            'candidate_q90':float(np.quantile(a[:,j],.9)),

'Q1_R2':float(quality.loc[t,'R2']),'Q1_MSE_skill_vs_style':float(quality.loc[t,'MSE_skill_vs_style']
),
            'reliable_for_emphasis':bool(quality.loc[t,'R2']>=.1 and
quality.loc[t,'MSE_skill_vs_style']>0 and t!='stale_hops')})
pd.DataFrame(sensrows).to_csv(OUT/'generated_sensory_summary_with_reliability.csv',index=False,encod
ing='utf-8-sig')
primary=mr[mr.alpha==1];fm=pd.DataFrame(foldrows);fc=pd.DataFrame(cvgen)
summ={'analysis_date':'2026-10-26','seed':SEED,'generated_each_style':NGEN,

'generated_styles':STYLES,'n_generated':len(gen),'model':'native_stored_scale_standardization_and_st
yle_LedoitWolf_Gaussian_with_range_projection',
      'holdout_mean_metrics':fm.groupby('method').mean(numeric_only=True).to_dict('index'),

'holdout_mean_correlation':fc.groupby('method').pooled_within_style_correlation_RMSE.mean().to_dict(
),
      'n_source_Pils_Lager':len(sources),'primary_alpha':1,'weights':ALPHAS,
      'primary_four_evaluable':int(primary.four_defects_evaluable_on_table1.sum()),
      'primary_four_nonincrease':int(primary.all_four_nonincrease.sum()),
      'primary_four_below_threshold':int(primary.all_four_below_threshold.sum()),

'primary_mean_target_Mahalanobis_reduction':float(primary.target_Mahalanobis_relative_reduction.mean
()),

'primary_mean_real_Tripel_NN_reduction':float(primary.nearest_real_Tripel_relative_reduction.mean())
,

'source_at_floor_counts':dr[dr.alpha==1].groupby('defect').source_at_supported_floor.sum().astype(in
t).to_dict(),
      'all_optimizer_runs_successful':bool(mr.optimizer_success.all()),

'all_observed_defects_nonincrease_all_weights':bool(mr.all_observed_defects_nonincrease.all()),
      'all_five_chemical_guarantee':False,
```

```
    'limitations':['missing_Diacetyl_sources_not_certified','floor_values_are_surrogates',

't2n_absolute_scale_unresolved','Stale_hops_not_chemically_observed','derived_coordinate_identities_
not_enforced',

'Gaussian_cannot_exactly_reproduce_point_masses','no_external_sensory_or_brewing_validation']}
(OUT/'gaussian_baseline_summary.json').write_text(json.dumps(summ,ensure_ascii=False,indent=2),encod
ing='utf-8')
print(json.dumps(summ,ensure_ascii=False,indent=2),flush=True)
```

analysis/q3/supplement_threshold_audit.py

```
"""Read-only source audit; writes analytical CSV/JSON/Markdown intermediates.
AI assistance: OpenAI Codex, 2026-10-26.
"""
from pathlib import Path
import json
import numpy as np
import pandas as pd

OUT=Path(__file__).resolve().parent
ROOT=OUT.parents[1]
DATA=next(ROOT.glob('2026年第七届*/2026*赛道A*'))/'附件'
x=pd.read_csv(DATA/'1.csv').set_index('beer')
c=pd.read_excel(DATA/'8.xlsx',sheet_name='Calibrated chemical
dataset').set_index('beer').loc[x.index]
t=pd.read_excel(DATA/'8.xlsx',sheet_name='Sensory thresholds')
cfg=json.loads((OUT/'defect_thresholds.json').read_text(encoding='utf-8'))
rows=[]
for name,s in cfg['measured_defects'].items():
    f=s['field'];a=x[f];r=c[f];fac=.001 if s['declared_raw_unit']=='ug/L' else 1
    floor=float(a.min());mask=a.eq(floor)
    count=int((r*fac>s['threshold_mg_L']).sum());valid=int(r.notna().sum())
    zero=r.eq(0)
    paired_zero=zero & a.notna()
    corroborated=bool(paired_zero.any() and a[paired_zero].eq(floor).all())
    rows.append({'defect':name,'field':f,'valid_calibrated':valid,'above_threshold':count,
                 'above_fraction':count/valid,'threshold_mg_L':s['threshold_mg_L'],

'table1_floor':floor,'floor_count':int(mask.sum()),'floor_fraction':float(mask.mean()),
                 'floor_is_supported_by_zero_pairing':corroborated,
                 'calibrated_zero_count':int(zero.sum()),'table1_missing':int(a.isna().sum()),
                 'source_threshold_aggregate_rows':int(t.field_name_R.eq(f).sum()),
                 'underlying_threshold_observation_count':'not_provided',
                 'threshold_medium':'not_provided_in_attachment'})
audit=pd.DataFrame(rows)
audit.to_csv(OUT/'threshold_and_floor_prevalence.csv',index=False,encoding='utf-8-sig')
hits=t[t.chemical_name.astype(str).str.contains('acid',case=False,regex=False) |
t.field_name_R.eq('acids_sum')]
hits[['field_name_R','chemical_name','unit']].to_csv(OUT/'measured_acid_fields.csv',index=False,enco
ding='utf-8-sig')
text=['# 阈值、下限状态与测量列补充核查','', '日期：2026-10-26。计数以附件8校准表为依据，按标注单位，不做1000倍修
正。零值只是记录状态，不能解释为精确测量零。','',
'| 缺陷 | 超主阈值/有效数 | 比例 | 附件1最小值占比 | 下限替代佐证 |',
'|---|---:|---:|---:|---|']
for a in audit.itertuples():
    text.append(f'| {a.defect} | {a.above_threshold}/{a.valid_calibrated} | {a.above_fraction:.2%} |
{a.floor_count}/250 ({a.floor_fraction:.1%}) | {"有校准零值配对" if
a.floor_is_supported_by_zero_pairing else "仅为最小观测值，不能认定检出下限"} |')
text += ['', 't2n较低阈值情景为249/249；主阈值几乎没有区分力。四项均以同尺度不增加为主，阈值判断为辅。',
'', '4VG有168/250个下限状态；Pils/Lager源谱的下限状态另在迁移表逐项标记。CS2虽非本次四项缺陷，最小值记录
209/250=83.6%，显示朴素高斯对点质量的保真能力有局限。',
'', '## 阈值的聚合来源', '',
'附件8每项只有一行汇总记录，提供threshold_mean与threshold_median，没有各次阈值试验、底层记录条数、记录权重或介质
（水/啤酒）的逐项信息。原研究补充工作簿同名表同样只有汇总字段。中位数直接取现成字段，不再对一行数据计算“中位数”；不能反推
出底层有几条记录，也不能声称全部来自啤酒基质。',
'', 't2n主情景引用Santos等2003；低情景引用Saison等2009的lager flavour阈值，区别于odor终点。二者不是统计区
```

```
间。详见原阈值报告与配置中的来源。',
'', '## 陈旧酒花相关测量', '',
'附件1及名称表不存在isovaleric acid/3-methylbutanoic acid字段。直接酸类只有乙酸、乳酸，另有总酸派生量。名称表中的Iso-alpha acids对应bitterness理化指标；名称含butyrate/isovalerate的多项是酯，不是相应游离酸。现有酸类没有足够特异性支持Stale hops代理约束，因此不新增代理化学保证。',
'', '## 下限和数值容差', '',
'对DMS、4VG经校准零值配对佐证的替代下限记录，要求候选不高于同一替代下限，单独报告“下限状态保持”；换算的替代浓度不等于真实浓度或实测检出限。对没有佐证的观测最小值不冒称检出限。',
'', '没有找到可用于本次四项验收的逐样本重复测量误差或校准不确定度，本版不设测量容差。计算最终采用边界投影与严格≤；这是一种保守设计口径，不是物理品评保证。']
(OUT/'阈值与下限补充核查.md').write_text('\n'.join(text)+'\n',encoding='utf-8')
print(audit.to_string(index=False))
```

analysis/q3/verify_saved_outputs.py

```
"""Independent saved-output integrity, missingness and constraint checks."""
from pathlib import Path
import json
import numpy as np
import pandas as pd
import joblib

OUT=Path(__file__).resolve().parent;ROOT=OUT.parents[1]
DATA=next(ROOT.glob('2026年第七届*/2026*赛道A*'))/'附件'
x=pd.read_csv(DATA/'1.csv').set_index('beer')
cfg=json.loads((OUT/'defect_thresholds.json').read_text(encoding='utf-8'))
g=pd.read_csv(OUT/'generated_231_coordinates.csv')
m=pd.read_csv(OUT/'migration_231_coordinates.csv')
checks=pd.read_csv(OUT/'migration_metrics_all_weights.csv')
defects=pd.read_csv(OUT/'migration_defect_checks.csv')
fit=joblib.load(OUT/'style_gaussians.joblib')
fields=fit['fields']
assert len(g)==900 and len(m)==66
assert g.groupby('style').size().eq(300).all()
assert g.candidate_id.is_unique
assert np.isfinite(g[fields]).all().all()
assert (g[fields].to_numpy()>=fit['low']-1e-12).all()
assert (g[fields].to_numpy()<=fit['high']+1e-12).all()
count=0;missing=0
for row in m.itertuples(index=False):
    values=dict(zip(m.columns,row));beer=int(values['beer'])
    for name,spec in cfg['measured_defects'].items():
        field=spec['field'];a=x.loc[beer,field];b=values[field]
        if pd.notna(a):
            assert b<=a,(beer,values['alpha'],field,a,b)
            count+=1
        else:missing+=1
assert count==252 and missing==12
assert not checks[checks.beer.isin([119,28])].four_defects_evaluable_on_table1.any()
assert checks[~checks.beer.isin([119,28])].all_four_nonincrease.all()
assert not checks.Stale_hops_chemically_verified.any()
assert defects[defects.source_at_supported_floor].floor_status_retained.all()
front=joblib.load(OUT/'sensory_forward_model.joblib')
pred=front['model'].predict(front['imputer'].transform(g.iloc[:5][fields]))
saved=pd.read_csv(OUT/'generated_sensory_standardized.csv')
assert np.allclose(pred,saved.iloc[:5][front['targets']].to_numpy(),rtol=1e-10,atol=1e-10)
splits=json.loads((OUT/'generator_holdout_splits.json').read_text(encoding='utf-8'))
assert all(not set(a['train_beer'])&set(a['test_beer']) for a in splits)
assert sorted(b for a in splits for b in a['test_beer'])==sorted(x.index.tolist())
result={'saved_candidates':len(g),'migration_candidates':len(m),'observed_defect_pairs_checked':coun
t,

'missing_source_defect_pairs_not_certified':missing,'floor_caps_checked':int(defects.source_at_suppo
rted_floor.sum()),

'portable_models_loaded':True,'forward_prediction_roundtrip_verified':True,'holdout_partition_verifi
ed':True,
```

```
        'interpretation':'File/algorithm integrity checks, not proof of chemical realism or sensory
safety.'}
(OUT/'saved_output_verification.json').write_text(json.dumps(result,ensure_ascii=False,indent=2),enc
oding='utf-8')
print(json.dumps(result,ensure_ascii=False,indent=2))
```

src/make_paper_figures.py

```
"""论文新增图表（v3.1 结果）。读取 results/ 下已保存结果，不重新计算模型。
AI assistance: Claude (Claude Opus 5.5), Anthropic, 2026-10-26. 用于代码实现，详见支撑材料《AI 工具使用详情
说明》。
"""
from pathlib import Path

import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from matplotlib.patches import FancyBboxPatch

ROOT = Path(__file__).resolve().parents[1]
RES, FIG = ROOT / 'results', ROOT / 'paper' / 'figures'
FIG.mkdir(parents=True, exist_ok=True)
plt.rcParams.update({'font.family': 'WenQuanYi Zen Hei', 'axes.unicode_minus': False, 'font.size':
10,
                     'axes.spines.top': False, 'axes.spines.right': False, 'axes.edgecolor':
'#52514e',
                     'axes.labelcolor': '#0b0b0b', 'xtick.color': '#52514e', 'ytick.color':
'#52514e',
                     'axes.grid': True, 'grid.color': '#e6e5e0', 'grid.linewidth': 0.6,
'axes.axisbelow': True,
                     'savefig.dpi': 220, 'savefig.bbox': 'tight', 'figure.facecolor': 'white'})
S1, S2, S3, MUTED, INK = '#2a78d6', '#eb6834', '#1baf7a', '#8a8984', '#0b0b0b'


# ---------- 图：技术路线 ----------
def roadmap():
    fig, ax = plt.subplots(figsize=(10, 6.2))
    ax.set_axis_off(); ax.set_xlim(0, 10); ax.set_ylim(0, 6.2)

    def box(x, y, w, h, text, fc='#eef4fc', ec=S1):
        ax.add_patch(FancyBboxPatch((x, y), w, h, boxstyle='round,pad=0.02,rounding_size=0.08',
fc=fc, ec=ec, lw=1.2))
        ax.text(x + w / 2, y + h / 2, text, ha='center', va='center', fontsize=9, color=INK,
linespacing=1.4)

    def arrow(x1, y1, x2, y2):
        ax.annotate('', xy=(x2, y2), xytext=(x1, y1), arrowprops=dict(arrowstyle='-|>', color=MUTED,
lw=1.1))

    box(0.2, 5.1, 9.6, 0.85, '数据核查与预处理：对照附件 8 核实 70 个共享指标尺度（37 列 log10、33 列未取对数）；逐
列变换字典\n'
        '训练折内插补/标准化；5 次重复×3 折风格分层；2.csv 经数值重算确认为 Spearman', fc='#f4f4f2', ec=MUTED)
    cols = [(0.2, '问题一 感官评分', S1, '#eef4fc',
             ['Extra Trees / PLS / 风格均值基线\n50 维连续感官响应（折外）',
              'numeric：OOF 锚点 P5/P95\n 相对评分 0–6',
              'categorical：父项门控\n 相对勾选倾向',
              '预测支持度 A/B/C + MSE skill\n12 张评分卡 + 250 款主表']),
            (3.45, '问题二 化合物–风味', S2, '#fdf0ea',
             ['2.csv 共表达网络\n|ρ|≥0.6，Louvain 模块',
              '100 次 bootstrap 袋外置换\nTop-3 入选率、模块命中率',
              '解释稳定性 + 高度相关备选\nET 与 PLS 的 Jaccard',
              '敏感性：全局置换 vs 风格内置换']),
            (6.7, '问题三 生成与迁移', S3, '#e9f7f1',
             ['共同经验 CDF 映射 G\n（保留下限点质量）',
              '风格条件收缩高斯生成\n 与首版基线同折配对评价',
```

```
                '共同空间高斯 OT + 缺陷投影\n 冻结四条验收标准',
                '陈化方向指数；离散度\n 容许域敏感性（负面结果）'])]
    for x, title, ec, fc, steps in cols:
        ax.text(x + 1.55, 4.75, title, ha='center', va='center', fontsize=11, color=INK,
fontweight='bold')
        arrow(x + 1.55, 5.1, x + 1.55, 4.95)
        for k, s in enumerate(steps):
            y = 3.75 - k * 1.0
            box(x, y, 3.1, 0.78, s, fc=fc, ec=ec)
            if k < len(steps) - 1:
                arrow(x + 1.55, y, x + 1.55, y - 0.2)
    arrow(1.75, 0.75, 1.75, 0.45); arrow(1.75, 0.45, 8.0, 0.45); arrow(8.0, 0.45, 8.0, 0.75)
    ax.text(4.9, 0.25, 'Q1 预测支持度用于 Q2 分档；最终前向模型 + 冻结锚点 → Q3 生成谱感官画像', ha='center',
fontsize=8, color=MUTED)
    fig.savefig(FIG / 'fig_roadmap.png'); plt.close(fig)


# ---------- 图：Q1 预测支持度与相对评分分布 ----------
def q1():
    s = pd.read_csv(RES / 'q1' / 'q1_prediction_support_tiers.csv', encoding='utf-8-sig',
index_col=0).sort_values('R2_mean_oof')
    fig, (a, b) = plt.subplots(1, 2, figsize=(11, 7.2), gridspec_kw={'width_ratios': [1.5, 1]})
    y = np.arange(len(s))
    for t, c, lab in [('numeric', S1, 'numeric（强度，26 项）'), ('categorical', S2, 'categorical（勾选，
24 项）')]:
        m = s.type == t
        a.scatter(s.R2_mean_oof[m], y[m], s=26, color=c, label=lab, zorder=3, edgecolor='white',
linewidth=0.6)
    for v, lab in [(0.1, 'B/C 分界 0.1'), (0.3, 'A/B 分界 0.3')]:
        a.axvline(v, color=MUTED, ls='--', lw=0.9); a.text(v + 0.005, len(s) - 0.3, lab, fontsize=8,
color=MUTED)
    neg = s.index[~s.skill_positive]
    for f in neg:
        a.annotate('未优于风格均值', (s.R2_mean_oof[f], y[list(s.index).index(f)]), xytext=(8, -3),
                   textcoords='offset points', fontsize=7.5, color='#52514e')
    a.set_yticks(y); a.set_yticklabels(s.index, fontsize=7); a.set_xlabel('5 次重复折外平均 $R^2$')
    a.set_title('预测支持度：A 15 / B 21 / C 14', fontsize=11); a.legend(loc='lower right',
fontsize=8, frameon=False)
    sc = pd.read_csv(RES / 'q1' / 'q1_numeric_relative_scores_250.csv', encoding='utf-8-sig',
index_col=0).drop(columns='tasting_category_fine')
    dist = sc.stack().value_counts(normalize=True).sort_index()
    b.bar(dist.index, dist.values * 100, color=S1, width=0.7)
    for k, v in dist.items():
        b.text(k, v * 100 + 0.4, f'{v * 100:.1f}%', ha='center', fontsize=8, color='#52514e')
    b.set_xticks(range(7)); b.set_xlabel('相对评分等级'); b.set_ylabel('占比（%）')
    b.set_title('26 项 numeric 相对评分分布（250 款）', fontsize=11); b.grid(axis='x', visible=False)
    fig.text(0.5, -0.01, '锚点为 250 款 5 次重复 OOF 均值的 P5/P95，冻结后统一使用；评分为基于折外预测的相对展示尺度，
非原始品评分。',
             ha='center', fontsize=8, color=MUTED)
    fig.tight_layout(); fig.savefig(FIG / 'fig_q1_support.png'); plt.close(fig)

    # 12 张评分卡热图
    v = pd.read_csv(RES / 'q1' / 'q1_scorecards_12_values.csv', encoding='utf-8-sig', index_col=0)
    cols = ['A_malt_all', 'A_hops_all', 'A_esters_all', 'F_malt_all', 'F_hops_all', 'F_esters_all',
'body', 'bitternes',
            'co2', 'aftertaste', 'alcohol', 'sweetness', 'acidity', 'dms', 'metallic', 'X4vg',
't2n', 'diacetyl',
            'stale_hops', 'orange', 'coriander', 'clove', 'lactic', 'acetic', 'barnyard', 'overall']
    M = v[cols].to_numpy(float)
    fig, ax = plt.subplots(figsize=(11, 4.6))
    im = ax.imshow(M, cmap='Blues', vmin=0, vmax=6, aspect='auto')
    for i in range(M.shape[0]):
        for j in range(M.shape[1]):
            ax.text(j, i, int(M[i, j]), ha='center', va='center', fontsize=7.5, color='white' if
M[i, j] >= 4 else INK)
```

```
    tiers = s.support_tier
    ax.set_xticks(range(len(cols))); ax.set_xticklabels([f'{c}{"*" if tiers[c] == "C" else ""}' for
c in cols], rotation=60, ha='right', fontsize=8)
    ax.set_yticks(range(len(v))); ax.set_yticklabels([f'beer {b} ({st}) ' for b, st in zip(v.index,
v.tasting_category_fine)], fontsize=8)
    ax.grid(False); cb = fig.colorbar(im, ax=ax, fraction=0.025); cb.set_label('相对评分')
    ax.set_title('12 张评分卡的 numeric 相对评分（* 为预测支持度 C 档，仅作参考分）', fontsize=11)
    fig.tight_layout(); fig.savefig(FIG / 'fig_q1_scorecards.png'); plt.close(fig)


# ---------- 图：Q2 全局置换 vs 风格内置换（排名） ----------
def q2():
    r = pd.read_csv(RES / 'q2' / 'q2_within_style_permutation_top3.csv', encoding='utf-8-sig')
    pairs = [('acidity', 'lactic_acid.mg.L.'), ('lactic', 'lactic_acid.mg.L.'), ('acetic',
'lactic_acid.mg.L.'),
             ('X4vg', 'X4VG'), ('F_hops_all', 'ethyl_isovalerate'), ('bitternes',
'ethyl_octanoate'),
             ('alcohol', 'ethanol..v.v.'), ('X4vg', 'lactic_acid.mg.L.'), ('alcohol',
'lactic_acid.mg.L.'),
             ('bitternes', 'lactic_acid.mg.L.')]
    d = pd.DataFrame([r[(r.target == t) & (r.field == f)].iloc[0] for t, f in pairs])
    fig, ax = plt.subplots(figsize=(9, 4.6))
    y = np.arange(len(d))[::-1]
    for yi, (_, row) in zip(y, d.iterrows()):
        ax.plot([row.global_rank_217, row.within_rank_217], [yi, yi], color='#c9c8c2', lw=2,
zorder=1)
    ax.scatter(d.global_rank_217, y, s=48, color=S1, label='全局置换排名', zorder=3,
edgecolor='white')
    ax.scatter(d.within_rank_217, y, s=48, color=S2, label='风格内置换排名', zorder=3,
edgecolor='white', marker='D')
    ax.set_xscale('log'); ax.set_xticks([1, 3, 10, 30, 100, 217]); ax.set_xticklabels(['1', '3',
'10', '30', '100', '217'])
    ax.set_yticks(y); ax.set_yticklabels([f'{t} ← {f}' for t, f in pairs], fontsize=8.5)
    ax.set_xlabel('在 217 个单体化学候选中的排名（对数刻度，越靠左越重要）')
    ax.axvline(3, color=MUTED, ls='--', lw=0.8); ax.text(3.1, y.max() + 0.5, 'Top-3', fontsize=8,
color=MUTED)
    ax.legend(loc='center right', frameon=False, fontsize=8.5); ax.set_ylim(-0.7, y.max() + 0.9)
    ax.set_title('原 Top-3 候选在风格内置换下的排名变化（15 个划分、同一模型与评价集）', fontsize=11)
    fig.tight_layout(); fig.savefig(FIG / 'fig_q2_within_style.png'); plt.close(fig)


# ---------- 图：Q3 生成保真 ----------
def q3_gen():
    gm = pd.read_csv(RES / 'q3' / 'generator_holdout_metrics_paired.csv', encoding='utf-8-sig')
    gc = pd.read_csv(RES / 'q3' / 'generator_holdout_correlation_paired.csv', encoding='utf-8-sig')
    agg = gm.groupby('method')[['mean_coordinate_Wasserstein_standardized',
'floor_fraction_MAE_floor_columns',
                                'mean_coordinate_MAE_standardized']].mean()
    agg['corr'] = gc.groupby('method').pooled_within_style_correlation_RMSE.mean()
    methods = [('latent_style', '潜在高斯（本文）', S1), ('baseline_style', '首版风格高斯', S2),
('baseline_unconditional', '无条件高斯', S3)]
    metrics = [('mean_coordinate_MAE_standardized', '逐坐标均值误差'),
('mean_coordinate_Wasserstein_standardized', '单变量 Wasserstein'),
               ('floor_fraction_MAE_floor_columns', '下限占比误差'), ('corr', '相关矩阵 RMSE')]
    fig = plt.figure(figsize=(12, 7.4))
    gs = fig.add_gridspec(2, 4, height_ratios=[1, 1.05], hspace=0.55, wspace=0.35)
    axes = [fig.add_subplot(gs[0, i]) for i in range(4)] + [fig.add_subplot(gs[1, 1:3])]
    for ax, (k, lab) in zip(axes[:4], metrics):
        vals = [agg.loc[m, k] for m, _, _ in methods]
        ax.bar(range(3), vals, color=[c for *_, c in methods], width=0.62)
        for i, v in enumerate(vals):
            ax.text(i, v * 1.01, f'{v:.3f}', ha='center', va='bottom', fontsize=8, color='#52514e')
        ax.set_xticks([]); ax.set_title(lab, fontsize=10); ax.grid(axis='x', visible=False)
        ax.set_ylim(0, max(vals) * 1.18); ax.set_xlim(-0.6, 2.6)
    axes[0].set_ylabel('留出评价（越小越好）')
```

```
    from matplotlib.patches import Patch
    fig.legend(handles=[Patch(color=c, label=l) for _, l, c in methods], loc='upper center',
bbox_to_anchor=(0.5, 0.535),
               ncol=3, frameon=False, fontsize=9)
    d = pd.read_csv(RES / 'q3' / 'generator_final_descriptive.csv', encoding='utf-8-sig')
    base = {'Tripel': 0.337, 'Kriek': 0.547, 'Lambic': 0.457}   # 首版基线（analysis/q3 首版结果）
    ax = axes[4]; x = np.arange(3); w = 0.26
    ax.bar(x - w, d.real_4VG_floor_fraction * 100, w, color='#52514e', label='实测')
    ax.bar(x, d.synthetic_4VG_floor_fraction * 100, w, color=S1, label='潜在高斯')
    ax.bar(x + w, [base[s] * 100 for s in d['style']], w, color=S2, label='首版风格高斯')
    ax.set_xticks(x); ax.set_xticklabels(d['style']); ax.set_ylabel('4VG处于下限的比例（%）')
    ax.set_xlim(-0.6, 2.6); ax.set_ylim(0, 118)
    ax.set_title('4VG下限点质量（全数据拟合）', fontsize=10); ax.legend(frameon=False, fontsize=8,
loc='upper center', ncol=3)
    ax.grid(axis='x', visible=False)
    fig.savefig(FIG / 'fig_q3_generator.png'); plt.close(fig)


# ---------- 图：Q3 迁移 ----------
def q3_mig():
    m = pd.read_csv(RES / 'q3' / 'migration_acceptance_all.csv', encoding='utf-8-sig')
    disp = pd.read_csv(RES / 'q3' / 'migration_dispersion.csv', encoding='utf-8-sig')
    fig, (a, b) = plt.subplots(1, 2, figsize=(11, 4.2))
    for meth, lab, c, mk in [('latent_OT', '共同空间OT', S1, 'o'), ('baseline_optimizer', '首版优化基
线', S2, 's')]:
        g = m[m.method == meth].groupby('alpha')[['source_change_RMS_latent',
'nn_reduction']].mean()
        a.plot(g.source_change_RMS_latent, g.nn_reduction * 100, marker=mk, color=c, lw=2, ms=7,
label=lab)
        for al, row in g.iterrows():
            if al > 0:
                a.annotate(f'α={al:g}', (row.source_change_RMS_latent, row.nn_reduction * 100),
xytext=(4, -11),
                           textcoords='offset points', fontsize=7.5, color='#52514e')
    a.axhline(10, color=MUTED, ls='--', lw=0.8); a.text(0.02, 11, '判据3门槛10%', fontsize=8,
color=MUTED)
    a.set_xlabel('源谱改变量（共同正态分数空间RMS）'); a.set_ylabel('到真实Tripel最近邻距离下降（%）')
    a.set_title('目标接近与干预幅度（11款平均）', fontsize=10.5); a.legend(frameon=False, fontsize=8.5,
loc='lower right')
    rows = []
    for _, r in disp.iterrows():
        g = r['group']
        if g.startswith('latent_OT') or g.startswith('baseline_optimizer'):
            meth, al = g.split(' alpha=')
            rows.append((meth, float(al), r.mean_pairwise_dist))
    dd = pd.DataFrame(rows, columns=['method', 'alpha', 'pair'])
    for meth, lab, c, mk in [('latent_OT', '共同空间OT', S1, 'o'), ('baseline_optimizer', '首版优化基
线', S2, 's')]:
        g = dd[dd.method == meth]
        b.plot(np.arange(len(g)), g.pair, marker=mk, color=c, lw=2, ms=7, label=lab)
        for i, (al, v) in enumerate(zip(g.alpha, g.pair)):
            b.annotate(f'{al:g}', (i, v), xytext=(0, 7 if meth == 'latent_OT' else -13),
textcoords='offset points',
                       fontsize=7.5, ha='center', color='#52514e')
    real = disp.set_index('group').loc['real_Tripel(28)', 'mean_pairwise_dist']
    b.axhline(real, color='#52514e', ls='--', lw=1); b.text(0.05, real + 0.03, f'真实Tripel
{real:.3f}', fontsize=8, color='#52514e')
    b.set_xticks([]); b.set_xlabel('权重/强度递增（点旁数字为α）')
    b.set_ylabel('迁移群体平均两两距离'); b.set_ylim(0, 1.35)
    b.set_title('多样性保留：是否向中心塌缩', fontsize=10.5); b.legend(frameon=False, fontsize=8.5,
loc='lower left')
    fig.text(0.5, -0.02, '两法的α含义与网格不同，仅作描述性对照；离散度只说明是否塌缩，不能独立说明是否属于
Tripel。',
             ha='center', fontsize=8, color=MUTED)
    fig.tight_layout(); fig.savefig(FIG / 'fig_q3_migration.png'); plt.close(fig)
```

```
if __name__ == '__main__':
    roadmap(); q1(); q2(); q3_gen(); q3_mig()
    print(sorted(p.name for p in FIG.glob('*.png')))
```

src/make_paper_figures_2.py

```
"""论文补充图表：数据探索、预测对比、生成画像与迁移细节。读取已保存结果，不重新训练模型。
AI assistance: Claude (Claude Opus 5.5), Anthropic, 2026-10-26. 用于代码实现，详见支撑材料《AI 工具使用详情说明》。
"""
import sys
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / 'src' / 'q3_style_generation'))
sys.path.insert(0, str(ROOT / 'src'))
from make_paper_figures import FIG, RES, S1, S2, S3, MUTED, INK   # noqa: E402  共用样式

x = pd.read_csv(ROOT / '1.csv').set_index('beer')
X = x.drop(columns=['beer_id', 'tasting_category_fine'])
style = x.tasting_category_fine
Y = pd.read_csv(ROOT / '4.csv').set_index('beer').loc[x.index].drop(columns=['beer_id',
'tasting_category_fine'])
GRAY = '#c9c8c2'


def eda_overview():
    fig, axes = plt.subplots(1, 3, figsize=(13, 4.4), gridspec_kw={'width_ratios': [1.25, 1, 1]})
    a = axes[0]; vc = style.value_counts().sort_values()
    a.barh(vc.index, vc.values, color=S1, height=0.7)
    for i, v in enumerate(vc.values):
        a.text(v + 0.3, i, str(v), va='center', fontsize=7.5, color='#52514e')
    a.set_title('(a) 22 种风格的样本数', fontsize=10); a.tick_params(axis='y', labelsize=7.5);
a.grid(axis='y', visible=False)
    b = axes[1]; na = X.isna().sum(); na = na[na > 0].sort_values()
    b.barh(na.index, na.values, color=S2, height=0.6)
    for i, v in enumerate(na.values):
        b.text(v + 0.3, i, str(v), va='center', fontsize=8, color='#52514e')
    b.set_title('(b) 含缺失值的 9 列（共 73 个）', fontsize=10); b.tick_params(axis='y', labelsize=7.5);
b.grid(axis='y', visible=False)
    c = axes[2]; fl = (X == X.min()).mean() * 100
    c.hist(fl, bins=np.arange(0, 90, 5), color=S3, edgecolor='white')
    c.axvline(10, color=MUTED, ls='--', lw=0.8)
    c.text(11, c.get_ylim()[1] * 0.9, f'下限占比≥10%：{(fl >= 10).sum()} 列\n下限占比≥50%：{(fl >=
50).sum()} 列', fontsize=8, color='#52514e')
    c.set_xlabel('取值等于列最小值（检出下限）的样本占比（%）'); c.set_ylabel('列数')
    c.set_title('(c) 231 列的下限点质量分布', fontsize=10)
    fig.tight_layout(); fig.savefig(FIG / 'fig_eda_overview.png'); plt.close(fig)


def eda_logcheck():
    c = pd.read_excel(ROOT / '8.xlsx', sheet_name='Calibrated chemical dataset').set_index('beer')
    m = X.join(c, rsuffix='_c', how='inner')
    cols = [('ethyl_acetate', 'log10'), ('X4VG', 'log10'), ('ethanol..v.v.', 'log10'), ('linalool',
'raw')]
    fig, axes = plt.subplots(1, 4, figsize=(13, 3.3))
    for ax, (col, kind) in zip(axes, cols):
        a, b = m[col], m[col + '_c']; ok = b > 0
        xv = np.log10(b[ok]) if kind == 'log10' else b[ok]
```

```
        ax.scatter(xv, a[ok], s=10, color=S1, alpha=0.7, edgecolor='none')
        lo, hi = float(min(xv.min(), a[ok].min())), float(max(xv.max(), a[ok].max()))
        ax.plot([lo, hi], [lo, hi], color=S2, lw=1.2, ls='--')
        ax.set_xlabel('log10(校准浓度)' if kind == 'log10' else '校准浓度（原值）', fontsize=8.5)
        ax.set_ylabel('附件1取值', fontsize=8.5)
        ax.set_title(f'{col}：{"log10 列" if kind == "log10" else "未取对数列"}', fontsize=9.5)
    fig.text(0.5, -0.03, '虚线为 y=x；四列中位绝对偏差均小于 1e-6。70 个共享指标中 37 列落在 log10 关系上、33 列落在原值关系上。',
             ha='center', fontsize=8, color=MUTED)
    fig.tight_layout(); fig.savefig(FIG / 'fig_eda_logcheck.png'); plt.close(fig)


def eda_corr():
    s5 = pd.read_csv(ROOT / '5.csv', index_col=0)
    s6 = pd.read_csv(ROOT / '6.csv', index_col=0)
    fig, (a, b) = plt.subplots(1, 2, figsize=(13, 6.2), gridspec_kw={'width_ratios': [1, 1.05]})
    im = a.imshow(s5.values, cmap='RdBu_r', vmin=-1, vmax=1)
    a.set_xticks(range(50)); a.set_yticks(range(50))
    short = [c.replace(' aroma', 'A').replace(' taste', 'F').replace(' - ', '-')[:18] for c in
s5.columns]
    a.set_xticklabels(short, rotation=90, fontsize=4.8); a.set_yticklabels(short, fontsize=4.8);
a.grid(False)
    a.set_title('(a) 50 个感官属性间 Spearman 相关（附件5）', fontsize=10)
    keep = s6.abs().max(axis=1).sort_values(ascending=False).index[:25]
    sub = s6.loc[keep]
    im2 = b.imshow(sub.values, cmap='RdBu_r', vmin=-0.85, vmax=0.85, aspect='auto')
    b.set_yticks(range(len(sub))); b.set_yticklabels(sub.index, fontsize=6.5)
    b.set_xticks(range(50)); b.set_xticklabels(short, rotation=90, fontsize=4.8); b.grid(False)
    b.set_title('(b) 化学指标×感官交叉相关（附件6，最大|ρ|前25行）', fontsize=10)
    fig.colorbar(im2, ax=b, fraction=0.03).set_label('Spearman ρ')
    fig.tight_layout(); fig.savefig(FIG / 'fig_eda_corr.png'); plt.close(fig)


def q1_pred_true():
    oof = np.load(ROOT / 'analysis/A_feasibility/floor_ablation/baseline_oof.npz',
allow_pickle=True)
    ref = pd.DataFrame(oof['prediction'].mean(0), index=oof['beer'],
columns=list(oof['fields'])).loc[x.index]
    tiers = pd.read_csv(RES / 'q1/q1_prediction_support_tiers.csv', encoding='utf-8-sig',
index_col=0)
    cols = ['acidity', 'bitternes', 'F_hops_all', 'sweetness', 'X4vg', 'dms']
    fig, axes = plt.subplots(2, 3, figsize=(12, 7.2))
    for ax, c in zip(axes.flat, cols):
        ax.scatter(Y[c], ref[c], s=12, color=S1, alpha=0.65, edgecolor='none')
        lo, hi = float(min(Y[c].min(), ref[c].min())), float(max(Y[c].max(), ref[c].max()))
        ax.plot([lo, hi], [lo, hi], color=S2, ls='--', lw=1)
        r2 = tiers.loc[c, 'R2_mean_oof']
        ax.set_title(f'{c}（{tiers.loc[c, "support_tier"]}档，R²={r2:.2f}）', fontsize=9.5)
        ax.set_xlabel('真实标准化值', fontsize=8.5); ax.set_ylabel('折外预测（5次重复均值）',
fontsize=8.5)
    fig.text(0.5, -0.01, '虚线为 y=x。预测点向均值收缩、斜率小于1，是回归预测的正常现象，也是 6.3 节采用折外预测分位锚点的原因。',
             ha='center', fontsize=8, color=MUTED)
    fig.tight_layout(); fig.savefig(FIG / 'fig_q1_pred_true.png'); plt.close(fig)


def q2_summary():
    d = pd.read_csv(RES / 'q2/q2_v31_top3_detail.csv', encoding='utf-8-sig')
    j = pd.read_csv(ROOT / 'analysis/q2/relaxed_cross_method_jaccard.csv', encoding='utf-8-sig')
    fig, (a, b) = plt.subplots(1, 2, figsize=(12, 4.3))
    vc = d.field.value_counts().head(10).sort_values()
    a.barh(vc.index, vc.values, color=S2, height=0.65)
    for i, v in enumerate(vc.values):
        a.text(v + 0.3, i, str(v), va='center', fontsize=8, color='#52514e')
    a.set_xlabel('出现在多少个感官维度的 Top-3 中'); a.set_title('(a) Top-3 中出现最多的 10 个化合物',
```

```
fontsize=10)
    a.tick_params(axis='y', labelsize=8); a.grid(axis='y', visible=False)
    cols = ['chemical_Jaccard', 'module_Jaccard']
    k = j.groupby('top_k')[cols].median()
    if True:
        xs = np.arange(len(k)); w = 0.36
        b.bar(xs - w / 2, k[cols[0]], w, color=S1, label='化合物层面')
        b.bar(xs + w / 2, k[cols[1]], w, color=S3, label='模块层面')
        for xi, (v1, v2) in enumerate(zip(k[cols[0]], k[cols[1]])):
            b.text(xi - w / 2, v1 + 0.005, f'{v1:.3f}', ha='center', fontsize=8); b.text(xi + w / 2,
v2 + 0.005, f'{v2:.3f}', ha='center', fontsize=8)
        b.set_xticks(xs); b.set_xticklabels([f'Top-{int(v)}' for v in k.index]);
b.legend(frameon=False, fontsize=8.5)
    b.set_ylabel('50 维 Jaccard 中位数'); b.set_title('(b) Extra Trees 与 PLS 名单的一致性',
fontsize=10); b.grid(axis='x', visible=False)
    fig.tight_layout(); fig.savefig(FIG / 'fig_q2_summary.png'); plt.close(fig)


def q3_space_and_details():
    import latent_gaussian_ot as L   # 重跑主脚本（确定性），取得共同映射 G
    fit = L.fit
    Z = fit.Z
    mu = Z.mean(0)
    U, Sv, Vt = np.linalg.svd(Z - mu, full_matrices=False)
    P = lambda A: (A - mu) @ Vt[:2].T
    ev = Sv ** 2 / (Sv ** 2).sum()
    gen = pd.read_csv(RES / 'q3/generated_latent_231_coordinates.csv', encoding='utf-8-sig')
    mc = pd.read_csv(RES / 'q3/migration_latent_OT_231_coordinates.csv', encoding='utf-8-sig')
    fig, (a, b) = plt.subplots(1, 2, figsize=(13, 5.4))
    pz = P(Z)
    a.scatter(pz[:, 0], pz[:, 1], s=10, color=GRAY, label='其他风格')
    for st, c in [('Tripel', S1), ('Kriek', S2), ('Lambic', S3)]:
        g = P(fit.G.transform(gen[gen['style'] == st][L.fields].to_numpy(float)))
        a.scatter(g[:, 0], g[:, 1], s=6, color=c, alpha=0.18, edgecolor='none')
        r = pz[labels_mask(st)]
        a.scatter(r[:, 0], r[:, 1], s=34, color=c, edgecolor='white', linewidth=0.8, label=f'{st} (实
测●／生成·) ')
    a.set_xlabel(f'PC1 ({ev[0] * 100:.1f}%) '); a.set_ylabel(f'PC2 ({ev[1] * 100:.1f}%) ')
    a.set_title('(a) 共同正态分数空间：实测样本与 900 条生成谱', fontsize=10); a.legend(frameon=False,
fontsize=8, loc='best')
    b.scatter(pz[:, 0], pz[:, 1], s=10, color=GRAY)
    rt = pz[labels_mask('Tripel')]; b.scatter(rt[:, 0], rt[:, 1], s=30, color=S1, edgecolor='white',
label='真实 Tripel')
    for beer in mc.beer.unique():
        path = mc[mc.beer == beer].sort_values('alpha')
        pp = P(fit.G.transform(path[L.fields].to_numpy(float)))
        b.plot(pp[:, 0], pp[:, 1], color=S2, lw=1.1, alpha=0.8)
        b.scatter(pp[0, 0], pp[0, 1], s=30, color=S2, marker='s', edgecolor='white', zorder=3)
        b.scatter(pp[2, 0], pp[2, 1], s=36, color=S2, marker='*', edgecolor='white', zorder=3)
    b.scatter([], [], color=S2, marker='s', label='Pils/Lager 源谱'); b.scatter([], [], color=S2,
marker='*', label='OT 候选 (α=0.5) ')
    b.set_xlabel(f'PC1 ({ev[0] * 100:.1f}%) '); b.set_ylabel(f'PC2 ({ev[1] * 100:.1f}%) ')
    b.set_title('(b) Pils/Lager → Tripel 的 OT 迁移路径 (α=0→1) ', fontsize=10);
b.legend(frameon=False, fontsize=8)
    fig.tight_layout(); fig.savefig(FIG / 'fig_q3_space.png'); plt.close(fig)

    # 缺陷浓度比与陈化指数
    acc = pd.read_csv(RES / 'q3/migration_acceptance_all.csv', encoding='utf-8-sig')
    sel = mc[mc.alpha == 0.5].set_index('beer')
    defects = {'DMS': ('DMS', True), '双乙酰': ('X2.3.butanedione', False), '4VG': ('X4VG', True),
't2n': ('X.E..2.nonenal', False)}
    R = pd.DataFrame(index=sel.index, columns=list(defects), dtype=float)
    for name, (f, islog) in defects.items():
        s0 = X.loc[sel.index, f]; s1 = sel[f]
        R[name] = (10 ** (s1 - s0)) if islog else (s1 / s0)
```

```
    fig, (a, b) = plt.subplots(1, 2, figsize=(12.5, 4.4), gridspec_kw={'width_ratios': [1.1, 1]})
    M = R.to_numpy(float)
    im = a.imshow(M, cmap='Blues_r', vmin=0, vmax=1, aspect='auto')
    for i in range(M.shape[0]):
        for k in range(M.shape[1]):
            t = '缺失' if np.isnan(M[i, k]) else f'{M[i, k]:.2f}'
            a.text(k, i, t, ha='center', va='center', fontsize=8, color='white' if (not
np.isnan(M[i, k]) and M[i, k] < 0.5) else INK)
    a.set_xticks(range(4)); a.set_xticklabels(list(defects)); a.set_yticks(range(len(R)));
a.set_yticklabels([f'beer {b_}' for b_ in R.index], fontsize=8)
    a.grid(False); fig.colorbar(im, ax=a, fraction=0.04).set_label('候选/源 浓度比')
    a.set_title('(a) OT 候选（α=0.5）四项缺陷浓度与源谱之比（≤1 即非增）', fontsize=9.5)
    g = acc[(acc.method == 'latent_OT') & (acc.alpha == 0.5)].set_index('beer').loc[R.index]
    y = np.arange(len(g))
    for yi, (_, r) in zip(y, g.iterrows()):
        if np.isfinite(r.aging_index_source):
            b.plot([r.aging_index_source, r.aging_index_candidate], [yi, yi], color=GRAY, lw=2)
    b.scatter(g.aging_index_source, y, color=S2, s=40, label='源谱', zorder=3)
    b.scatter(g.aging_index_candidate, y, color=S1, s=40, marker='D', label='OT 候选', zorder=3)
    b.set_yticks(y); b.set_yticklabels([f'beer {b_}' for b_ in g.index], fontsize=8)
    b.set_xlabel('陈化方向指数 A(x)（越小越"新鲜"）'); b.legend(frameon=False, fontsize=8.5)
    b.invert_yaxis(); b.set_title('(b) 陈化方向指数变化（源缺项样本仅显示候选）', fontsize=9.5)
    fig.tight_layout(); fig.savefig(FIG / 'fig_q3_defects.png'); plt.close(fig)


def labels_mask(st):
    return (style == st).to_numpy()


def q3_profile():
    p = pd.read_csv(RES / 'q3/generated_sensory_profile.csv', encoding='utf-8-sig')
    p = p[p.support_tier.isin(['A', 'B']) & (p.type == 'numeric')]
    order = p[p['style'] == 'Tripel'].sort_values('real_style_mean_standardized').target.tolist()
    fig, axes = plt.subplots(1, 3, figsize=(13, 5.6), sharey=True)
    for ax, st, c in zip(axes, ['Tripel', 'Kriek', 'Lambic'], [S1, S2, S3]):
        g = p[p['style'] == st].set_index('target').loc[order]
        yy = np.arange(len(g))
        for yi, (_, r) in zip(yy, g.iterrows()):
            ax.plot([r.real_style_mean_standardized, r.pred_mean_standardized], [yi, yi],
color=GRAY, lw=1.6)
        ax.scatter(g.real_style_mean_standardized, yy, color='#52514e', s=24, label='实测风格均值',
zorder=3)
        ax.scatter(g.pred_mean_standardized, yy, color=c, s=30, marker='D', label='生成谱预测均值',
zorder=3)
        ax.axvline(0, color=MUTED, lw=0.8)
        ax.set_title(st, fontsize=10.5); ax.set_xlabel('标准化感官值'); ax.legend(frameon=False,
fontsize=8, loc='lower right')
    axes[0].set_yticks(range(len(order))); axes[0].set_yticklabels(order, fontsize=8)
    fig.text(0.5, -0.01, '仅列 A/B 档 numeric 属性；生成谱感官值由问题一最终前向模型给出，幅度因回归收缩而偏小。',
 ha='center', fontsize=8, color=MUTED)
    fig.tight_layout(); fig.savefig(FIG / 'fig_q3_profile.png'); plt.close(fig)


if __name__ == '__main__':
    eda_overview(); eda_logcheck(); eda_corr(); q1_pred_true(); q2_summary(); q3_profile();
q3_space_and_details()
    print('done')
```

src/q1_sensory_imputation/build_relative_scores.py

```
"""v3.1 第1项：基于折外预测的相对评分、预测支持度分档、250 款主表与12 张评分卡。

只读取已有的5 次重复 OOF 预测与逐维评价，不重新训练模型。
规则见 docs/解题思路.md 第1.4–1.7 节。
AI assistance: Claude (Claude Opus 5.5), Anthropic, 2026-10-26. 用于代码实现，详见支撑材料《AI 工具使用详情
说明》。
"""
```

```python
import json
from pathlib import Path

import numpy as np
import openpyxl
import pandas as pd

ROOT = Path(__file__).resolve().parents[2]
FEAS = ROOT / 'analysis' / 'A_feasibility'
OUT = ROOT / 'results' / 'q1'
OUT.mkdir(parents=True, exist_ok=True)

# ---------- 读取已有结果 ----------
oof = np.load(FEAS / 'floor_ablation' / 'baseline_oof.npz', allow_pickle=True)
pred, beers, fields = oof['prediction'], oof['beer'].astype(int), list(oof['fields'])
assert pred.shape == (5, len(beers), len(fields))
splits = json.load(open(FEAS / 'floor_ablation' / 'split_and_floor_log.json', encoding='utf-8'))
assert len(splits) == 15

y = pd.read_csv(ROOT / '4.csv').set_index('beer')
assert list(y.columns[2:]) == fields
y = y[fields]
meta = pd.read_csv(FEAS / 'day1' / 'sensory_metadata.csv', encoding='utf-8-sig').set_index('R_name')
types = meta.loc[fields, 'DataType']
numeric = [f for f in fields if types[f] == 'numeric']
categorical = [f for f in fields if types[f] == 'categorical']
dims = pd.read_csv(FEAS / 'q1_metrics' / 'ExtraTrees_dimension_summary.csv', encoding='utf-8-sig',
index_col=0).loc[fields]


def parent_of(col):
    return '_'.join(col.split('_')[:2]) + '_all'


for c in categorical:
    assert types[parent_of(c)] == 'numeric'

# ---------- 参考预测：每款啤酒5次重复OOF均值 ----------
ref = pd.DataFrame(pred.mean(axis=0), index=beers, columns=fields)
ref.index.name = 'beer'

# 每款啤酒在其作为测试集的5个划分中，对应训练折的真实均值，再跨重复平均
train_mean = pd.DataFrame(0.0, index=beers, columns=fields)
count = pd.Series(0, index=beers)
test_seen = {r: set() for r in range(5)}
for s in splits:
    tr = s['train_beer']
    te = sorted(set(beers) - set(tr))
    test_seen[s['repeat']].update(te)
    train_mean.loc[te] += y.loc[tr].mean().values
    count.loc[te] += 1
assert (count == 5).all() and all(len(v) == len(beers) for v in test_seen.values())
train_mean = train_mean.div(count, axis=0)

# ---------- numeric：P5/P95 锚点（冻结） ----------
q05, q95 = ref[numeric].quantile(0.05), ref[numeric].quantile(0.95)
no_info = (q95 - q05) <= 1e-12
anchors = pd.DataFrame({'q05_oof': q05, 'q95_oof': q95, 'no_discrimination': no_info})


def relative_score(z):
    """z: DataFrame of standardized predictions for numeric fields -> integer 0–6 (NaN if no
discrimination)."""
    s = 6 * (z[numeric] - q05) / (q95 - q05)
    s = s.clip(0, 6).round()
```

```
    s.loc[:, no_info[no_info].index] = np.nan
    return s


score_num = relative_score(ref)

# ---------- categorical：相对勾选倾向 + 父项门控 ----------
tick = pd.DataFrame(index=ref.index, columns=categorical, dtype=int)
for c in categorical:
    tick[c] = ((score_num[parent_of(c)] > 0) & (ref[c] > train_mean[c])).astype(int)
display = pd.DataFrame(index=ref.index, columns=categorical, dtype=float)
for c in categorical:
    display[c] = tick[c] * score_num[parent_of(c)]

# ---------- 预测支持度分档 ----------
r2 = dims['R2']
tier = pd.Series(np.where(r2 >= 0.3, 'A', np.where(r2 >= 0.1, 'B', 'C')), index=fields)
support = pd.DataFrame({
    'type': types, 'R2_mean_oof': r2, 'Spearman': dims['Spearman'], 'MAE': dims['MAE'],
    'MSE_skill_vs_style': dims['MSE_skill_vs_style'],
    'support_tier': tier,
    'skill_positive': dims['MSE_skill_vs_style'] > 0,
})
support['flag'] = np.where(~support.skill_positive, '未优于风格均值', '')
support.loc[categorical, 'final_tick_rate'] = tick.mean()
support.loc[categorical, 'parent_positive_rate'] = [(score_num[parent_of(c)] > 0).mean() for c in
categorical]
support.loc[numeric, 'score_dist'] = [
    json.dumps(score_num[c].value_counts().sort_index().astype(int).to_dict()) for c in numeric]
support = support.join(anchors)

# ---------- 写出主表 ----------
style = pd.read_csv(ROOT / '1.csv').set_index('beer')['tasting_category_fine']
ref.join(style).to_csv(OUT / 'q1_oof_mean_standardized_250.csv', encoding='utf-8-sig')
train_mean.to_csv(OUT / 'q1_train_fold_mean_reference_250.csv', encoding='utf-8-sig')
score_num.join(style).to_csv(OUT / 'q1_numeric_relative_scores_250.csv', encoding='utf-8-sig')
tick.join(style).to_csv(OUT / 'q1_categorical_ticks_250.csv', encoding='utf-8-sig')
display.join(style).to_csv(OUT / 'q1_categorical_template_display_250.csv', encoding='utf-8-sig')
support.to_csv(OUT / 'q1_prediction_support_tiers.csv', encoding='utf-8-sig')
anchors.to_json(OUT / 'q1_frozen_anchors.json', force_ascii=False, indent=1, orient='index')

# ---------- 填写12张评分卡（不改动原始3.xlsx） ----------
# 强度行（各自的0–6标记行号）
A_ROWS = {'malt': 7, 'hops': 20, 'esters': 33}
SUB_ROWS = {  # 子项勾选行，顺序与4.csv 字段一致
    'malt': {'grain': 10, 'bread': 12, 'cara': 14, 'burn': 16},
    'hops': {'citrus': 23, 'tropical': 25, 'noble': 27, 'woody': 29},
    'esters': {'ethac': 36, 'isoaa': 38, 'flower': 40, 'fruity': 42},
}
LEFT_GRADE_COLS = ['C', 'E', 'G', 'I', 'K', 'M', 'O']
RIGHT_GRADE_COLS = ['S', 'U', 'W', 'Y', 'AA', 'AC', 'AE']
SINGLE = {  # field -> (row, side)
    'body': (50, 'L'), 'alcohol': (50, 'R'), 'bitternes': (54, 'L'), 'sweetness': (54, 'R'),
    'co2': (58, 'L'), 'acidity': (58, 'R'), 'aftertaste': (62, 'L'),
    'dms': (68, 'L'), 't2n': (68, 'R'), 'metallic': (72, 'L'), 'diacetyl': (72, 'R'),
    'X4vg': (76, 'L'), 'stale_hops': (76, 'R'),
    'orange': (82, 'L'), 'lactic': (82, 'R'), 'coriander': (86, 'L'), 'acetic': (86, 'R'),
    'clove': (90, 'L'), 'barnyard': (90, 'R'), 'overall': (94, 'L'),
}
MARK = 'X'
wb = openpyxl.load_workbook(ROOT / '3.xlsx')
filled = []
for k in range(1, 13):
    ws = wb[f'beer_{k}']
    b = int(ws['S3'].value)
```

```
    assert b == k and b in score_num.index
    grade_cols = {'L': LEFT_GRADE_COLS, 'R': RIGHT_GRADE_COLS}

    def put_grade(row, side, val):
        if pd.isna(val):
            return
        ws[f'{grade_cols[side][int(val)]}{row}'] = MARK

    for side, prefix in (('L', 'A'), ('R', 'F')):
        for grp, row in A_ROWS.items():
            put_grade(row, side, score_num.loc[b, f'{prefix}_{grp}_all'])
            for sub, srow in SUB_ROWS[grp].items():
                if tick.loc[b, f'{prefix}_{grp}_{sub}'] == 1:
                    ws[f'{"C" if side == "L" else "S"}{srow}'] = MARK
    for f, (row, side) in SINGLE.items():
        put_grade(row, side, score_num.loc[b, f])
    c_fields = [f for f in fields if tier[f] == 'C']
    ws['S95'] = ('相对评分（基于折外预测，非原始品评分）。预测支持度C档（参考分）：' + ', '.join(c_fields))
    filled.append(b)

# 模板 summary 表中 barnyard 列（AX）误引用了 Stale hops 单元格 $AG$76，应为 $AG$90
fixes = []
sm = wb['summary']
for r in range(2, 14):
    cell = sm[f'AX{r}']
    if isinstance(cell.value, str) and cell.value.endswith('!$AG$76'):
        cell.value = cell.value.replace('!$AG$76', '!$AG$90')
        fixes.append(cell.coordinate)
wb.save(OUT / '3_filled_v31.xlsx')

# 评分卡对应数值（便于不经 Excel 重算直接核对）
cards = score_num.loc[filled].join(display.loc[filled]).join(style)
cards.to_csv(OUT / 'q1_scorecards_12_values.csv', encoding='utf-8-sig')

summary = {
    'rule_numeric': 'round(clip(6*(z-q05_oof)/(q95_oof-q05_oof),0,6)); anchors from 250 five-repeat
OOF means, frozen',
    'rule_categorical': 'tick = 1(parent relative score>0) * 1(OOF mean > train-fold mean averaged
over the beer\'s 5 test folds)',
    'tier_rule': 'A: R2>=0.3, B: 0.1<=R2<0.3, C: R2<0.1 (5-repeat mean OOF R2)',
    'tier_counts': tier.value_counts().sort_index().to_dict(),
    'tier_counts_numeric': tier[numeric].value_counts().sort_index().to_dict(),
    'tier_counts_categorical': tier[categorical].value_counts().sort_index().to_dict(),
    'skill_nonpositive_fields': support.index[~support.skill_positive].tolist(),
    'no_discrimination_fields': no_info[no_info].index.tolist(),
    'numeric_score_distribution':
score_num.stack().value_counts(normalize=True).sort_index().round(4).to_dict(),
    'final_tick_rate_range': [float(tick.mean().min()), float(tick.mean().max())],
    'final_tick_rate_median': float(tick.mean().median()),
    'scorecards_filled_beer': filled,
    'template_formula_fixes': fixes,
    'template_rows_left_blank': ['Acetaldehyde (row 45): no corresponding target in 4.csv'],
}
json.dump(summary, open(OUT / 'summary.json', 'w', encoding='utf-8'), ensure_ascii=False, indent=1,
default=str)
print(json.dumps(summary, ensure_ascii=False, indent=1, default=str))
```

src/q2_compound_flavor/build_v31_tables.py

```
"""v3.1 第2项：Q2结果表改用统一预测支持度 + 解释稳定性 + 高度相关备选。

只重组已有100次bootstrap结果（analysis/q2/），不重训模型、不重做bootstrap。
规则见 docs/解题思路.md 第1.6节与问题2。
AI assistance: Claude (Claude Opus 5.5), Anthropic, 2026-10-26. 用于代码实现，详见支撑材料《AI 工具使用详情
说明》。
"""
import json
```

```python
from pathlib import Path

import numpy as np
import pandas as pd

ROOT = Path(__file__).resolve().parents[2]
Q2 = ROOT / 'analysis' / 'q2'
OUT = ROOT / 'results' / 'q2'
OUT.mkdir(parents=True, exist_ok=True)
ALT_RHO, ALT_MAX = 0.8, 3

support = pd.read_csv(ROOT / 'results' / 'q1' / 'q1_prediction_support_tiers.csv', encoding='utf-8-
sig', index_col=0)
old = pd.read_csv(Q2 / 'reliability_tiers.csv', encoding='utf-8-sig').set_index('target')
top3 = pd.read_csv(Q2 / 'top3_with_MC_frequency_intervals.csv', encoding='utf-8-sig')
mapping = pd.read_csv(Q2 / 'chemical_mapping_and_modules.csv', encoding='utf-8-
sig').set_index('field')
rho = pd.read_csv(ROOT / '2.csv', index_col=0)
assert mapping.matrix_name.isin(rho.index).all()

# ---------- 解释稳定性（不含 skill 条件；skill 在预测支持度维度单列） ----------
imp_pos = top3.groupby('target').mean_OOB_permutation_MAE_increase.apply(lambda s: bool((s >
0).all()))
min_freq = top3.groupby('target').top3_frequency.min()
stab = pd.Series('探索性', index=old.index)
stab[old.max_non_singleton_module_frequency >= 0.7] = '模块线索'
stab[imp_pos & (min_freq >= 0.5)] = '单体稳定'

# ---------- 高度相关备选：同模块且 |rho|>=0.8，最多 3 个 ----------
# 非单体化学指标（理化/派生/混合，如 kcalperc）不作为化合物备选，单列为"派生相关指标"
def correlated_peers(field, eligible):
    mod = mapping.loc[field, 'module']
    peers = mapping.index[(mapping.module == mod) & (mapping.index != field)
                          & (mapping.eligible_primary_top3 == eligible)]
    if len(peers) == 0:
        return ''
    r = rho.loc[mapping.loc[field, 'matrix_name'], mapping.loc[peers, 'matrix_name']]
    r.index = peers
    r = r[r.abs() >= ALT_RHO]
    r = r.reindex(r.abs().sort_values(ascending=False).index)[:ALT_MAX]
    return '; '.join(f'{f}({"+" if v > 0 else "−"}{abs(v):.2f})' for f, v in r.items())


top3['high_corr_alternatives'] = top3.field.map(lambda f: correlated_peers(f, True))
top3['derived_correlated_indicators'] = top3.field.map(lambda f: correlated_peers(f, False))
top3['module_size'] = top3.module.map(mapping.module.value_counts())

# ---------- 汇总表 ----------
t = pd.DataFrame(index=old.index)
t['type'] = support.loc[t.index, 'type']
t['R2_mean_oof'] = support.loc[t.index, 'R2_mean_oof']
t['support_tier'] = support.loc[t.index, 'support_tier']
t['MSE_skill_vs_style'] = support.loc[t.index, 'MSE_skill_vs_style']
t['skill_flag'] = np.where(t.MSE_skill_vs_style > 0, '', '未优于风格均值')
t['interpretation_stability'] = stab
t['mean_top3_freq'] = old.mean_final_top3_selection_frequency
t['min_top3_freq'] = min_freq
t['all_top3_importance_positive'] = imp_pos
t['max_non_singleton_module_freq'] = old.max_non_singleton_module_frequency
t['old_combined_tier_historical'] = old.tier
t['top3'] = top3.sort_values('rank').groupby('target').field.apply(' | '.join)


def usage(r):
    if r.support_tier == 'C':
```

```
        return '探索性（预测支持度C，不支撑主要化学结论）'
    if r.skill_flag:
        return '谨慎：未优于风格均值'
    return {'单体稳定': '可重点报告单体候选', '模块线索': '报告模块线索', '探索性': '仅列候选'}
[r.interpretation_stability]


t['usage'] = t.apply(usage, axis=1)
order = {'A': 0, 'B': 1, 'C': 2}
t = t.sort_values(['support_tier', 'R2_mean_oof'], key=lambda s: s.map(order) if s.name ==
'support_tier' else -s)
t.to_csv(OUT / 'q2_v31_target_summary.csv', encoding='utf-8-sig')

cols = ['target', 'rank', 'field', 'chemical_name', 'origin', 'module', 'module_size',
        'mean_OOB_permutation_MAE_increase', 'top3_frequency', 'frequency_MC_Wilson_lower',
'frequency_MC_Wilson_upper',
        'module_top3_frequency', 'observed_spearman_descriptive', 'high_corr_alternatives',
        'derived_correlated_indicators']
detail = top3[cols].merge(t[['support_tier', 'skill_flag', 'interpretation_stability', 'usage']],
                          left_on='target', right_index=True)
detail.to_csv(OUT / 'q2_v31_top3_detail.csv', index=False, encoding='utf-8-sig')

cross = pd.crosstab(t.support_tier, t.interpretation_stability)
summary = {
    'support_tier_counts': t.support_tier.value_counts().sort_index().to_dict(),
    'interpretation_stability_counts': t.interpretation_stability.value_counts().to_dict(),
    'crosstab': cross.to_dict(),
    'usage_counts': t.usage.value_counts().to_dict(),
    'n_top3_with_alternatives': int((detail.high_corr_alternatives != '').sum()),
    'n_top3_with_derived_indicators': int((detail.derived_correlated_indicators != '').sum()),
    'n_top3_total': len(detail),
    'stability_rule': '单体稳定: 3个候选平均置换重要性均>0且入选率均>=50%; 模块线索: 非单点模块最高命中率>=70%;
其余探索性',
    'alternatives_rule': f'same module, |Spearman rho| >= {ALT_RHO} (2.csv), at most {ALT_MAX}; non-
compound indicators listed separately as derived_correlated_indicators',
}
json.dump(summary, open(OUT / 'summary.json', 'w', encoding='utf-8'), ensure_ascii=False, indent=1)
print(json.dumps(summary, ensure_ascii=False, indent=1))
print(t[['support_tier', 'skill_flag', 'interpretation_stability', 'top3']].to_string())
```

src/q2_compound_flavor/within_style_permutation.py

```
"""Q2 敏感性实验：全局置换 vs 风格内置换（同一测试折、同一模型、同一评价样本集合）。

沿用Q1 正式OOF 的15 个划分与Extra Trees 固定配置重新拟合15 次（不搜索超参数）；
不改动原OOF 预测与100 次bootstrap，不重算入选率，不替换原Top-3 主名单。
风格内置换只支持"在已记录风格分组内仍存在预测依赖"，不构成化学因果结论。
AI assistance: Claude (Claude Opus 5.5), Anthropic, 2026-10-26. 用于代码实现，详见支撑材料《AI 工具使用详情
说明》。
"""
import json
from pathlib import Path

import numpy as np
import pandas as pd
from sklearn.ensemble import ExtraTreesRegressor
from sklearn.impute import SimpleImputer
from threadpoolctl import threadpool_limits

ROOT = Path(__file__).resolve().parents[2]
FEAS = ROOT / 'analysis' / 'A_feasibility'
Q2 = ROOT / 'analysis' / 'q2'
OUT = ROOT / 'results' / 'q2'
R, SEED, BATCH = 3, 20261027, 12
threadpool_limits(2)

x = pd.read_csv(ROOT / '1.csv').set_index('beer')
```

```
y = pd.read_csv(ROOT / '4.csv').set_index('beer').loc[x.index]
X = x.drop(columns=['beer_id', 'tasting_category_fine'])
Y = y.drop(columns=['beer_id', 'tasting_category_fine'])
style = x.tasting_category_fine
fields, targets = list(X), list(Y)
mapping = pd.read_csv(Q2 / 'chemical_mapping_and_modules.csv', encoding='utf-8-
sig').set_index('field').loc[fields]
eligible = np.where(mapping.eligible_primary_top3.to_numpy())[0]
top3 = pd.read_csv(Q2 / 'top3_chemical_candidates.csv', encoding='utf-8-sig')
splits = json.loads((FEAS / 'floor_ablation' / 'split_and_floor_log.json').read_text(encoding='utf-
8'))
oof = np.load(FEAS / 'floor_ablation' / 'baseline_oof.npz', allow_pickle=True)
assert list(oof['fields']) == targets and (oof['beer'] == X.index.to_numpy()).all()
pos = {b: i for i, b in enumerate(X.index)}

imp = {k: np.zeros((15, len(targets), len(fields))) for k in ['global', 'within']}
log, max_oof_diff = [], 0.0
for idx, s in enumerate(splits):
    tr = np.array([pos[b] for b in s['train_beer']])
    te = np.array([pos[b] for b in s['test_beer']])
    imputer = SimpleImputer(strategy='median')
    at = imputer.fit_transform(X.to_numpy(float)[tr])
    bt = imputer.transform(X.to_numpy(float)[te])
    et = ExtraTreesRegressor(n_estimators=160, min_samples_leaf=3, max_features=.7,
                             random_state=20261025 + idx, n_jobs=2).fit(at, Y.to_numpy()[tr])
    pred_all = et.predict(bt)
    max_oof_diff = max(max_oof_diff, float(np.abs(pred_all - oof['prediction'][s['repeat']]
[te]).max()))
    # 评价集合：测试折内同风格至少2款的样本（两种置换共用）
    st = style.to_numpy()[te]
    counts = pd.Series(st).value_counts()
    keep = np.isin(st, counts.index[counts >= 2])
    E, yE, groups = bt[keep], Y.to_numpy()[te][keep], st[keep]
    n = len(E)
    base_err = np.abs(et.predict(E) - yE).mean(0)
    rng = np.random.default_rng(SEED + idx)
    gidx = [np.where(groups == g)[0] for g in np.unique(groups)]
    for first in range(0, len(fields), BATCH):
        cols = list(range(first, min(first + BATCH, len(fields))))
        batch = np.tile(E, (len(cols) * R * 2, 1))
        for u, j in enumerate(cols):
            for r in range(R):
                pg = rng.permutation(n)
                pw = np.arange(n)
                for g in gidx:
                    pw[g] = rng.permutation(g)
                for k, perm in enumerate([pg, pw]):
                    sl = slice(((u * R + r) * 2 + k) * n, ((u * R + r) * 2 + k + 1) * n)
                    batch[sl, j] = E[perm, j]
        P = et.predict(batch).reshape(len(cols), R, 2, n, len(targets))
        d = np.abs(P - yE[None, None, None]).mean(3).mean(1) - base_err[None, None]   # (cols, 2,
targets)
        imp['global'][idx][:, cols] = d[:, 0].T
        imp['within'][idx][:, cols] = d[:, 1].T
    log.append({'split': idx, 'repeat': s['repeat'], 'fold': s['fold'], 'n_test': len(te),
'n_evaluable': int(n),
                'n_styles_evaluable': len(gidx), 'excluded_styles': counts.index[counts <
2].tolist()})
    print(f'split {idx + 1}/15: evaluable {n}/{len(te)}', flush=True)

G, W = imp['global'].mean(0), imp['within'].mean(0)
pd.DataFrame(G, index=targets, columns=fields).to_csv(OUT / 'perm_global_importance_mean15.csv',
encoding='utf-8-sig')
pd.DataFrame(W, index=targets, columns=fields).to_csv(OUT /
'perm_within_style_importance_mean15.csv', encoding='utf-8-sig')
```

```
def rank_among_eligible(vec, j):
    order = eligible[np.argsort(-vec[eligible], kind='stable')]
    return int(np.where(order == j)[0][0]) + 1


rows = []
for _, r in top3.iterrows():
    t, j = targets.index(r.target), fields.index(r.field)
    g, w = imp['global'][:, t, j], imp['within'][:, t, j]
    rows.append({'target': r.target, 'rank_original': r['rank'], 'field': r.field,
                 'global_imp': G[t, j], 'within_imp': W[t, j],
                 'retention_ratio': W[t, j] / G[t, j] if G[t, j] > 0 else np.nan,
                 'within_positive_splits': int((w > 0).sum()), 'paired_within_minus_global_mean':
float((w - g).mean()),
                 'global_rank_217': rank_among_eligible(G[t], j), 'within_rank_217':
rank_among_eligible(W[t], j)})
res = pd.DataFrame(rows)
res['within_top3'] = res.within_rank_217 <= 3
res['within_top10'] = res.within_rank_217 <= 10
res.to_csv(OUT / 'q2_within_style_permutation_top3.csv', index=False, encoding='utf-8-sig')
pd.DataFrame(log).to_csv(OUT / 'q2_within_style_permutation_log.csv', index=False, encoding='utf-8-
sig')

per_field = res.groupby('field').agg(n_targets=('target', 'size'),
median_retention=('retention_ratio', 'median'),
                                     share_within_positive=('within_imp', lambda v: float((v >
0).mean())),
                                     share_within_top10=('within_top10',
'mean')).sort_values('n_targets', ascending=False)
per_field.to_csv(OUT / 'q2_within_style_permutation_by_field.csv', encoding='utf-8-sig')

# 原 Top-3 的排序在风格内置换下是否保留（同一目标的3 个候选之间）
kept = []
for t, g in res.groupby('target'):
    kept.append(list(g.sort_values('rank_original').within_imp.rank(ascending=False).astype(int)) ==
[1, 2, 3])
summary = {
    'refit_models': 15, 'refit_matches_saved_OOF_max_abs_diff': max_oof_diff,
    'evaluable_samples_total': int(sum(l['n_evaluable'] for l in log)),
    'test_samples_total': int(sum(l['n_test'] for l in log)),
    'top3_entries': len(res),
    'share_global_imp_positive': float((res.global_imp > 0).mean()),
    'share_within_imp_positive': float((res.within_imp > 0).mean()),
    'median_retention_ratio': float(res.retention_ratio.median()),
    'share_within_top10_of_217': float(res.within_top10.mean()),
    'share_within_top3_of_217': float(res.within_top3.mean()),
    'targets_with_original_order_kept_within': int(sum(kept)),
    'top_fields': per_field.head(6).round(3).to_dict('index'),
    'permutation_repeats': R,
}
json.dump(summary, open(OUT / 'within_style_permutation_summary.json', 'w', encoding='utf-8'),
ensure_ascii=False, indent=1)
print(json.dumps(summary, ensure_ascii=False, indent=1))
```

src/q3_style_generation/latent_gaussian_ot.py

```
"""v3.1 第3、4 项：经验边际潜在高斯生成 + 共同空间高斯最优传输迁移。

  x --G--> z --(风格条件收缩高斯 / OT)--> z' --G^-1--> x' --缺陷投影--> x''

G 由训练集全体风格共用的逐列经验 CDF 构建（中间秩处理并列，逆变换保留并列点质量）。
生成器与 analysis/q3 首版基线使用相同的三折划分、相同的随机数，并在相同的标准化坐标中评价。
规则见 docs/解题思路.md 第3 节。
AI assistance: Claude (Claude Opus 5.5), Anthropic, 2026-10-26. 用于代码实现，详见支撑材料《AI 工具使用详情
说明》。
```

```
"""
import json
from pathlib import Path

import joblib
import numpy as np
import pandas as pd
from scipy.spatial.distance import cdist
from scipy.stats import norm, wasserstein_distance
from sklearn.covariance import LedoitWolf
from sklearn.impute import SimpleImputer
from sklearn.model_selection import StratifiedKFold
from sklearn.preprocessing import StandardScaler

ROOT = Path(__file__).resolve().parents[2]
BASE = ROOT / 'analysis' / 'q3'
OUT = ROOT / 'results' / 'q3'
OUT.mkdir(parents=True, exist_ok=True)
SEED, NGEN, STYLES = 20261026, 300, ['Tripel', 'Kriek', 'Lambic']   # 与首版基线一致
ALPHAS = [0, 0.25, 0.5, 0.75, 1.0]
CENTER_GAIN, NN_GAIN = 0.20, 0.10                                    # 冻结验收门槛
FLOOR_SHARE_MIN = 0.10                                               # 统计"下限点质量"的列：训练下限占比
≥10%

x = pd.read_csv(ROOT / '1.csv').set_index('beer')
X = x.drop(columns=['beer_id', 'tasting_category_fine'])
labels = x.tasting_category_fine.to_numpy()
fields = list(X)
p = len(fields)
config = json.loads((BASE / 'defect_thresholds.json').read_text(encoding='utf-8'))
DEFECTS = {k: v['field'] for k, v in config['measured_defects'].items()}


# ---------------------------------------------------------------------- 共同映射 G
class EmpiricalMarginals:
    """逐列经验 CDF。并列值取中间秩；逆变换对并列块返回常数（保留点质量），单点之间线性插值。"""
    EPS = 1e-9

    def fit(self, B):
        self.fwd, self.inv = [], []
        n = B.shape[0]
        for j in range(B.shape[1]):
            vals, cnt = np.unique(B[:, j], return_counts=True)
            b = np.cumsum(cnt) / n
            a = b - cnt / n
            mid = (a + b) / 2
            self.fwd.append((vals, mid))
            U, V = [], []
            for v, c, lo, hi, m in zip(vals, cnt, a, b, mid):
                if c > 1:
                    U += [lo + self.EPS, hi - self.EPS]
                    V += [v, v]
                else:
                    U.append(m)
                    V.append(v)
            self.inv.append((np.array(U), np.array(V)))
        return self

    def cdf(self, B):
        return np.column_stack([np.interp(B[:, j], *self.fwd[j]) for j in range(B.shape[1])])

    def transform(self, B):
        return norm.ppf(self.cdf(B))

    def inverse(self, Z):
```

```
        Uq = norm.cdf(Z)
        return np.column_stack([np.interp(Uq[:, j], *self.inv[j]) for j in range(Z.shape[1])])


class LatentStyles:
    def fit(self, A, styles):
        self.imp = SimpleImputer(strategy='median')
        B = self.imp.fit_transform(np.asarray(A, dtype=float))
        self.G = EmpiricalMarginals().fit(B)
        self.Z = self.G.transform(B)
        self.models = {'__all__': LedoitWolf().fit(self.Z)}
        for s in np.unique(styles):
            self.models[s] = LedoitWolf().fit(self.Z[styles == s])
        self.roots = {}
        for k, m in self.models.items():
            v, Q = np.linalg.eigh(m.covariance_)
            self.roots[k] = Q * np.sqrt(np.maximum(v, 0))[None, :]
        return self

    def sample(self, key, normals):
        return self.G.inverse(normals @ self.roots[key].T + self.models[key].location_)

    def transform(self, A):
        return self.G.transform(self.imp.transform(np.asarray(A, dtype=float)))


class BaselineStyles:
    """首版基线（analysis/q3/run_gaussian_baseline.py 的GaussianStyles），用于同折配对比较与评价坐标。"""

    def fit(self, A, styles):
        self.imp = SimpleImputer(strategy='median')
        B = self.imp.fit_transform(np.asarray(A, dtype=float))
        self.scaler = StandardScaler().fit(B)
        self.Z = self.scaler.transform(B)
        self.low, self.high = B.min(axis=0), B.max(axis=0)
        self.models = {'__all__': LedoitWolf().fit(self.Z)}
        for s in np.unique(styles):
            self.models[s] = LedoitWolf().fit(self.Z[styles == s])
        self.roots = {}
        for k, m in self.models.items():
            v, Q = np.linalg.eigh(m.covariance_)
            self.roots[k] = Q * np.sqrt(np.maximum(v, 0))[None, :]
        return self

    def sample(self, key, normals):
        m = self.models[key]
        B = self.scaler.inverse_transform(normals @ self.roots[key].T + m.location_)
        return np.clip(B, self.low, self.high)

    def transform(self, A):
        return self.scaler.transform(self.imp.transform(np.asarray(A, dtype=float)))


def corr(A):
    B = A - A.mean(0)
    den = np.sqrt((B * B).sum(0))
    return (B.T @ B) / np.maximum(np.outer(den, den), 1e-15), den > 1e-12


def offdiag_share(cov):
    d = np.sqrt(np.diag(cov))
    C = cov / np.outer(d, d)
    return float((C ** 2).sum() - (np.diag(C) ** 2).sum()) / float((C ** 2).sum())
```

```
# ---------------------------------------------------------------- 生成：三折留出保真（与基线配对）
splits = json.loads((BASE / 'generator_holdout_splits.json').read_text(encoding='utf-8'))
rows, corr_rows = [], []
for fold, (train, test) in enumerate(StratifiedKFold(3, shuffle=True, random_state=SEED).split(X,
labels)):
    assert X.index[train].tolist() == splits[fold]['train_beer']
    base = BaselineStyles().fit(X.iloc[train], labels[train])        # 评价坐标 = 基线的训练折标准化
    lat = LatentStyles().fit(X.iloc[train], labels[train])
    Btr = base.imp.transform(X.iloc[train].to_numpy(float))
    floor = Btr.min(axis=0)
    floor_cols = np.where((Btr == floor).mean(0) >= FLOOR_SHARE_MIN)[0]
    rng = np.random.default_rng(SEED + fold)
    centered = {m: ([], []) for m in ['latent_style', 'baseline_style', 'baseline_unconditional']}
    for style in STYLES:
        at = test[labels[test] == style]
        testz = base.transform(X.iloc[at])
        trainz = base.Z[labels[train] == style]
        Bte = base.imp.transform(X.iloc[at].to_numpy(float))
        noise = rng.normal(size=(NGEN, p))
        realnn = cdist(testz, trainz).min(1).mean() / np.sqrt(p)
        for method, B in [('latent_style', lat.sample(style, noise)),
                          ('baseline_style', base.sample(style, noise)),
                          ('baseline_unconditional', base.sample('__all__', noise))]:
            Z = base.transform(B)
            dnn = cdist(Z, trainz).min(1).mean() / np.sqrt(p)
            syn_floor = (B[:, floor_cols] == floor[floor_cols]).mean(0)
            real_floor = (Bte[:, floor_cols] == floor[floor_cols]).mean(0)
            j4 = fields.index('X4VG')
            rows.append({
                'fold': fold, 'style': style, 'method': method, 'n_test_style': len(at),
                'mean_coordinate_MAE_standardized': float(abs(Z.mean(0) - testz.mean(0)).mean()),
                'median_coordinate_MAE_standardized': float(np.median(abs(Z.mean(0) -
testz.mean(0)))),
                'mean_coordinate_Wasserstein_standardized': float(np.mean(
                    [wasserstein_distance(Z[:, j], testz[:, j]) for j in range(p)])),
                'synthetic_to_real_NN_ratio': float(dnn / realnn),
                'floor_fraction_MAE_floor_columns': float(abs(syn_floor - real_floor).mean()),
                'n_floor_columns': len(floor_cols),
                'synthetic_4VG_floor_fraction': float((B[:, j4] == floor[j4]).mean()),
                'holdout_4VG_floor_fraction': float((Bte[:, j4] == floor[j4]).mean()),
            })
            centered[method][0].append(Z - Z.mean(0))
            centered[method][1].append(testz - testz.mean(0))
    for method, (aa, bb) in centered.items():
        Cg, vg = corr(np.vstack(aa))
        Cr, vr = corr(np.vstack(bb))
        ids = np.where(vg & vr)[0]
        i, j = np.triu_indices(len(ids), 1)
        corr_rows.append({'fold': fold, 'method': method, 'pooled_within_style_correlation_RMSE':
float(
            np.sqrt(np.mean((Cg[np.ix_(ids, ids)][i, j] - Cr[np.ix_(ids, ids)][i, j]) ** 2)))})
    print(f'generator fold {fold + 1}/3 done', flush=True)

gm = pd.DataFrame(rows)
gc = pd.DataFrame(corr_rows)
gm.to_csv(OUT / 'generator_holdout_metrics_paired.csv', index=False, encoding='utf-8-sig')
gc.to_csv(OUT / 'generator_holdout_correlation_paired.csv', index=False, encoding='utf-8-sig')

# 复现核对：本脚本重跑的基线应与首版保存的数值一致
saved = pd.read_csv(BASE / 'generator_holdout_metrics.csv', encoding='utf-8-sig')
saved = saved[saved.method == 'style_conditional'].set_index(['fold', 'style'])
mine = gm[gm.method == 'baseline_style'].set_index(['fold', 'style'])
baseline_reproduced = float((saved.mean_coordinate_MAE_standardized -
mine.mean_coordinate_MAE_standardized).abs().max())
```

```python
# ---------------------------------------------------------------- 最终生成（全数据拟合）
fit = LatentStyles().fit(X, labels)
rng = np.random.default_rng(SEED + 100)
gen, desc = [], []
Bfull = fit.imp.transform(X.to_numpy(float))
for style in STYLES:
    B = fit.sample(style, rng.normal(size=(NGEN, p)))
    df = pd.DataFrame(B, columns=fields)
    df.insert(0, 'style', style)
    df.insert(0, 'candidate_id', [f'{style}_{i + 1:03}' for i in range(NGEN)])
    gen.append(df)
    real = fit.Z[labels == style]
    syn = fit.G.transform(B)
    nn = cdist(syn, real).min(1) / np.sqrt(p)
    m = fit.models[style]
    j4 = fields.index('X4VG')
    desc.append({
        'style': style, 'n_real': int((labels == style).sum()), 'n_generated': NGEN,
        'LedoitWolf_shrinkage': float(m.shrinkage_),
        'offdiag_share_of_correlation_Frobenius': offdiag_share(m.covariance_),
        'exact_training_copy_count': int((nn < 1e-12).sum()),
        'real_4VG_floor_fraction': float((Bfull[labels == style, j4] == Bfull[:, j4].min()).mean()),
        'synthetic_4VG_floor_fraction': float((B[:, j4] == Bfull[:, j4].min()).mean()),
    })
gen = pd.concat(gen, ignore_index=True)
gen.to_csv(OUT / 'generated_latent_231_coordinates.csv', index=False, encoding='utf-8-sig')
gdesc = pd.DataFrame(desc)
gdesc.to_csv(OUT / 'generator_final_descriptive.csv', index=False, encoding='utf-8-sig')

# ---------------------------------------------------------------- 迁移：共同空间高斯 OT + 投影
def sqrtm_psd(S):
    v, Q = np.linalg.eigh(S)
    return (Q * np.sqrt(np.maximum(v, 0))) @ Q.T


src_mask, tgt_mask = labels == 'Pils/Lager', labels == 'Tripel'
src_m, tgt_m = fit.models['Pils/Lager'], fit.models['Tripel']
mu_s, mu_t = src_m.location_, tgt_m.location_
Ss, St = src_m.covariance_, tgt_m.covariance_
Ss_h = sqrtm_psd(Ss)
Ss_ih = np.linalg.inv(Ss_h)
A = Ss_ih @ sqrtm_psd(Ss_h @ St @ Ss_h) @ Ss_ih
A = (A + A.T) / 2
ot_check = float(np.abs(A @ Ss @ A - St).max() / np.abs(St).max())   # 闭式映射应满足 A Σs A = Σt

real_tripel_z = fit.Z[tgt_mask]
sources = X.loc[src_mask]

# 陈化方向指数：附件8 阈值表 aging 列固定名单
th = pd.read_excel(ROOT / '8.xlsx', sheet_name='Sensory thresholds')
J_plus = [f for f in th.loc[th.aging == 'increase', 'field_name_R'] if f in fields]
J_minus = [f for f in th.loc[th.aging == 'decrease', 'field_name_R'] if f in fields]
jp, jm = [fields.index(f) for f in J_plus], [fields.index(f) for f in J_minus]


def aging_index(raw_row):
    """raw_row: 原储存尺度（含 NaN）。任一名单项缺失 -> NaN（不可完整评价）。"""
    if np.isnan(raw_row[jp + jm]).any():
        return np.nan
    F = fit.G.cdf(raw_row[None, :])[0]
    return 0.5 * (F[jp].mean() + (1 - F[jm]).mean())


def evaluate(beer, source_raw, cand_raw, z0, alpha, method):
    """cand_raw: 投影后的候选（原储存尺度，无缺失）。距离在共同正态分数空间中计算。"""
```

```
    zc = fit.G.transform(cand_raw[None, :])[0]
    d0c = np.linalg.norm(z0 - mu_t) / np.sqrt(p)
    d1c = np.linalg.norm(zc - mu_t) / np.sqrt(p)
    d0n = cdist(z0[None], real_tripel_z).min() / np.sqrt(p)
    d1n = cdist(zc[None], real_tripel_z).min() / np.sqrt(p)
    defect_ok, evaluable, floor_kept = [], [], []
    for name, f in DEFECTS.items():
        j = fields.index(f)
        a, b = source_raw[j], cand_raw[j]
        if np.isnan(a):
            evaluable.append(False)
            continue
        evaluable.append(True)
        defect_ok.append(bool(b <= a))
        if a == Bfull[:, j].min():
            floor_kept.append(bool(b == a))
    a0, a1 = aging_index(source_raw), aging_index(cand_raw)
    c1 = all(evaluable) and all(defect_ok) and all(floor_kept)
    c2 = 1 - d1c / d0c >= CENTER_GAIN
    c3 = 1 - d1n / d0n >= NN_GAIN
    c4_eval = not (np.isnan(a0) or np.isnan(a1))
    c4 = c4_eval and a1 <= a0 + 1e-12
    complete = all(evaluable) and c4_eval
    return {
        'method': method, 'beer': int(beer), 'alpha': alpha,
        'center_dist_source': d0c, 'center_dist': d1c, 'center_reduction': 1 - d1c / d0c,
        'nn_dist_source': d0n, 'nn_dist': d1n, 'nn_reduction': 1 - d1n / d0n,
        'source_change_RMS_latent': float(np.sqrt(np.mean((zc - z0) ** 2))),
        'four_defects_evaluable': all(evaluable), 'observed_defects_nonincrease': all(defect_ok),
        'floor_states_kept': all(floor_kept), 'n_floor_defects': len(floor_kept),
        'aging_index_source': a0, 'aging_index_candidate': a1, 'aging_evaluable': c4_eval,
        'C1_defects': c1, 'C2_center': bool(c2), 'C3_nn': bool(c3), 'C4_aging': bool(c4),
        'complete_evaluation': complete,
        'all_criteria': bool(c1 and c2 and c3 and c4),
    }


fwd = joblib.load(BASE / 'sensory_forward_model.joblib')
assert fwd['fields'] == fields
stale_j = fwd['targets'].index('stale_hops')

mig_rows, mig_coords = [], []
for beer, srow in sources.iterrows():
    source_raw = srow.to_numpy(float)
    z0 = fit.transform(srow.to_frame().T)[0]
    Tz = mu_t + A @ (z0 - mu_s)
    for alpha in ALPHAS:
        za = z0 + alpha * (Tz - z0)
        cand = fit.G.inverse(za[None, :])[0]                  # 已在观测范围内
        for name, f in DEFECTS.items():                        # 缺陷非增投影（源缺失则无法约束）
            j = fields.index(f)
            if not np.isnan(source_raw[j]):
                cand[j] = min(cand[j], source_raw[j])
        r = evaluate(beer, source_raw, cand, z0, alpha, 'latent_OT')
        mig_rows.append(r)
        rec = dict(zip(fields, cand))
        rec.update({'beer': int(beer), 'alpha': alpha})
        mig_coords.append(rec)

# 现有优化基线在相同标准下评价
bm = pd.read_csv(BASE / 'migration_231_coordinates.csv', encoding='utf-8-sig')
for _, r in bm.iterrows():
    beer = int(r.beer)
    source_raw = sources.loc[beer].to_numpy(float)
    z0 = fit.transform(sources.loc[[beer]])[0]
```

```
    mig_rows.append(evaluate(beer, source_raw, r[fields].to_numpy(float), z0, float(r.alpha),
'baseline_optimizer'))

mr = pd.DataFrame(mig_rows)
mc = pd.DataFrame(mig_coords)
# Stale hops 预测：仅参考展示
src_imp = fwd['imputer'].transform(sources[fields])
src_stale = dict(zip(sources.index, fwd['model'].predict(src_imp)[:, stale_j]))
cand_stale = fwd['model'].predict(fwd['imputer'].transform(mc[fields]))[:, stale_j]
mr.loc[mr.method == 'latent_OT', 'stale_hops_pred_change_reference'] = cand_stale -
mc.beer.map(src_stale).to_numpy()
mr.to_csv(OUT / 'migration_acceptance_all.csv', index=False, encoding='utf-8-sig')
mc.to_csv(OUT / 'migration_latent_OT_231_coordinates.csv', index=False, encoding='utf-8-sig')


def select(df):
    out = []
    for (method, beer), g in df[df.alpha > 0].sort_values('alpha').groupby(['method', 'beer']):
        ok = g[g.all_criteria]
        if not g.complete_evaluation.all():
            status = '不可完整验收'
        elif len(ok):
            status = '达标'
        else:
            status = '未达到预设标准'
        out.append({'method': method, 'beer': beer, 'status': status,
                    'selected_alpha': float(ok.alpha.iloc[0]) if status == '达标' else np.nan})
    return pd.DataFrame(out)


sel = select(mr)
sel.to_csv(OUT / 'migration_selected_candidates.csv', index=False, encoding='utf-8-sig')

# ---------------------------------------------------------------------- 感官画像（Q1 最终前向模型 + 冻结锚点）
anchors = pd.read_json(ROOT / 'results' / 'q1' / 'q1_frozen_anchors.json', orient='index')
support = pd.read_csv(ROOT / 'results' / 'q1' / 'q1_prediction_support_tiers.csv', encoding='utf-8-
sig', index_col=0)
targets = fwd['targets']
pred = pd.DataFrame(fwd['model'].predict(fwd['imputer'].transform(gen[fields])), columns=targets)
pred.insert(0, 'style', gen['style'])
y_real = pd.read_csv(ROOT / '4.csv').set_index('beer').loc[x.index, targets]
prof = []
for style in STYLES:
    P = pred[pred['style'] == style][targets]
    for t in targets:
        row = {'style': style, 'target': t, 'type': support.loc[t, 'type'], 'support_tier':
support.loc[t, 'support_tier'],
               'skill_flag': '' if support.loc[t, 'skill_positive'] else '未优于风格均值',
               'pred_mean_standardized': float(P[t].mean()),
               'real_style_mean_standardized': float(y_real.loc[labels == style, t].mean())}
        if t in anchors.index:
            q05, q95 = anchors.loc[t, 'q05_oof'], anchors.loc[t, 'q95_oof']
            s = np.clip(6 * (P[t] - q05) / (q95 - q05), 0, 6).round()
            row['relative_score_median'] = float(s.median())
        prof.append(row)
prof = pd.DataFrame(prof)
prof.to_csv(OUT / 'generated_sensory_profile.csv', index=False, encoding='utf-8-sig')
pred.to_csv(OUT / 'generated_sensory_standardized.csv', index=False, encoding='utf-8-sig')

# ---------------------------------------------------------------------- 汇总
agg = gm.groupby('method')[['mean_coordinate_MAE_standardized',
'median_coordinate_MAE_standardized',
                            'mean_coordinate_Wasserstein_standardized',
'synthetic_to_real_NN_ratio',
                            'floor_fraction_MAE_floor_columns']].mean()
```

```
agg['pooled_within_style_correlation_RMSE'] =
gc.groupby('method').pooled_within_style_correlation_RMSE.mean()
by_alpha = mr.groupby(['method', 'alpha'])[['center_reduction', 'nn_reduction',
'source_change_RMS_latent',
                                            'C1_defects', 'C2_center', 'C3_nn', 'C4_aging',
'all_criteria']].mean()
summary = {
    'baseline_reproduction_max_abs_diff': baseline_reproduced,
    'generator_holdout_mean': agg.round(4).to_dict('index'),
    'generator_final': gdesc.round(4).to_dict('records'),
    'ot_covariance_check_rel_err': ot_check,
    'aging_J_plus': J_plus, 'aging_J_minus': J_minus,
    'migration_by_alpha': {f'{m}|{a}': v for (m, a), v in
by_alpha.round(3).to_dict('index').items()},
    'migration_status_counts': sel.groupby(['method', 'status']).size().to_dict(),
    'migration_selected': sel.to_dict('records'),
}
summary['migration_status_counts'] = {f'{k[0]}|{k[1]}': int(v) for k, v in
summary['migration_status_counts'].items()}
json.dump(summary, open(OUT / 'summary.json', 'w', encoding='utf-8'), ensure_ascii=False, indent=1,
default=float)
print(json.dumps(summary, ensure_ascii=False, indent=1, default=float))
```

src/q3_style_generation/sensitivity_and_checks.py

```
"""Q3 补充（结果出现后的敏感性分析与核查，不替换冻结主结果）。

1. 判据2 替换为目标风格经验容许域：d(x'', mu_T) <= q95{ d(x_i, mu_T,-i) }（28 款真实Tripel 留一中心距离），其余
三条不变。
2. 迁移群体离散度：防止"全部收缩到中心"被当作风格保真。
3. 两法共同达标样本上、各自首次达标点的配对改变量（描述性；两法 alpha 网格不同）。
4. 首版基线拟合参数核对 + 运行环境记录。
AI assistance: Claude (Claude Opus 5.5), Anthropic, 2026-10-26. 用于代码实现，详见支撑材料《AI 工具使用详情
说明》。
"""
import json
import platform
from pathlib import Path

import joblib
import numpy as np
import pandas as pd
import scipy
import sklearn
from scipy.spatial.distance import pdist
from threadpoolctl import threadpool_info

import latent_gaussian_ot as L   # 复用共同映射与模型（导入即重跑主脚本，结果确定）

OUT = L.OUT
fit, X, labels, p = L.fit, L.X, L.labels, L.p
mr = pd.read_csv(OUT / 'migration_acceptance_all.csv', encoding='utf-8-sig')
mc = pd.read_csv(OUT / 'migration_latent_OT_231_coordinates.csv', encoding='utf-8-sig')
bm = pd.read_csv(L.BASE / 'migration_231_coordinates.csv', encoding='utf-8-sig')

# ---------- 1. 留一经验容许域 ----------
Zt = fit.Z[labels == 'Tripel']
loo = np.array([np.linalg.norm(Zt[i] - np.delete(Zt, i, 0).mean(0)) for i in range(len(Zt))]) /
np.sqrt(p)
r95 = float(np.quantile(loo, 0.95))
mr['C2_envelope'] = mr.center_dist <= r95
mr['all_criteria_envelope'] = mr.C1_defects & mr.C2_envelope & mr.C3_nn & mr.C4_aging


def select(df, flag):
    out = []
    for (method, beer), g in df[df.alpha > 0].sort_values('alpha').groupby(['method', 'beer']):
```

```
        ok = g[g[flag]]
        status = '不可完整验收' if not g.complete_evaluation.all() else ('达标' if len(ok) else '未达到
预设标准')
        out.append({'method': method, 'beer': beer, 'status': status,
                    'selected_alpha': float(ok.alpha.iloc[0]) if status == '达标' else np.nan})
    return pd.DataFrame(out)


frozen = select(mr, 'all_criteria')
envelope = select(mr, 'all_criteria_envelope')
comp = frozen.merge(envelope, on=['method', 'beer'], suffixes=('_frozen', '_envelope'))
comp.to_csv(OUT / 'sensitivity_envelope_selection.csv', index=False, encoding='utf-8-sig')
mr.to_csv(OUT / 'migration_acceptance_with_envelope.csv', index=False, encoding='utf-8-sig')

# ---------- 2. 迁移群体离散度（共同正态分数空间） ----------
def disp(Z):
    return {'mean_dist_to_Tripel_center': float(np.linalg.norm(Z - fit.models['Tripel'].location_,
axis=1).mean() / np.sqrt(p)),
            'mean_pairwise_dist': float(pdist(Z).mean() / np.sqrt(p))}


rows = [dict(group='real_Tripel(28)', **disp(Zt)),
        dict(group='source_Pils/Lager(11)', **disp(fit.Z[labels == 'Pils/Lager']))]
for alpha in sorted(mc.alpha.unique()):
    rows.append(dict(group=f'latent_OT alpha={alpha}', **disp(fit.G.transform(mc[mc.alpha == alpha]
[L.fields].to_numpy(float)))))
for alpha in sorted(bm.alpha.unique()):
    rows.append(dict(group=f'baseline_optimizer alpha={alpha}', **disp(fit.G.transform(bm[bm.alpha
== alpha][L.fields].to_numpy(float)))))
dispersion = pd.DataFrame(rows)
dispersion.to_csv(OUT / 'migration_dispersion.csv', index=False, encoding='utf-8-sig')

# ---------- 3. 配对改变量（冻结判据下两法共同达标样本） ----------
both = frozen.pivot(index='beer', columns='method', values='selected_alpha').dropna()
chg = {}
for method in ['latent_OT', 'baseline_optimizer']:
    sub = mr[mr.method == method].set_index(['beer', 'alpha']).source_change_RMS_latent
    chg[method] = [float(sub.loc[(b, both.loc[b, method])]) for b in both.index]
paired = pd.DataFrame({'beer': both.index, 'alpha_OT': both.latent_OT.values, 'alpha_baseline':
both.baseline_optimizer.values,
                       'change_OT': chg['latent_OT'], 'change_baseline': chg['baseline_optimizer']})
paired.to_csv(OUT / 'paired_change_at_first_pass.csv', index=False, encoding='utf-8-sig')

# ---------- 4. 首版基线参数核对 + 环境 ----------
saved = joblib.load(L.BASE / 'style_gaussians.joblib')
refit = L.BaselineStyles().fit(X, labels)
param = {
    'scaler_mean_max_abs_diff': float(np.abs(saved['scaler'].mean_ - refit.scaler.mean_).max()),
    'scaler_scale_max_abs_diff': float(np.abs(saved['scaler'].scale_ - refit.scaler.scale_).max()),
    'style_location_max_abs_diff': float(max(np.abs(saved['style_models'][k].location_ -
refit.models[k].location_).max()
                                             for k in refit.models)),
    'style_covariance_max_abs_diff': float(max(np.abs(saved['style_models'][k].covariance_ -
refit.models[k].covariance_).max()
                                               for k in refit.models)),
}
env = {'python': platform.python_version(), 'numpy': np.__version__, 'pandas': pd.__version__,
'scipy': scipy.__version__,
       'scikit-learn': sklearn.__version__, 'joblib': joblib.__version__,
       'blas': [{k: i.get(k) for k in ('internal_api', 'version', 'threading_layer', 'num_threads')}
for i in threadpool_info()]}
json.dump(env, open(OUT / 'environment_versions.json', 'w', encoding='utf-8'), indent=1)

summary = {
    'loo_center_dist_real_Tripel': {'mean': float(loo.mean()), 'min': float(loo.min()), 'max':
```

```
float(loo.max()), 'q95_radius': r95},
    'status_frozen': {f'{m}|{st}': int(v) for (m, st), v in frozen.groupby(['method',
'status']).size().items()},
    'status_envelope': {f'{m}|{st}': int(v) for (m, st), v in envelope.groupby(['method',
'status']).size().items()},
    'selected_alpha_envelope': {f'{m}|{a}': int(v) for (m, a), v in

envelope.dropna().groupby('method').selected_alpha.value_counts().items()},
    'paired_first_pass_change': {'n': len(paired), 'OT_mean': float(paired.change_OT.mean()),
                                 'baseline_mean': float(paired.change_baseline.mean()),
                                 'OT_smaller_count': int((paired.change_OT <
paired.change_baseline).sum())},
    'dispersion': dispersion.round(3).to_dict('records'),
    'baseline_parameter_check': param,
}
json.dump(summary, open(OUT / 'sensitivity_summary.json', 'w', encoding='utf-8'),
ensure_ascii=False, indent=1)
print(json.dumps(summary, ensure_ascii=False, indent=1))
print(comp.to_string())
```

paper/build_paper.py

```
"""把 paper/abstract.md + paper/body.md 套入大赛 Word 模板，生成 paper/论文.docx（及 PDF 预览）。

流程：组装附录（结果表 + 全部源程序）→ pandoc（以模板为 reference-doc）生成正文 →
把模板的封面/摘要/目录两节注入正文前部，正文节使用模板的页脚（页码从 1 开始、无页眉）。
AI assistance: Claude (Claude Opus 5.5), Anthropic, 2026-10-26. 用于代码实现，详见支撑材料《AI 工具使用详情
说明》。
"""
import re
import shutil
import subprocess
import sys
import zipfile
from pathlib import Path
from xml.sax.saxutils import escape

import pandas as pd

P = Path(__file__).resolve().parent
ROOT = P.parent
BUILD = P / 'build'
TITLE = '基于折外校准、风格敏感性分析与潜在高斯最优传输的啤酒风味建模'
KEYWORDS = '感官评分；Extra Trees；置换重要性；风格内置换；经验 CDF；Ledoit–Wolf 收缩；高斯最优传输'
TEAM = 'MCB260XXXX'
TRACK = 'A'


def md_table(df, widths=None):
    cols = list(df.columns)
    widths = widths or [1] * len(cols)
    lines = ['| ' + ' | '.join(cols) + ' |', '|' + '|'.join(':' + '-' * (6 * w) for w in widths) +
'|']
    for _, r in df.iterrows():
        lines.append('| ' + ' | '.join(str(v).replace('|', '/') for v in r.values) + ' |')
    return '\n'.join(lines)


def appendix_tables():
    t = pd.read_csv(ROOT / 'results/q2/q2_v31_target_summary.csv', encoding='utf-8-sig',
index_col=0)
    a1 = pd.DataFrame({'感官维度': t.index, 'R²': t.R2_mean_oof.round(3), '支持度': t.support_tier,
                       '解释稳定性': t.interpretation_stability, 'Top-3 候选': t.top3.str.replace(' |
', '；')})
    w = pd.read_csv(ROOT / 'results/q2/q2_within_style_permutation_by_field.csv', encoding='utf-8-
sig')
    w = w[w.n_targets >= 3]
```

```
    a2 = pd.DataFrame({'化合物': w.field, '出现维度数': w.n_targets, '保留比例中位数':
w.median_retention.round(3),
                       '风格内为正比例': (w.share_within_positive * 100).round(1).astype(str) + '%',
                       '风格内仍在 Top-10': (w.share_within_top10 * 100).round(1).astype(str) + '%'})
    s = pd.read_csv(ROOT / 'results/q3/sensitivity_envelope_selection.csv', encoding='utf-8-sig')
    s['method'] = s.method.map({'latent_OT': '共同空间 OT', 'baseline_optimizer': '首版优化基线'})
    a3 = s.rename(columns={'method': '方法', 'beer': '源样本', 'status_frozen': '冻结判据',
                           'selected_alpha_frozen': '首次达标 α', 'status_envelope': '容许域判据',
                           'selected_alpha_envelope': '容许域首次达标 α'}).fillna('—')
    return ('表 A1  50 维 Top-3 候选与两维分档\n\n' + md_table(a1, [3, 1, 1, 2, 9]) +
            '\n\n表 A2  风格内置换按化合物汇总（出现于至少 3 个维度的 Top-3）\n\n' + md_table(a2) +
            '\n\n表 A3  迁移逐样本验收（冻结判据与事后容许域判据；容许域 P95 半径 1.183）\n\n' + md_table(a3))


def code_listing():
    files = sorted((ROOT / 'analysis').rglob('*.py')) + sorted((ROOT / 'src').rglob('*.py')) + [P /
'build_paper.py']
    out = []
    for f in files:
        rel = f.relative_to(ROOT).as_posix()
        out.append(f'**{rel}**\n\n```python\n{f.read_text(encoding="utf-8").rstrip()}\n```\n')
    return '\n'.join(out)


def build_markdown():
    body = (P / 'body.md').read_text(encoding='utf-8')
    body = re.sub(r'【附表 A1.*?】', lambda m: appendix_tables(), body, flags=re.S)
    body = body.replace('【全部源程序清单由排版脚本自动附于此处。】', '### 源程序清单\n\n' + code_listing())
    (BUILD / 'paper_full.md').write_text(body, encoding='utf-8')


def pandoc():
    subprocess.run(['pandoc', str(BUILD / 'paper_full.md'), '-o', str(BUILD / 'body.docx'),
                    '--reference-doc', str(P / 'template.docx'), '--resource-path', str(P),
                    '-f', 'markdown+pipe_tables+tex_math_dollars+implicit_figures'], check=True)


# ---------------------------------------------------------------------- 样式（模板缺少的正文样式由此补充）
FONT = '<w:rFonts w:ascii="Times New Roman" w:hAnsi="Times New Roman" w:eastAsia="宋体" w:cs="Times
New Roman"/>'
HEI = '<w:rFonts w:ascii="Times New Roman" w:hAnsi="Times New Roman" w:eastAsia="黑体" w:cs="Times
New Roman"/>'
NEW_STYLES = {
    'Heading2': f'<w:style w:type="paragraph" w:styleId="Heading2"><w:name w:val="heading
2"/><w:basedOn w:val="1"/><w:next
w:val="BodyText"/><w:qFormat/><w:pPr><w:keepNext/><w:keepLines/><w:spacing w:before="200"
w:after="80" w:line="300" w:lineRule="auto"/><w:jc w:val="left"/><w:outlineLvl
w:val="1"/></w:pPr><w:rPr>{HEI}<w:b/><w:sz w:val="28"/><w:szCs w:val="28"/></w:rPr></w:style>',
    'Heading3': f'<w:style w:type="paragraph" w:styleId="Heading3"><w:name w:val="heading
3"/><w:basedOn w:val="1"/><w:next w:val="BodyText"/><w:qFormat/><w:pPr><w:keepNext/><w:spacing
w:before="120" w:after="60"/><w:outlineLvl w:val="2"/></w:pPr><w:rPr>{HEI}<w:b/><w:sz
w:val="24"/><w:szCs w:val="24"/></w:rPr></w:style>',
    'BodyText': f'<w:style w:type="paragraph" w:styleId="BodyText"><w:name w:val="Body
Text"/><w:basedOn w:val="1"/><w:qFormat/><w:pPr><w:spacing w:before="0" w:after="0" w:line="312"
w:lineRule="auto"/><w:ind w:firstLine="480"/><w:jc w:val="both"/></w:pPr><w:rPr>{FONT}<w:sz
w:val="24"/><w:szCs w:val="24"/></w:rPr></w:style>',
    'FirstParagraph': '<w:style w:type="paragraph" w:styleId="FirstParagraph"><w:name w:val="First
Paragraph"/><w:basedOn w:val="BodyText"/><w:next w:val="BodyText"/><w:qFormat/></w:style>',
    'Compact': f'<w:style w:type="paragraph" w:styleId="Compact"><w:name w:val="Compact"/><w:basedOn
w:val="1"/><w:qFormat/><w:pPr><w:spacing w:before="20" w:after="20"/><w:jc
w:val="center"/></w:pPr><w:rPr>{FONT}<w:sz w:val="20"/><w:szCs w:val="20"/></w:rPr></w:style>',
    'ImageCaption': f'<w:style w:type="paragraph" w:styleId="ImageCaption"><w:name w:val="Image
Caption"/><w:basedOn w:val="1"/><w:qFormat/><w:pPr><w:spacing w:before="40" w:after="160"/><w:jc
w:val="center"/></w:pPr><w:rPr>{HEI}<w:sz w:val="21"/><w:szCs w:val="21"/></w:rPr></w:style>',
    'TableCaption': f'<w:style w:type="paragraph" w:styleId="TableCaption"><w:name w:val="Table
```

```
Caption"/><w:basedOn w:val="1"/><w:qFormat/><w:pPr><w:keepNext/><w:spacing w:before="160"
w:after="60"/><w:jc w:val="center"/></w:pPr><w:rPr>{HEI}<w:sz w:val="21"/><w:szCs
w:val="21"/></w:rPr></w:style>',
    'CaptionedFigure': '<w:style w:type="paragraph" w:styleId="CaptionedFigure"><w:name
w:val="Captioned Figure"/><w:basedOn w:val="1"/><w:pPr><w:keepNext/><w:spacing w:before="120"/><w:jc
w:val="center"/></w:pPr></w:style>',
    'Figure': '<w:style w:type="paragraph" w:styleId="Figure"><w:name w:val="Figure"/><w:basedOn
w:val="1"/><w:pPr><w:jc w:val="center"/></w:pPr></w:style>',
    'Table': '<w:style w:type="table" w:styleId="Table"><w:name w:val="Table"/><w:basedOn
w:val="15"/><w:tblPr><w:jc w:val="center"/><w:tblBorders><w:top w:val="single" w:sz="12" w:space="0"
w:color="000000"/><w:bottom w:val="single" w:sz="12" w:space="0"
w:color="000000"/></w:tblBorders><w:tblCellMar><w:left w:w="80" w:type="dxa"/><w:right w:w="80"
w:type="dxa"/></w:tblCellMar></w:tblPr><w:tblStylePr
w:type="firstRow"><w:rPr><w:b/></w:rPr><w:tcPr><w:tcBorders><w:bottom w:val="single" w:sz="6"
w:space="0" w:color="000000"/></w:tcBorders></w:tcPr></w:tblStylePr></w:style>',
    'SourceCode': '<w:style w:type="paragraph" w:customStyle="1" w:styleId="SourceCode"><w:name
w:val="Source Code"/><w:basedOn w:val="1"/><w:link w:val="VerbatimChar"/><w:pPr><w:wordWrap
w:val="0"/><w:spacing w:before="0" w:after="0" w:line="220" w:lineRule="exact"/><w:jc
w:val="left"/></w:pPr><w:rPr><w:rFonts w:ascii="Consolas" w:hAnsi="Consolas" w:eastAsia="宋体"/><w:sz
w:val="15"/><w:szCs w:val="15"/></w:rPr></w:style>',
}


def fix_styles(styles):
    for sid, xml in NEW_STYLES.items():
        styles = re.sub(rf'<w:style [^>]*w:styleId="{sid}".*?</w:style>', '', styles, flags=re.S)
        styles = styles.replace('</w:styles>', xml + '</w:styles>')
    # 一级标题：居中、三号黑体，压缩段距
    styles = re.sub(r'(<w:style
[^>]*w:styleId="2".*?<w:pPr>).*?(</w:pPr><w:rPr>).*?(</w:rPr></w:style>)',
                    lambda m: m.group(1) + '<w:keepNext/><w:keepLines/><w:spacing w:before="360"
w:after="200" w:line="300" w:lineRule="auto"/><w:jc w:val="center"/><w:outlineLvl w:val="0"/>'
                    + m.group(2) + HEI + '<w:b/><w:bCs/><w:kern w:val="44"/><w:sz
w:val="32"/><w:szCs w:val="32"/>' + m.group(3),
                    styles, count=1, flags=re.S)
    return styles


def fix_body(doc):
    # 表格属性顺序：jc 必须在 tblLook 之前
    doc = re.sub(r'(<w:tblLook [^>]*/>)<w:jc w:val="start" />', r'<w:jc w:val="center"/>\1', doc)
    doc = re.sub(r'(<w:tblLook [^>]*/>)<w:jc w:val="start"/>', r'<w:jc w:val="center"/>\1', doc)
    # “表X …”段落改用表题样式
    doc = re.sub(r'<w:pStyle w:val="(?:BodyText|FirstParagraph)" ?/>((?:(?!</w:p>).)*?<w:t[^>]*>表(?:
\d|A\d))',
                 r'<w:pStyle w:val="TableCaption"/>\1', doc, flags=re.S)
    return doc


# ---------------------------------------------------------------- docx 拼装
def para_text(pPr, text, rPr=''):
    return f'<w:p>{pPr}<w:r>{rPr}<w:t xml:space="preserve">{escape(text)}</w:t></w:r></w:p>'


def assemble():
    tpl = zipfile.ZipFile(P / 'template.docx')
    tdoc = tpl.read('word/document.xml').decode('utf-8')
    body_start = tdoc.index('<w:body>') + len('<w:body>')
    secs = [m.end() for m in re.finditer(r'</w:sectPr></w:pPr>.*?</w:p>', tdoc)]
    front = tdoc[body_start:secs[1]]                        # 封面/摘要节+目录节
    final_sect = re.search(r'<w:sectPr>(?:(?!<w:sectPr>).)*</w:sectPr></w:body>', tdoc,
re.S).group(0)[:-len('</w:body>')]

    # 赛道、题目
    front = re.sub(r' (A/B) ', TRACK, front)
```

```
    front = front.replace('论文题目', escape(TITLE))
    # 摘要正文：复制占位段落的格式，逐段替换
    m = re.search(r'<w:p\b[^>]*>(?:(?!<w:p\b).)*?（摘要正文）.*?</w:p>', front, re.S)
    ab_p = m.group(0)
    pPr = re.search(r'<w:pPr>.*?</w:pPr>', ab_p, re.S)
    pPr = pPr.group(0) if pPr else ''
    rPr = re.search(r'<w:r>(<w:rPr>.*?</w:rPr>)', ab_p, re.S)
    rPr = rPr.group(1) if rPr else ''
    paras = [x.strip() for x in (P / 'abstract.md').read_text(encoding='utf-8').split('\n\n') if
x.strip()]
    if '<w:ind' not in pPr:
        ind = '<w:ind w:firstLineChars="200" w:firstLine="480"/>'
        ind = '<w:spacing w:line="276" w:lineRule="auto"/>' + ind
        pPr = (re.sub(r'(<w:jc |<w:rPr>|</w:pPr>)', ind + r'\1', pPr, count=1)) if pPr else \
            '<w:pPr><w:ind w:firstLineChars="200" w:firstLine="480"/><w:jc w:val="both"/></w:pPr>'
    front = front.replace(ab_p, ''.join(para_text(pPr, x, rPr) for x in paras))
    # 摘要与关键词之间模板留有多个空段落，删去以保证摘要页不溢出
    k = front.index('<w:t>关键词：</w:t>')
    kp = front.rfind('<w:p ', 0, k)
    last_ab = front.index('</w:p>', front.rfind(escape(paras[-1][-12:]), 0, kp))
    seg = front[:kp]
    head, tail = seg[:last_ab], seg[last_ab:]
    tail = re.sub(r'<w:p\b[^>]*>(?:<w:pPr>(?:(?!</w:pPr>).)*</w:pPr>)?</w:p>|<w:p\b[^>]*/>', '',
tail, flags=re.S)
    front = head + tail + front[kp:]
    # 关键词
    front = re.sub(r'(<w:t>关键词：</w:t></w:r>)', r'\1<w:r><w:rPr><w:sz w:val="24"/><w:szCs
w:val="24"/></w:rPr><w:t xml:space="preserve">'
                   + escape(KEYWORDS) + '</w:t></w:r>', front, count=1)
    # 目录：替换示例条目为 TOC 域
    toc_items = re.findall(r'<w:p\b[^>]*><w:pPr><w:pStyle w:val="(?:10|13|5)"/>.*?</w:p>', front,
re.S)
    toc_field = ('<w:p><w:pPr><w:pStyle w:val="10"/><w:tabs><w:tab w:val="right" w:leader="dot"
w:pos="9070"/></w:tabs></w:pPr>'
                 '<w:r><w:fldChar w:fldCharType="begin" w:dirty="true"/></w:r><w:r><w:instrText
xml:space="preserve"> TOC \\o "1-2" \\h \\z \\u </w:instrText></w:r>'
                 '<w:r><w:fldChar w:fldCharType="separate"/></w:r><w:r><w:t>（目录：在 Word 中右键“更新
域”生成）</w:t></w:r>'
                 '<w:r><w:fldChar w:fldCharType="end"/></w:r></w:p>')
    front = front.replace(toc_items[0], toc_field, 1)
    for it in toc_items[1:]:
        front = front.replace(it, '', 1)

    # 读取 pandoc 正文
    src = zipfile.ZipFile(BUILD / 'body.docx')
    doc = fix_body(src.read('word/document.xml').decode('utf-8'))
    rels = src.read('word/_rels/document.xml.rels').decode('utf-8')
    ctypes = src.read('[Content_Types].xml').decode('utf-8')

    # 模板页眉页脚并入，重新编号
    trels = tpl.read('word/_rels/document.xml.rels').decode('utf-8')
    idmap = {}
    for rid, target in re.findall(r'Id="(rId\d+)"[^>]*Target="((?:header|footer)\d\.xml)"', trels):
        new = f'rIdTpl{target.split(".")[0]}'
        idmap[rid] = new
        kind = 'header' if target.startswith('header') else 'footer'
        rels = rels.replace('</Relationships>', f'<Relationship Id="{new}"
Type="http://schemas.openxmlformats.org/officeDocument/2006/relationships/{kind}"
Target="tpl_{target}"/></Relationships>')
        ct = f'application/vnd.openxmlformats-officedocument.wordprocessingml.{kind}+xml'
        ctypes = ctypes.replace('</Types>', f'<Override PartName="/word/tpl_{target}"
ContentType="{ct}"/></Types>')
    for old, new in idmap.items():
        front = front.replace(f'r:id="{old}"', f'r:id="{new}"')
        final_sect = final_sect.replace(f'r:id="{old}"', f'r:id="{new}"')
```

```
    # 模板的"行网格"会把 1.3 倍行距吸附成两行网格，改为不吸附
    grid = re.compile(r'<w:docGrid [^>]*/>')
    front = grid.sub('<w:docGrid w:linePitch="312"/>', front)
    final_sect = grid.sub('<w:docGrid w:linePitch="312"/>', final_sect)
    b0 = doc.index('<w:body>') + len('<w:body>')
    doc = doc[:b0] + front + doc[b0:]
    doc = re.sub(r'<w:sectPr\b(?:(?!<w:sectPr).)*</w:sectPr>\s*</w:body>', final_sect + '</w:body>',
doc, flags=re.S)
    # 模板根元素命名空间（front 中含 w14/v/o 等）
    ns = {}
    for root in (re.search(r'<w:document[^>]*>', doc).group(0), re.search(r'<w:document[^>]*>',
tdoc).group(0)):
        for k, v in re.findall(r'(xmlns:\w+)="([^"]*)"', root):
            ns.setdefault(k, v)
    ign = re.search(r'mc:Ignorable="([^"]*)"', re.search(r'<w:document[^>]*>', tdoc).group(0))
    root = '<w:document ' + ' '.join(f'{k}="{v}"' for k, v in ns.items()) + (f'
mc:Ignorable="{ign.group(1)}"' if ign else '') + '>'
    doc = re.sub(r'<w:document[^>]*>', root, doc, count=1)

    out = P / '论文.docx'
    with zipfile.ZipFile(out, 'w', zipfile.ZIP_DEFLATED) as z:
        for item in src.infolist():
            data = src.read(item.filename)
            if item.filename == 'word/document.xml':
                data = doc.encode('utf-8')
            elif item.filename == 'word/_rels/document.xml.rels':
                data = rels.encode('utf-8')
            elif item.filename == 'word/styles.xml':
                data = fix_styles(data.decode('utf-8')).encode('utf-8')
            elif item.filename == '[Content_Types].xml':
                data = ctypes.encode('utf-8')
            z.writestr(item, data)
        for target in set(re.findall(r'Target="((?:header|footer)\d\.xml)"', trels)):
            z.writestr(f'word/tpl_{target}', tpl.read(f'word/{target}'))
    return out


if __name__ == '__main__':
    BUILD.mkdir(exist_ok=True)
    build_markdown()
    pandoc()
    out = assemble()
    print(out)
```